算法分析与设计
实验教程

黄海于 杜圣东 喻琇瑛 编著

西南交通大学信息科学与技术学院

前　言

算法是求解问题的步骤和方法，是计算机程序的重要组成部分。高质量的算法可以极大地降低运算的复杂度，提高程序的运行效率。在信息技术高速发展的今天，虽然计算机硬件的性能得到了很大的提升，但人们对速度的追求却永无止境，如何提升算法的性能是一个永恒的课题。

本实验教程通过一系列的实例详细介绍了如何描述一个算法，分析算法的时间复杂度，理解算法的执行过程，比较不同算法求解同一问题时的效率差异以及如何利用所学算法解决实际应用问题。并通过相关的实验，强化学生对算法的表达、分析、理解以及应用能力。

本实验教程共分为 8 章。第 1 章算法描述及程序实现方法，介绍了如何描述一个算法以及常见算法的程序实现方法；第 2 章算法效率分析与比较，通过实例介绍了影响算法效率的主要因素以及算法复杂度的分析方法；第 3～8 章分别是穷举法、分治法、动态规划法、贪心法、回溯法以及分支限界法，通过典型实例介绍了这些算法的基本思想，求解过程，与其它算法的时间效率比较，复杂度分析过程以及典型应用等。

由于编者水平有限，且时间仓促，编写过程中错误在所难免，不足之处敬请批评指正。

2021.2.20

目　　录

实验 1 算法描述及程序实现方法 1
1.1 算法描述方法 1
1.2 算法执行过程分析 3
1.3 算法的程序实现 8
实验 2 算法效率分析与比较 9
2.1 数据输入规模对算法执行时间的影响 9
2.2 算法对时间复杂度的影响 14
2.3 算法时间效率分析 19
实验 3 穷举法设计与实验 26
3.1 穷举法的求解过程 26
3.2 穷举法时间复杂度分析 36
3.3 穷举法的应用 37
实验 4 分治法设计与实验 40
4.1 分治算法的求解过程 40
4.2 分治算法设计实例及时间复杂度分析 42
4.3 分治算法在实际问题中的应用 46
实验 5 动态规划算法设计与实验 50
5.1 动态规划算法的求解过程 50
5.2 动态规划算法时间复杂度分析比较 54
5.3 动态规划算法在实际中的应用 55
实验 6 贪心算法设计与实验 63
6.1 贪心算法的求解过程 63
6.2 贪心算法设计实例分析 65
6.3 贪心算法在实际问题中的应用 70
实验 7 回溯法设计与实验 73
7.1 回溯法求解过程分析 73
7.2 回溯法时间复杂度分析比较 84
7.3 回溯法在实际中的应用 89
实验 8 分支限界法设计与实验 97
8.1 分支限界法求解过程分析 97
8.2 分支限界法时间复杂度分析比较 112
8.3 分支限界法的应用 113

实验 1 算法描述及程序实现方法

实验目的：

1. 掌握算法的描述方法。
2. 理解算法的执行过程。
3. 根据给定的算法编写相应的程序。

1.1 算法描述方法

算法的描述方法主要有**自然语言描述法，流程图描述法以及伪代码描述法**。无论何种描述方法，都必须明确算法是对数据的处理方法，是将一种形式的数据转换成另一种形式的数据的过程描述。因此必须包含:

（1） 输入和输出，定义相应的输入和输出变量。

（2） 算法的执行过程，也就是如何将输入变量通过一系列的处理过程转换成输出变量。

例1.1 请用自然语言描述法描述求两个数的最大公约数的算法

解：该问题用自然语言描述如下:

输入：两个整数 m,n;

输出：m,n 的最大公约数 r.

算法求解过程如下:

（1） 变量 r=m%n;

（2） 如果 r=0,则 r=n,返回 r;否则执行步骤（3）;

（3） m=n,n=r; 转（1）。

例1.2 设整形数组 list 中有 m 个元素，设计算法求 list 中元素的全排列，请绘制算法流程图。

解：设 k 指向 list 中第 k 个元素，m 指向 list 中第 m 个元素，函数 perm(int list[],int k,int m)实现 list 中元素的全排列。

输入：待全排列的数组 list, k 和 m

输出：全排列后的数组 list.

算法实现的程序流程图如图 1-1 所示。

例1.3 请用伪代码的描述方法描述求最小生成树的算法

解：采用 Prime 算法求解最小生成树，其伪代码如图 1-2 所示。其中 V 表示图中所有顶点的集合，U 表示生成树上顶点的集合，V\U 表示 V 中不包含 U 中的顶点集合，$l(u,v)$表示顶点 u 和顶点 v 之间的边权值，$\pi(v)$表示与 v∈V\U 与

U 中的顶点所构成的最小边权值；T 为求得的最小生成树，F 为生成树上边的集合；N（U）表示与集合 U 中的顶点相邻的图中的其它的顶点。

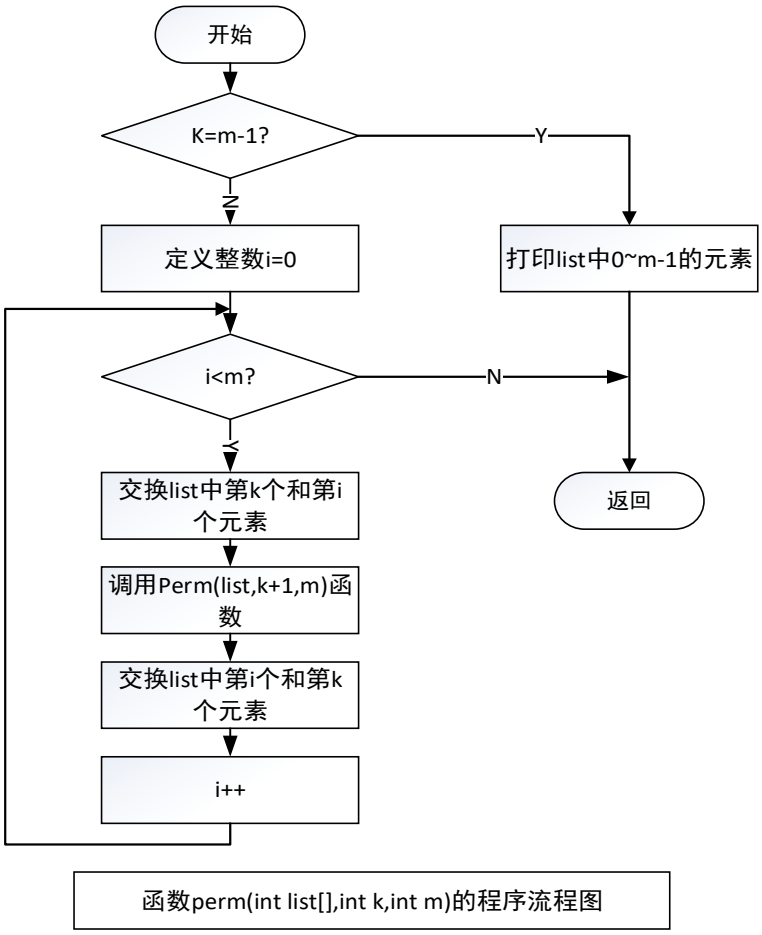


图 1-1 求 list 中元素全排列的程序流程图

input : a connected graph $G = (V, E)$ with weights $\ell : E \to \mathbb{N}$
output: a minimum spanning tree $T = (V, F)$ of G

```
1  begin
2      for v ∈ V do π(v) ← ∞;
3      ;
4      choose an arbitrary vertex u ∈ V;
5      U ← {u}; F ← ∅;
6      while N(U) ≠ ∅ do
7          for v ∈ N(u) \ U do
8              if π(v) > ℓ(uv) then
9                  π(v) ← ℓ(uv); p(v) ← u
10         choose v ∈ V \ U with minimal π(v);
11         U ← U ∪ {v}; F ← F ∪ {p(v)v}; u ← v
```

图 1-2 Prime 算法求最小生成树伪代码

实验 1.1 求两个圆相交部分的面积，精确到小数点后 3 位。

实验预习：

（1） 用自然语言描述法描述上述问题的求解算法。

（2） 绘制算法程序流程图。

（3） 用伪代码描述上述问题的求解算法。

（4） 根据相关的描述方法，编写程序。

上机实验：

（5） 设计相应的测试数据，上机验证程序执行结果是否正确。

（6） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

1.2 算法执行过程分析

正确理解算法的执行过程是理解算法的基础。在给定算法或代码的情况下，如何正确理解算法的执行过程?

为了分析算法的执行过程，需要了解算法执行过程中每个变量的变化情况。往往通过特定输入情况下变量的变化情况来理解算法的执行过程，变量的变化可通过列表或者树形结构来表示。

例1.4 某算法的伪代码如图 1-3 所示，请分析算法在图 1-4 输入情况下的执行过程及结果（要求按字母顺序处理输入图形）。

```
input  : a graph G = (V, E)
output: a DFS-forest T = (V, F) of G and a DFS-numbering σ : V → ℕ
1  begin
2      i ← 1; F ← ∅;
3      for v ∈ V do mark v unvisited;
4      ;
5      while there is an unvisited vertex v ∈ V do DFS-visit(v);
6  procedure DFS-visit(v)
7      mark v visited;
8      σ(v) ← i; i ← i + 1;
9      for w ∈ N(v) do
10         if w is unvisited then
11             F ← F ∪ {vw};
12             DFS-visit(w)
```

图 1-3 某算法伪代码

分析：

算法的输入为一个图，包括图中的顶点 V 和边 E，输出为树 F 和顶点输出的编号σ(v)。在算法执行过程中，有中间变量 i, v,w,N(v)，visited[v]这些变量的变化对理解算法的执行过程都非常重要。对于非递归算法而言，可采用列表的形式来描述算法在样例输入情况下变量的变化过程。可建立表 1-1 所示的结构及变量处理结果。

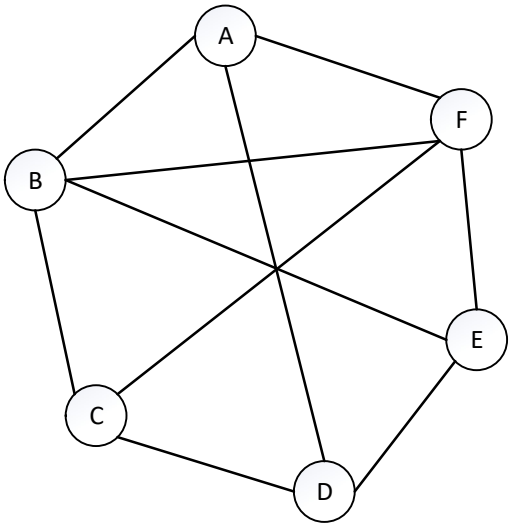


图 1-4 图 3 算法的输入 G

表 1-1 图 1-3 算法执行过程分析

v	w	Visited[v]	i	N(v)	σ(v)	F
A	B	visited	1	{B,D,F}	1	{{AB}}
B	C	visited	2	{A,C,E,F}	2	{{AB}{BC}}
C	D	visited	3	{B,D,F}	3	{{AB}{BC}{CD}}
D	E	visited	4	{A,C,E}	4	{{AB}{BC}{CD}{DE}}
E	F	visited	5	{A,B,C,E}	5	{{AB}{BC}{CD}{DE}{EF}}

例1.5 请分析下面程序在样例输入下的程序执行过程。

样例输入：

```
4 6 4 3 2 2 1 1
0 0
```

```
#include <cstdio>
using namespace std;
int total, n, a[13], b[13];
bool NONE = 1;
void sumItUp(int pos, int num, int sum) {
  if (sum > total) {
      return;
  }
  else if (sum == total) {
      NONE = 0;
      for (int i = 0; i < num; ++i) {
          if (i == 0) {
              printf("%d", b[i]);
          }
          else {
              printf("+%d", b[i]);
          }
```

```
18          }
19          printf("\n");
20      }
21      else {
22          for (int k = pos; k < n; ++k) {
23              b[num] = a[k];
24              sumItUp(k + 1, num + 1, sum + a[k]);
25              while (a[k] == a[k+ 1]) {
26                  ++k;
27              }
28          }
29      }
30  }

31  int main()
32  {
33      while (~scanf("%d%d", &total, &n) && n) {
34          for (int i = 0; i < n; ++i) {
35              scanf("%d", &a[i]);
36          }
37          NONE = 1;
38          printf("Sums of %d:\n", total);
39          sumItUp(0, 0, 0);
40          if (NONE) {
41              printf("NONE\n");
42          }
43      }
44      return 0;
45  }
```

分析：

该程序采用递归调用，其求解过程类似于树形结构。树中结点的值即为函数 sumItUp 主要变量的值，主要包括 sum,num,pos,b[]。其他变量如 a,n 和 total 为全局性的变量，按照样例输入这些变量的值为：

total=4, n=6, a[]={4,3,2,2,1,1}

这些变量在程序执行过程中没有变化，因此可以不作为树中结点的值。上述程序主程序部分完成 total，n,a 变量的输入，并输出 NONE=1 情况下的结果。算法主要部分集中在函数 sumItUp 中，该函数的求解过程可用图 1-5 来描述。

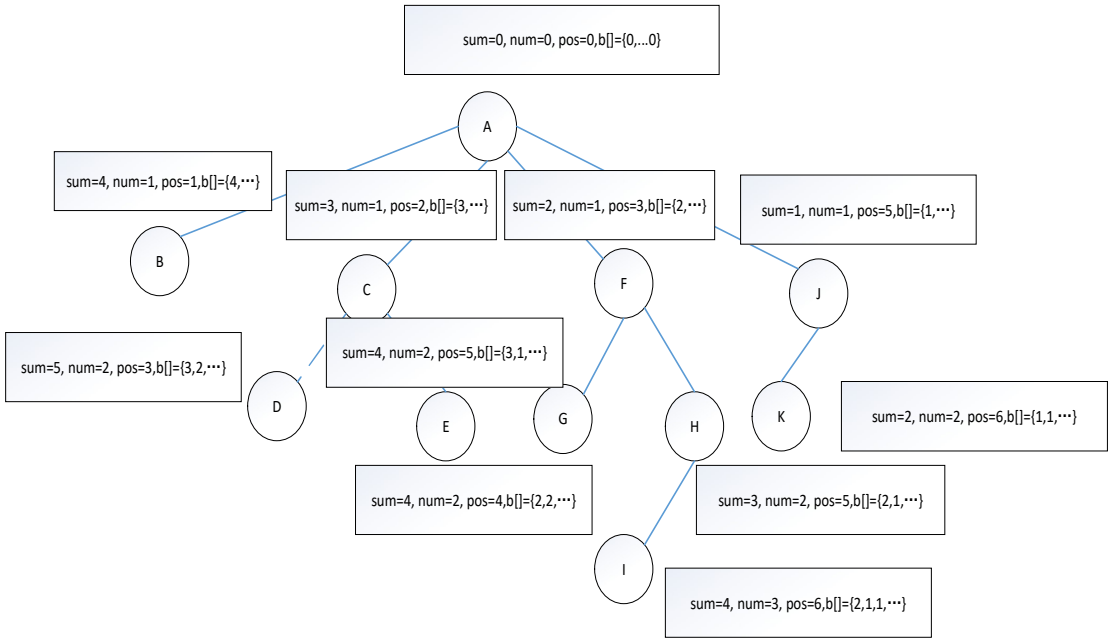


图 1-5　例 1.5 函数 sumItUp 执行过程

实验 1.2 针对实验 1.1 中的问题及算法描述。

实验预习：

（1） 设计测试输入样例，分析算法执行过程中变量变化情况。

上机实验：

（2） 上机调试，验证（1）的分析结果是否与实验结果一致。

（3） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

实验 1.3 程序执行过程分析。

实验预习：

（1） 分析下面的算法在样例输入情况下程序的执行过程及结果

上机实验：

（2） 上机实验，验证（1）的分析过程及结果是否正确性。

（3） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

```
1     #include<cstdio>
2     #include<cstring>
3     #include<iostream>
4     using namespace std;
5     char chess[10][10];
6     int sign[10];
7     int n, k, ans;
8     void dfs(int x, int k)
9     {
```

```
10      if (k == 0) {
11          ans++;
12          return;
13      }
14      if (x > n) {
15          return;
16      }
17      for (int i = x; i <= n; i++) {
18              for (int j = 1; j <= n; j++) {
19                  if (!sign[j] && chess[i][j] == '#') {
20                  sign[j] = 1;
21                  dfs(i + 1, k - 1);
22                  sign[j] = 0;
23              }
24          }
25      }
26     }
27     int main() {
28      while (cin >> n >> k) {
29          memset(chess, 0, sizeof(chess));
30          memset(sign, 0, sizeof(sign));
31          if (n == -1 || k == -1) {
32              break;
33          }
34          for (int i = 1; i <= n; i++) {
35              for (int j = 1; j <= n; j++) {
36                  cin >> chess[i][j];
37              }
38          }
39          ans = 0;
40          dfs(1, k);
41          cout << ans << endl;
42      }
43      return 0;
44     }
```

样例输入：

3 2

#.#

###

.#.

1.3 算法的程序实现

在给定算法的情况下应掌握如何用相应的计算机语言实现该算法。程序是数据结构，算法及计算机语言的结合。算法的程序结构一般分为递归和非递归两种形式，所采用的数据结构包括线性结构，树型结构以及图的结构，相应的计算机语言包括 C/C++, Java, python, Pascal 等。本教程要求使用 C/C++实现相应的算法。

实验 1.4 图 1-6 中的算法实现了有向图的拓扑排序。

实验预习：

（1） 设计至少 1 组测试用例，分析算法的执行过程。

（2） 用 C/C++语言实现该算法。

上机实验：

（3） 根据（1）中设计的测试用例及分析结果，运行（2）中设计的程序验证分析过程是否正确。

（4） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

```
input  : a dag G = (V, A)
output: a topological sort σ : V → ℕ of G
1  begin
2      i ← |V|;
3      for v ∈ V do mark v unvisited;
4      ;
5      while there is an unvisited vertex v ∈ V do TS-visit(v);
6  procedure TS-visit(v)
7      if v is marked stacked then stop;
8      ;
9      if v is marked unvisited then
10         mark v stacked;
11         for w ∈ N⁺(v) do TS-visit(w);
12         ;
13         mark v visited;
14         σ(v) ← i; i ← i − 1
```

图 1-6 拓扑排序算法伪代码

实验总结：

本部分通过一系列的实例介绍了如何描述一个算法，如何分析理解及实现一个算法。通过一系列的实验，加深学生对算法基础知识的理解，掌握算法的描述，分析及实现方法。

实验 2 算法效率分析与比较

实验目的：

（1） 实验比较数据输入规模对算法执行时间的影响。

（2） 实验比较不同算法对问题求解时间的影响。

（3） 掌握算法时间效率的分析方法。

2.1 数据输入规模对算法执行时间的影响

一般而言，算法的执行时间会随着数据输入规模的增大而增大，输入的数据越多，处理数据所花的时间也会越长。但具体呈现什么样的关系，与采用的算法有关。

例2.1 设某排序算法采用直接插入排序，要求：

（1） 编写程序计算待排序序列（随机序列）在长度为 n 时的程序执行时间（单位：ms）。

（2） 设待排序随机序列的起始长度为 1000，序列长度每次递增 1000，直到 100000，绘制序列长度与计算机执行直接插入排序算法时间之间的关系曲线。横轴为序列长度，纵轴为算法执行时间。

解：（1）首先应该生成长度为 n 的随机序列作为待排序序列，然后通过直接插入排序算法实现排序。在排序之前和排序结束之后分别获得相应的系统时间，计算二者之差即为直接插入排序算法的执行时间。

算法的运行时间显然与计算机的配置以及计算机运行时 CPU 的状态有关，要评估算法在一台计算机上运行时间的准确性，应以算法多次运行的平均值作为评价依据才能得到比较准确的结果。

相应的程序代码如下：

```
1    #include<stdio.h>
2    #include<malloc.h>
3    #include<stdlib.h>
4    #include<time.h>
5    #include <sys/timeb.h>
6    #include "string.h"
     //序列存储结构定义
7    typedef struct RedType{
8      int   key;              //待排序序列关键字项
9    }RedType;
10   typedef struct SqList{
```

```
11      RedType  *r;       //存储所有的数据元素，0号存储单元用作监视哨
12      int  Length;        //待排序序列的长度
13    }SqList;
      //生成的长度为sqListLen的随机整数序列
14    SqList* CreateRandomSqList(int sqListLen)
15    {
16      SqList* sq;
17      int   i;
18      //分配存储空间
19      sq = (SqList*)malloc(sizeof(SqList));
20      //输入序列长度
21      sq->Length = sqListLen;
22      //给序列的每个元素分配存储空间
23      sq->r = (RedType*)malloc(sizeof(RedType) * (sq->Length + 1));
24      //生成随机序列
25      srand((unsigned)(time(NULL)));
26      for (i = 1; i <= sq->Length; i++) {
27          sq->r[i].key = int(rand());
28      }
29      //返回序列起始地址
30      return sq;
31    }
32    void InsertSort(SqList *L)
33    {
34      int i,j;
35        //从第二个元素开始，将后面的元素插入到前面的有序序列
36      for(i=2;i<=L->Length;++i){
37          //如果后面元素的关键字小于前面元素的关键字，则继续向前查找
38          if(L->r[i].key<L->r[i-1].key){
39              //复制为监视哨
40              L->r[0]=L->r[i];;
41              //元素后移
42              for(j=i-1;L->r[0].key<L->r[j].key;--j)
43                  L->r[j+1]=L->r[j];
44              //实现元素插入
45              L->r[j+1]=L->r[0];
46          }
47      }
48      return;
49    }

50    int main()
51    {
52     //定义待排序序列
```

```
53      SqList *L;
54        //序列长度
55        int n;
56     //存储算法运行开始时间和结束时间
57      struct __timeb64 stime,etime;
58       //计算对应的秒和毫秒数
59      long int rmtime,rstime;
60       scanf("%d",&n);
61     L = CreateRandomSqList(n);
62     //获取算法执行前的系统时间
63     _ftime64(&stime);
64     InsertSort(L);
65     //获取算法执行后的系统时间
66     _ftime64(&etime);
67     free(L->r);
68     free(L);
69     //计算时间差
70     rstime = etime.time - stime.time;
71     rmtime = rstime * 1000;
72     rmtime += etime.millitm - stime.millitm;
73        printf("Execute time=%d", rmtime);
74       return 1;
75     }
```

(2) 要求解不同长度情况下排序算法的执行时间，可修改主程序，添加多次循环调用序列生成和排序算法部分代码，并将每次算法的执行时间保存到表格中，然后利用 Excel 软件绘制相应的曲线。主程序部分的代码修改如下:

```
1      int main()
2      {
3        //定义待排序的随机序列
4       SqList *L;
5        //定义算法执行前和执行后的系统时间
6       struct __timeb64 stime,etime;
7        //定义算法执行时间秒和毫秒
8       long int rmtime,rstime;
9       char ch[20];
10      FILE* fp;
11       //打开结果存储文件
12      fp = fopen("Curv.csv", "w");
13       //循环调用序列生成和排序算法部分代码
14      for (int i = 1000; i <= 100000; i = i + 1000) {
15          itoa(i, ch, 10);
16          strcat(ch, ",");
17            //将横坐标值转换后写入文件
```

```
18          fwrite(ch, sizeof(char), strlen(ch), fp);
19      }
20      fwrite("\n", sizeof(char), 1, fp);
21      for (int i = 1000; i <= 100000; i = i + 1000) {
22          L = CreateRandomSqList(i);
23          _ftime64(&stime);
24          InsertSort(L);
25          _ftime64(&etime);
26          free(L->r);
27          free(L);
28          rstime = etime.time - stime.time;
29          rmtime = rstime * 1000;
30          rmtime += etime.millitm - stime.millitm;
31          itoa(rmtime, ch, 10);
32          strcat(ch, ",");
33            //将纵坐标值转换后写入文件
34          fwrite(ch, sizeof(char), strlen(ch), fp);
35      }
36      fwrite("\n", sizeof(char), 1, fp);
37      fclose(fp);
38       return 1;
39  }
```

运行上述程序,在如下计算机软硬件配置情况下得到图 2-1 所示曲线：

（1） 操作系统：Windows 10

（2） 开发软件：Visual studio.net 2019

（3） CPU: Intel®Core™ i7-7700HQ CPU@2.80GHz*8

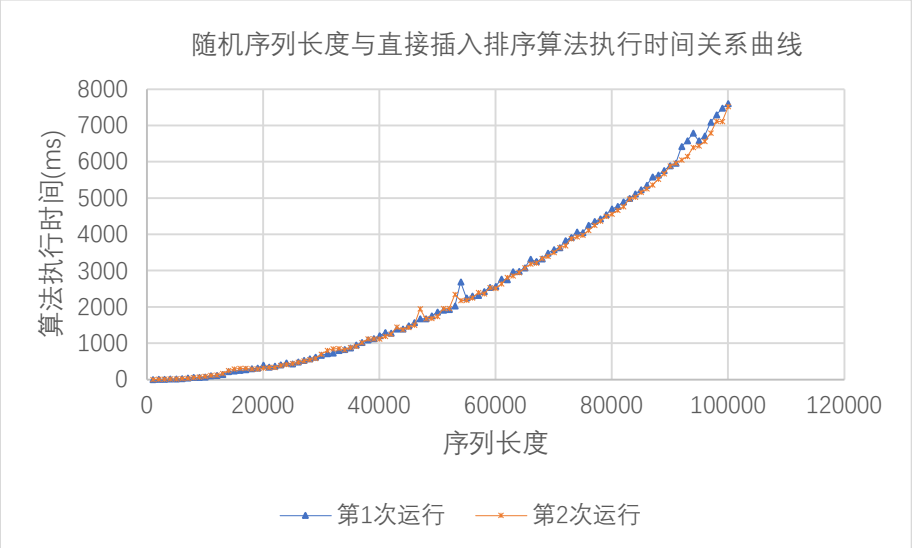


图 2-1 某次运行时随机序列直接插入排序算法执行时间与序列长度关系曲线

图 2-1 展示了两次运行该算法的执行时间。从图中来看，两次运行的时间略有差异，但差异不大。

按照算法时间复杂度分析方法，我们可以得到插入排序算法的复杂度为 $T(n)=a_1n^2+a_2n+a_3=O(n^2)$。上面的曲线取 $a_1=7\times10^{-7}$，$a_2=8\times10^{-4}$，$a_3=0$ 的情况下可以得到图 2-2 所示的曲线（包含前 2 次的实际的运行时间）。

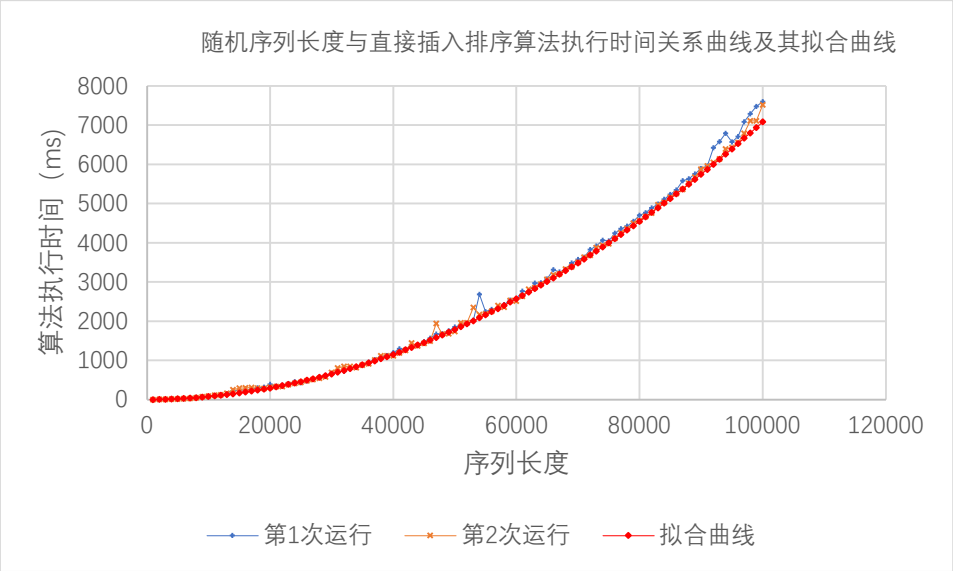


图 2-2 算法运行时间及其拟合曲线

从图 2-2 可以看出，拟合曲线可以较好的与实际运行时间吻合，从而验证算法复杂度分析结果与实际结果基本一致。

实验 2.1 采用快速排序对某序列进行排序。

实验预习：

（1） 编写快速排序算法对应的程序，并计算序列长度为 n 时算法的运行时间。要求将运行时间保存到文件中。文件分成两行，第 1 行为 n 的值，第 2 行为该算法的执行时间，以 ms 为单位。

（2） 编写程序，计算 n=1000~100000，每次递增 1000 情况下算法的运行时间，要求将运行时间保存到文件中。文件分为两行，第 1 行为 n 的值，用逗号隔开，第 2 行为对应 n 的执行时间，以 ms 为单位，用逗号隔开。

上机实验：

（3） 上机运行步骤（2）得到的程序，利用 Excel 打开保存的文件，并绘制相应的序列长度-运行时间曲线；重复执行（2）中的程序 3 次，比较对应的时间曲线。

（4） 结合快速排序算法的时间复杂度计算方法，绘制其拟合曲线，分析比较（3）中绘制的曲线和时间复杂度曲线。

（5） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

2.2 算法对时间复杂度的影响

不同的算法即使处理同一问题，其时间复杂度也不相同，当然算法执行时的运行时间也不相同。

例2.2 在例 2.1 程序的基础上，采用归并排序算法对序列进行排序，比较相同条件下算法的执行时间。

解：在例 2.1 程序的基础上，添加归并排序算法，并修改主程序部分，得到两种算法在相同输入序列及长度时对应的执行时间。

归并排序算法实现如下：

```
1     void MergeSort(SqList *L)
2     {
3       MSort(L->r,L->r,1,L->Length);
4        return;
5     }
6     void MSort(RedType *SR, RedType *TR1, int s,int t)
7     {
8       RedType *TR2;
9       int  m;
10      //分配临时存储空间
11      TR2=(RedType*)malloc(sizeof(RedType)*(t+1));
12      //如果起始位置和终止位置相同，则直接将待归并序列保存到结果中
13      if(s==t)
14          TR1[s]=SR[s];
15      //递归地将原序列分成两个部分，然后再归并成一个序列
16      else{
17          m=(s+t)/2;
18          MSort(SR,TR2,s,m);
19          MSort(SR,TR2,m+1,t);
20          Merge(TR2,TR1,s,m,t);
21      }
22        free(TR2);
23       return;
24    }
25    void Merge(RedType *SR, RedType *TR, int i,int m,int n)
26    {
27          int j,l,k;
28          //将有序地SR[i...m]和SR[m+1...n]归并为有序地TR[i...n]
29       for(j=m+1,k=i;i<=m&&j<=n;++k){
30          //如果第一个序列的第i个关键字小于第二个序列的第j个关键字，
31          //则将第i个关键字送到TR中，反之，将第j个关键字送到TR中
32           if(SR[i].key<=SR[j].key)
33               TR[k]=SR[i++];
```

```
            else
                TR[k]=SR[j++];
        }
        //将剩余的关键字（包括前面部分和后面部分）送到TR中
        if(i<=m){
            for(l=k;i<=m;l++,i++)
                TR[l]=SR[i];
        }
        if(j<=n){
            for(l=k;j<=n;l++,j++)
                TR[l]=SR[j];
        }
      return;
    }
```

主程序部分修改如下：

```
    int main()
    {
       //为了便于曲线绘制，因此先将计算时间存储在内存中
       //然后再写入文件
     SqList *L,*L1;
     struct __timeb64 stime,etime;
     long int rmtime,rstime;
     char ch[20];
     int inSortTime[105], mergeSortTime[105];
     FILE* fp;
     fp = fopen("Curv.csv", "w");
     for (int i = 1000; i <= 100000; i = i + 1000) {
         itoa(i, ch, 10);
         strcat(ch, ",");
         fwrite(ch, sizeof(char), strlen(ch), fp);
     }
     fwrite("\n", sizeof(char), 1, fp);
     int k = 0;
     for (int i = 1000; i <= 100000; i = i + 1000) {
         L = CreateRandomSqList(i);
         L1 = CopyRandomSqList(L);
         _ftime64(&stime);
         InsertSort(L);
         _ftime64(&etime);
         free(L->r);
         free(L);
         rstime = etime.time - stime.time;
         rmtime = rstime * 1000;
         rmtime += etime.millitm - stime.millitm;
```

```
77          inSortTime[k] = rmtime;
78          _ftime64(&stime);
79          MergeSort(L1);
80          _ftime64(&etime);
81          free(L1->r);
82          free(L1);
83          rstime = etime.time - stime.time;
84          rmtime = rstime * 1000;
85          rmtime += etime.millitm - stime.millitm;
86          mergeSortTime[k] = rmtime;
87          k++;
88       }
89
90      WritetoFile(100, inSortTime, fp);
91      WritetoFile(100, mergeSortTime, fp);
92      fclose(fp);
93        return 1;
94      }
        //创建一个与csq一样的存储空间，便于进行其他排序算法
95      SqList* CopyRandomSqList(SqList* csq)
96      {
97       SqList* sq;
98       int   i;
99       //分配存储空间
100      sq = (SqList*)malloc(sizeof(SqList));
101      //输入序列长度
102      sq->Length = csq->Length;
103      //给序列的每个元素分配存储空间
104      sq->r = (RedType*)malloc(sizeof(RedType) * (sq->Length + 1));
105      //输入序列中的元素
106      for (i = 1; i <= sq->Length; i++) {
107          sq->r[i].key = csq->r[i].key;
108      }
109      //返回序列起始地址
110      return sq;
111     }
        //将sortTime中num个数写入fp指向的文件中
112     void WritetoFile(int num,int sortTime[],FILE *fp) {
113
114      char ch[20];
115      for (int i = 0; i <num ; i++) {
116          itoa(sortTime[i], ch, 10);
117          strcat(ch, ",");
118          fwrite(ch, sizeof(char), strlen(ch), fp);
```

```
119     }
120     fwrite("\n", sizeof(char), 1, fp);
121   }
```

上述程序运行后的结果如图 2-3 所示。

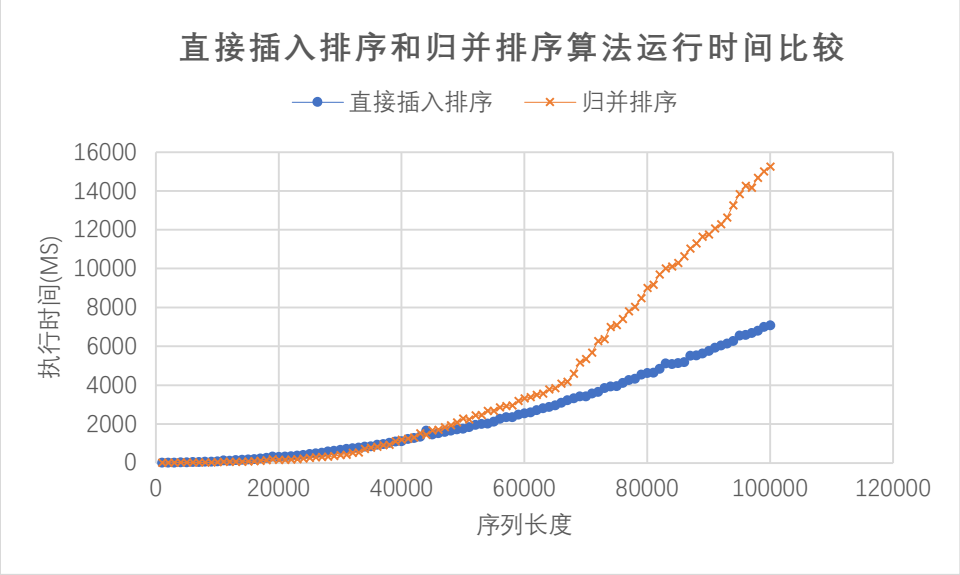


图 2-3 直接插入排序和归并排序算法执行时间比较

从图 2-3 可以看出，采用不同算法对同一序列进行排序时其执行时间并不相同。从算法的时间复杂度来看，归并排序算法的时间复杂度应该为 O (nlogn)，效率应高于直接插入排序。但从图 2-3 的曲线来看在序列长度超过一定长度时直接插入排序的执行时间反而比归并排序执行时间要少，是什么原因导致出现上述现象？

分析前面归并排序算法的 MSort 函数会发现，该函数每调用一次，就要进行一次内存的分配，而内存分配需要花费不少时间，特别是序列长度较长时更是如此。同时在调用 Merge 函数时，也需要将数分别拷贝到两个不同的内存空间，同样需要花费不少的时间。虽然归并排序本身的复杂度并不高，但执行这些语句所花费的时间却很多。特别是在序列长度较长的情况下，内存分配和拷贝所花的时间会明显增加。

对上述归并排序算法进行如下的改进，可以降低归并排序算法的执行时间。

```
1    void aMergeSort(SqList* L)
2    {
3     aMSort(L->r,1, L->Length);
4     return;
5    }
6    void aMSort(RedType* SR,int s, int t)
7    {
8     int  m;
9     //递归地将原序列分成两个部分，然后再归并成一个序列
```

```
10      if (s<t){
11          m = (s + t) / 2;
12          aMSort(SR,s, m);
13          aMSort(SR,m + 1, t);
14          aMerge(SR,s, m, t);
15      }
16      return;
17     }
18     void aMerge(RedType* SR, int i, int m, int n)
19     {
20      int j, l, k;
21      //将有序地SR[i...m]和SR[m+1...n]归并为有序地TR[i...n]
22      for (j = m + 1; j <= n; j++) {
23          //如果第一个序列的第i个关键字小于第二个序列的第j个关键字,
24          //则采用直接插入排序方法将第
25          if (SR[j].key < SR[j - 1].key) {
26              int temp = SR[j].key;
27              for (k = j - 1; temp < SR[k].key && k >= i; --k)
28                  SR[k + 1].key = SR[k].key;
29              SR[k + 1].key = temp;
30          }
31      }
32      return;
33     }
```

主程序中调用 aMergeSort 函数替代原来的 MergeSort 函数，可得到如图 2-4 所示的比较曲线。

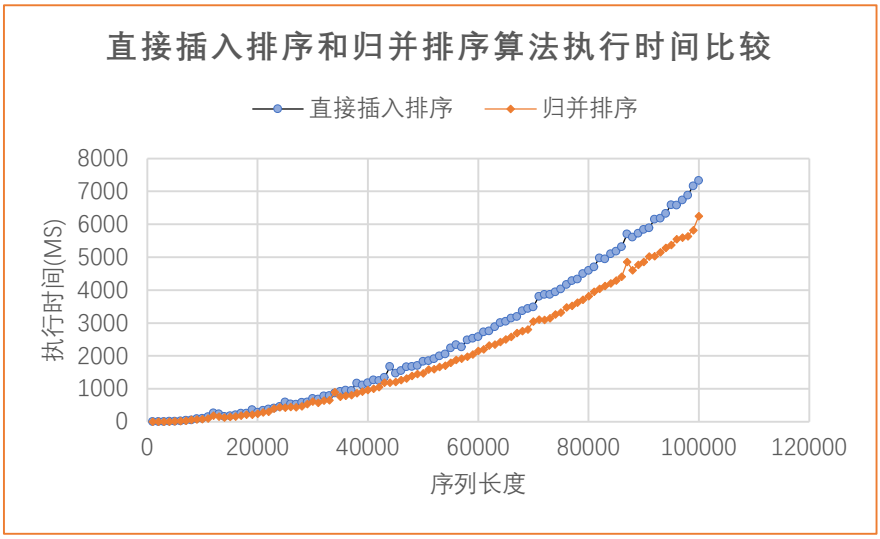


图 2-4 直接插入排序与修改后的归并排序算法执行时间比较

从图 2-4 可以看出，修改后的归并排序算法比直接插入排序算法的执行时间要少，时间效率得到提升。因此程序的执行时间受很多因素的影响，包括所采用的函数，实现算法的方式等。只有既了解算法也了解程序实现语言，才能编写高

效率的算法程序。

实验 2.2 设计实验比较快速排序，归并排序以及堆排序算法的时间效率。

实验预习：

（1） 编写程序实现堆排序，并求解在输入序列长度为 n 情况下的执行时间。

上机实验：

（2） 上机实验，比较堆排序，快速排序以及归并排序在相同输入情况下的执行时间，绘制相应的序列长度-运行时间曲线，分析影响算法执行时间的主要因素以及如何降低算法的执行时间。

（3） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

2.3 算法时间效率分析

算法的程序结构一般分为递归和非递归两种形式。

对于非递归的程序结构可计算出每一条语句的频度，乘以每条语句的执行时间（就具体的计算机而言每条语句的执行时间是常数）得到该条语句总的执行时间，然后计算出整个算法中所有语句的执行时间，得到执行时间与输入数据规模之间的关系表达式，再对表达式化简，按照渐进性函数的定义求得相应算法的时间复杂度。

对于递归的程序结构，可设在输入数据规模情况下运行该算法所需时间，以函数形式来表示，然后根据算法的执行过程，得到该函数的递归表达式，再采用数学分析的方法求解时间表达式与输入数据规模之间的关系，从而得到相应算法的时间复杂度。

例2.3 设某图有 n 个顶点，m 条边，分析下面算法的时间复杂度。

```
input  : a graph G = (V, E) with weights ℓ : E → ℕ and a source s ∈ V
output: the distances d(s, v) for all v ∈ V and
        a shortest path tree T = (V, F) of G with source s
1  begin
2      for v ∈ V do π(v) ← ∞;
3      ;
4      d(s, s) ← 0; π(s) ← 0; U ← {s}; u ← s; F ← ∅;
5      while N(U) ≠ ∅ do
6          for v ∈ N(u) \ U do
7              if π(v) > π(u) + ℓ(uv) then
8                  π(v) ← π(u) + ℓ(uv); p(v) ← u
9          choose v ∈ V \ U with minimal π(v);
10         d(s, v) ← π(v);
11         U ← U ∪ {v}; F ← F ∪ {p(v)v}; u ← v
```

分析：

算法第 2 行的时间复杂度为 O（n）,第 4、10，11 行的时间复杂度为 O(1),第 5 行的时间复杂度为 O(n),第 9 行的时间复杂度为 O(nlogn)（如果采用堆的存储结构）,第 6 行的执行时间为 O（m）,如果采用堆的存储结构，第 7，8 行的时间复杂度为 O（mlogn）,因此整个算法的时间复杂度为:

T(n)=O(n)+O(nlogn)+O(mlogn)+O(1)+O(m)=O((m+n)logn)

例2. 4 分析下面算法的时间复杂度。

```
1 function fibonacci(n)
2 |   (x, y) ← pair(n);
3 |   return x;
4 function pair(n)
5 |   if n = 0 then
6 |   |   return (0, 1)
7 |   else
8 |   |   (x, y) ← pair(n − 1);
9 |   |   return (y, x + y)
```

分析：

设函数 Fibonacci 的执行时间为 T(n)，函数 pair 的执行时间为 P(n)，则

T(n)=P(n)=P(n-1)+1　且 P(n)=1

显然:

T(n)=T(n-1)+1=T(n-2)+2=T(n-3)+3=T(0)+n=n+1=O(n)

例2. 5 设某算法的程序如下所示，请分析该算法的时间复杂度。

```
#include "stdafx.h"
#include <iostream>
using namespace std;
void dfs(int k, int n);
int sum = 0;
int main()
{
   int n;
1   cin >> n;
2   dfs(0, n);
3   cout << sum-1<< endl;
4   return 0;
}
void dfs(int k, int n){
5   for (int i =k+1; i <=n; i++) {
6       if (n <= 2 * i) {
7           sum++;
8           break;
        }
9       dfs(i, n - i);
    }
```

```
10      return;
    }
```

分析:

行 1，3，4 的执行时间为常数，设函数 dfs 的执行时间为 T(0,n),则主程序 main()的执行时间为 T(0,n)+1。

函数 dfs（i，n）在 n<2*i 时执行 7，8 行，其执行时间为常数，也就是:

当 i>=n-i 时，T（i,n-i）=1

否则:

$T(0,n)=T(1,n-1)+T(2,n-2)+\cdots+T(i,n-i)+\cdots+T(\lceil\frac{n}{2}\rceil-1,n+1-\lceil\frac{n}{2}\rceil)+1$

采用递推可以得到:

$T(0,n)=2^{\lceil\frac{n}{2}\rceil-1}=O(2^{\lceil\frac{n}{2}\rceil})$

实验 2.3 已知某算法的程序如下所示。

实验预习：

（1） 请分析该算法的时间复杂度。

上机实验：

（2）上机实验，绘制出 m 不变时，变量 n-算法时间复杂度和变量 n-运行时间曲线，并进行简要分析比较。

（3）上机实验，绘制出 n 不变时，变量 m-算法时间复杂度和变量 k-运行时间曲线，并进行简要分析比较。

（4）综合（2）（3）在同一个图中绘制出 m 在几种变化情况下的变量 n-算法时间复杂度和变量 n-运行时间曲线。

（5）撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

```
1.  #include "stdafx.h"
2.  include "iostream"
3.  using namespace std;
4.  class Color{
5.  friend int mColoring (int,int,int **);
6.  public:
7.  bool Ok(int k);
8.  void Backtrack(int t);
9.  int n,m,**a,*x;
10. long   sum;
```

```
};
int mColoring (int n,int m,int **a);
int main()
{
    int n,m,**a,sum;
    cout<<"Please input the number of colors:";
    cin>>m;
    cout<<"Please input the number of nodes:";
    cin>>n;
    a=new int *[n+1];
    for(int i=0;i<=n;i++)
        a[i]=new int[n+1];
     cout<<"Please input the relation of the nodes:
                    1-->Connected, 0-->Not connected:";
     for(int i=1;i<=n;i++){
            for(int j=i+1;j<=n;j++){
                cout<<"Please input the relation of "<<i<<" and "<<j<<": ";
                cin>>a[i][j];
                a[j][i]=a[i][j];
}
}
      sum=mColoring(n,m,a);
      return 0;
}
bool Color::Ok(int k)
{
      for(int j=1;j<=n;j++)
          if((a[k][j]==1)&&(x[j]==x[k]))
                return false;
       return true;
}

void Color::Backtrack(int t)
{
```

```
45.       if(t>n){
46.           sum++;
47.           for(int i=1;i<=n;i++)
48.               cout<<x[i]<<" ";
49.           cout<<endl;
50.             }
51.        else{
52.            for(int i=1;i<=m;i++){
53.                x[t]=i;
54.                if(Ok(t))
55.                      Backtrack(t+1);
56.                x[t]=0;
57.             }
58.  }
59.  }
60.  int mColoring (int n,int m,int **a)
61.  {
62.        Color X;
63.         X.n=n;
64.         X.m=m;
65.         X.a=a;
66.         X.sum=0;
67.         int *p=new int[n+1];
68.         for(int i=0;i<=n;i++)
69.              p[i]=0;
70.         X.x=p;
71.         X.Backtrack(1);
72.         delete [] p;
73.       return X.sum;
74.  }
```

实验 2.4 已知某算法的程序如下所示。

实验预习：

（1） 请分析该算法的时间复杂度。

上机实验：

（2） 上机实验，绘制不同输入情况下的运行时间曲线，并与（1）中的分析结果进行比较。

（3） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

```
#include "stdio.h"
#include "string.h"
#include "stdlib.h"
#include "math.h"
#define Max  110
int n1,n2,m,t;
int match[Max][Max],f[Max][Max],g[Max][Max];
int s1[Max],s2[Max];
int max(int a,int b)
{
    return a>b?a:b;
}
int main()
{
  int i,j,k;
  int ii,jj;
  scanf("%d",&m);
  while(m){
      m--;
      scanf("%d%d",&n1,&n2);
      for(i=1;i<=n1;i++)
      scanf("%d",&s1[i]);
      for(i=1;i<=n2;i++)
          scanf("%d",&s2[i]);
       memset(f,0,sizeof(f));
       memset(g,0,sizeof(g));
       for(i=1;i<=n1;i++){
            for(j=1;j<=n2;j++){
                if(s1[i]==s2[j-1])
                    f[i][j]=j-1;
```

```
                else
                    f[i][j]=f[i][j-1];
                }
            }
        for(i=1;i<=n2;i++){
            for(j=1;j<=n1;j++){
                if(s2[i]==s1[j-1])
                    g[i][j]=j-1;
                else
                    g[i][j]=g[i][j-1];
                }
            }
        memset(match,0,sizeof(match));
        for(i=1;i<=n1;i++){
            for(j=1;j<=n2;j++){
                match[i][j]=max(match[i-1][j],match[i][j-1]);
                if(s1[i]==s2[j]) continue;
                ii=f[i][j];jj=g[j][i];
                if(ii==0||jj==0) continue;
                match[i][j]=max(match[i][j],match[jj-1][ii-1]+2);
                }
            }
        printf("%d\n",match[n1][n2]);
        }
    return 0;
    }
```

实验总结：

本部分实验比较了不同数据输入规模情况下算法的执行时间，不同算法对问题求解效率的影响，影响算法运行时间的一些因素以及如何分析算法的时间效率等。通过一系列的实验，加深学生对算法效率的理解以及如何针对具体问题分析算法的时间效率。

实验 3 穷举法设计与实验

实验目的：

（1） 理解穷举法典型算法的求解过程。

（2） 学习穷举法的时间复杂度分析方法，并通过实验验证算法的执行效率。

（3） 学会如何利用穷举法求解具体问题，了解穷举法的局限性。

3.1 穷举法的求解过程

穷举法是通过穷举罗列出问题的所有可能情形，然后根据题目要求去除掉不符合要求的解而得到问题的解，也称为枚举法或者穷举法。

例 3.1 组合问题求解过程分析。已知有 n 个数，求这 n 个数的所有组合。

分析：

设有 1,2,3,4 四个数，这些数的所有组合为：{1} ，{2}， {3}， {4}， {1,2}，{1,3}， {1,4}， {2,3}， {2,4}， {3,4}， {1,2,3}， {1,2,4}， {1,3,4}，{2,3,4}， {1,2,3,4}，共计 15 个组合数。

考察 1~16 对应的二进制数会发现，如果分别用 1，2，3，4 表示其二进制数中的每一位，当该位为 1 时，则对应的数就是组合数中的数，否则就不是。这样 5 的二进制数 0101 就表示{2，4}的组合。

对于 n 个数而言，只要能生成 1 ~ 2^n-1 个数的二进制数，并通过上面的处理方式，即可得到对应的 n 个数的所有组合，共计 2^n-1 个。

要生成 1 ~ 2^n-1 的二进制数，可以通过构建满二叉树来得到。如果将二叉树的每一个分支分别用 0 或 1 来表示，这样从根开始到最后一层结点的所有分支形成的数除 00…0（n 个 0）之外，其它的结点对应的二进制数即可用于表示对应的 n 个数的组合。图 3-1 表示 n=4 时的满二叉树情况。

通过下面的程序可以得到 n 的所有组合数。

```
1.  #include "stdafx.h"
2.  #include<conio.h>
3.  #include <malloc.h>
4.  void Perm(int);
5.  int *slist,*list,n;
6.  int main()
7.  {
```

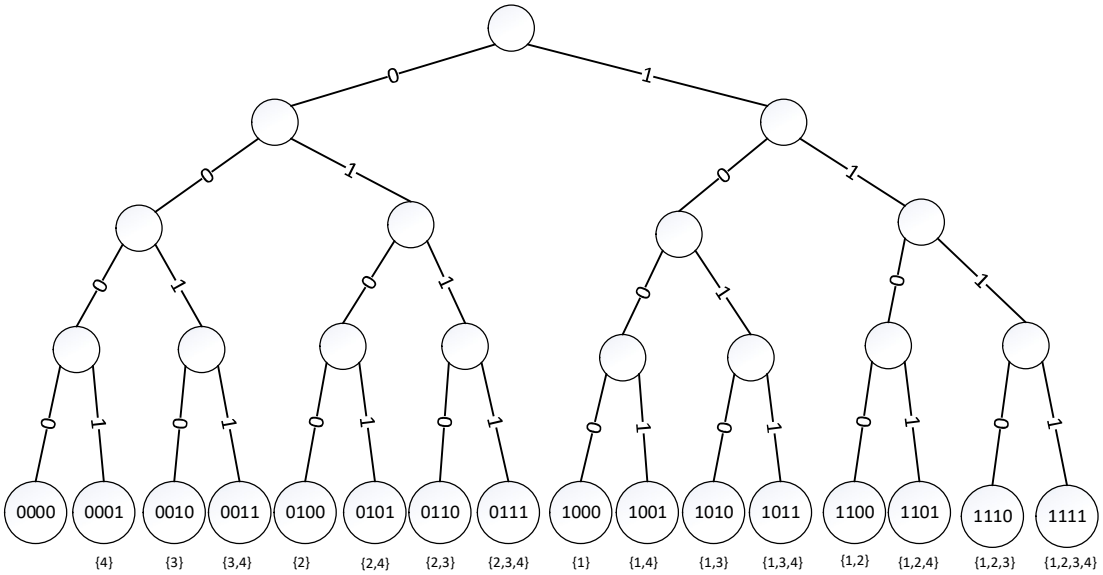


图 3-1 n=4 时的组合数

```
8.      scanf("%d", &n);
9.      slist = (int*)malloc(sizeof(int) * (n + 1));
10.     list = (int*)malloc(sizeof(int) * (n + 1));
11.     for (int i = 0; i < n; i++) {
12.         scanf("%d", &list[i]);
13.     }
14.     Perm(0);
15.     return 0;
16. }
17. void Perm(int k)
18. {
19.     int i;
20.     if(k>=n){
21.         for (i = 0; i <n; i++) {
22.             if(slist[i]==1)
23.                 printf("%d ", list[i]);
24.         }
25.         printf("\n");
26.     }
27.     else{
28.         for(i=0;i<=1;i++){
29.             slist[k] = i;
```

```
30.                     Perm(k+1);
31.                 }
32.         }
33.     return;
34. }
```

上面程序中 list 为 n 个数的集合，slist 中保存了对应的二进制数，通过深度遍历的方法构造二叉树，因此 29 行中的循环次数为 2，二叉树的深度由 n 确定，也就是待组合的数的个数。

在 n=4 时程序的求解过程如图 3-2 所示。图中的箭头表示求解过程，结点中的值表示求解过程中对应变量的值（该程序中 k 和 slist 作为主要变量），每一次递归调用产生一个新的结点（对应的 k 值和 slist 如图中结点的值）。在遍历完所有的结点之后，也就是得到 list 中所有数的组合时，输出对应的组合数。图 3-2 也称为该问题的**解空间树**，是一棵**子集树**。

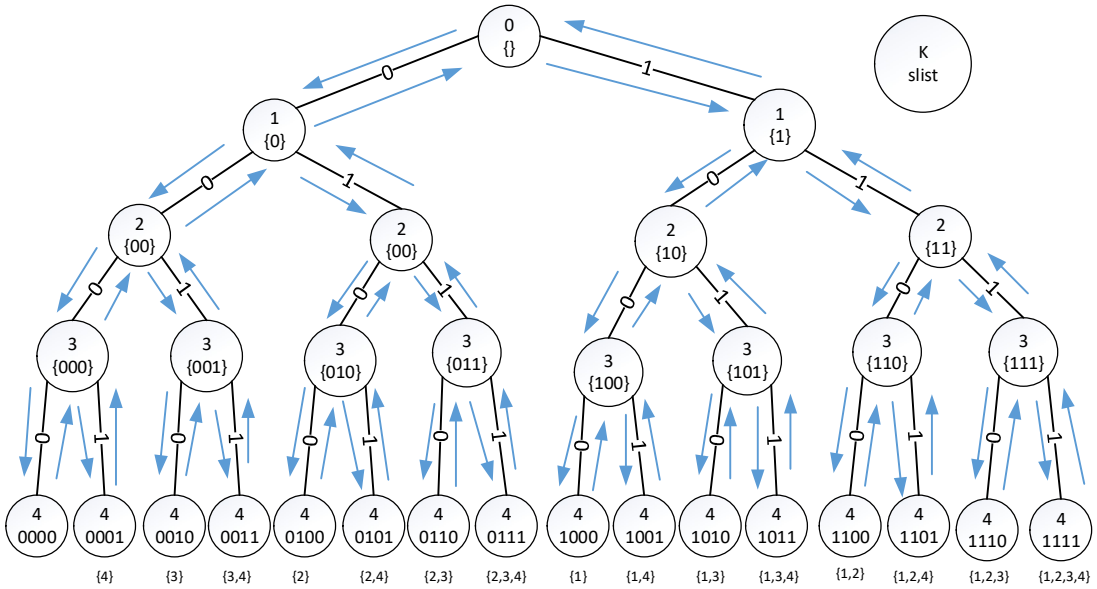


图 3-2 n=4 时程序的执行过程以及变量变化情况

例 3.2 排列问题求解过程分析。已知 n 个数，求这 n 个数的全排列。

分析：

设有 3 个数，分别是 1，2，3，由这 3 个数形成的全排列为{1,2,3}，{1,3,2}，{2,1,3}，{2,3,1}，{3,1,2}，{3,2,1}，共计 6 种排列方法。

同样将上面的排列过程看成是一个树，该树的第一层有 3 个结点，分别是值为 1，2，3 的结点。树的第二层中，结点 1 下面有两个结点，值分别为 2，3；结点 2 下面有两个结点值分别为 1，3；结点 3 下面有两个结点，值分别为 1，2。树的第三层中，结点 2 下面有 1 个结点，值为 3；结点 3 下面有一个结点，值为 2,…。树中从根往下直到叶子结点就构成其中的一个排列，如图 3-3 所示。

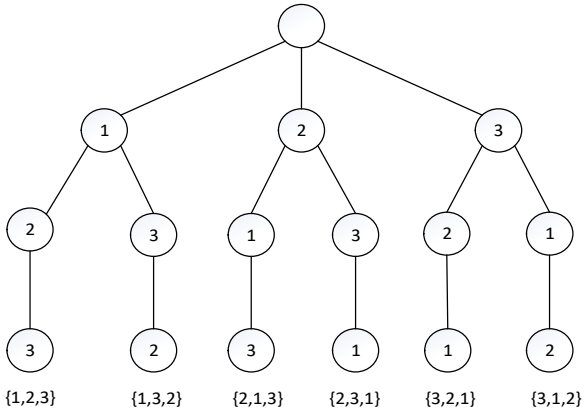


图 3-3 n=3 时的全排列

该树也是 n=3 时该问题的**解空间树**，是一棵**排序树**。要生成上面的树结构，可通过如下的程序实现：

```
1.  #include<conio.h>
2.  #include<iostream>
3.  void Perm(int *,int,int);
4.  void Swap(int &, int &);
5.  using namespace std;
6.  int main()
7.  {
8.      int *list,n;
9.      cin >> n;
10.     list = new int[n];
11.     for (int i = 0; i < n; i++)
12.         cin >> list[i];
13.     Perm(list,0,n-1);
14.     return 0;
15. }
16. void Perm(int *list, int k, int m)
17. {
18.     int i;
19.     if(k==m){
20.         for(i=0;i<=m;i++)
21.             printf("%d ",list[i]);
22.          printf("\n");
23.     }
```

```
24.         else{
25.             for(i=k;i<=m;i++){
26.                 Swap(list[k],list[i]);
27.                 Perm(list,k+1,m);
28.                 Swap(list[k],list[i]);
29.             }
30.         }
31.     return;
32. }
33. void Swap(int &i, int &j)
34. {
35.         int temp;
36.         temp=i;
37.         i=j;
38.         j=temp;
39.         return;
40. }
```

Perm 函数中，list 为待排序的数的集合，k 和 m 分别指向 list 中第 k 和 m 个元素。求解过程中通过不断交换使得前面已排序的元素不再参与后续的排序操作，确保 list 中新产生的的元素不会重复。程序 26 行的循环次数表示树中下一层结点的个数，树的深度由 m 确定。由于循环次数不断减少，因此越往下走，树的孩子结点越少。在 n=3 时，该程序 Perm 函数的执行过程如图 3-4 所示。

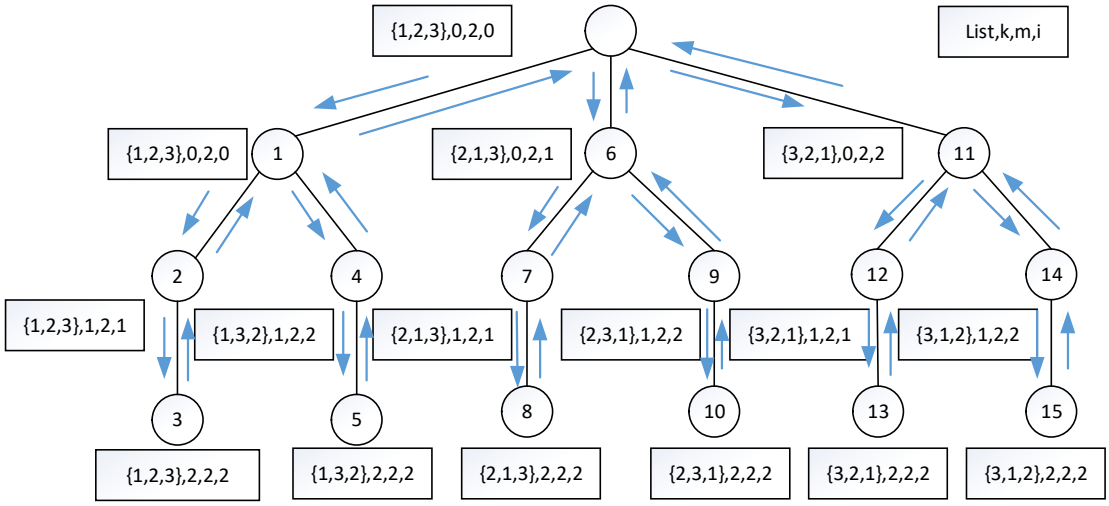


图 3-4 n=3 时函数 Perm 构造排序树的过程

在程序 27 行设置一个断点，通过调试观察 list,k,m,i 的值可以得到如图 3-5 所示的结果，分别与图 3-4 中的结点对应。

名称	值	类型
list[0]	1	int
list[1]	3	int
list[2]	2	int
k	1	int
m	2	int
i	2	int

图 3-5A 与图 3-4 结点 4 对应

名称	值	类型
list[0]	2	int
list[1]	1	int
list[2]	3	int
k	0	int
m	2	int
i	1	int

图 3-5B 与图 3-4 结点 6 对应

名称	值	类型
list[0]	2	int
list[1]	3	int
list[2]	1	int
k	1	int
m	2	int
i	2	int

图 3-5C 与图 3-4 中结点 9 对应

名称	值	类型
list[0]	3	int
list[1]	2	int
list[2]	1	int
k	0	int
m	2	int
i	2	int

图 3-5D 与图 3-4 结点 11 对应

图 3-5 Perm 函数执行过程中部分变量的调试结果

上面的 Perm 函数是构造树形结构的基本程序框架，也是深度遍历常用的递归调用程序框架。正确理解该程序框架有利于算法的具体程序实现。在该函数中，i~m 表示每一层的结点的起始和终止位置，也就是对应的结点个数，m 也表示树的深度。通过修改 i,k,m 的值可以得到不同的树形结构。

思考：

（1） 将上面程序中 25 行修改为如下的语句，对应的树形结构是什么？

```
for(int i=0;i<m;i++)
```

（2） 将上面程序中 25 行修改为如下的语句，对应的树形结构是什么？

```
for(int i=0;i<n;i++)
```

其中 n 可以是不同于 m 的一个整数。

（3） 将上面程序中 25 行修改为如下的语句，对应的树形结构是什么？

```
for(int i=0;i<1;i++)
```

实验 3.1 某木匠在制作家具前往往需要将木料分割成长度不同的木块，在每次分割时都会损坏一定长度的木头。现需制作某件家具，已知每块木料的长度，该家具制作时所需的木块的长度以及分割时损坏的木块的长度，请分析采用穷举法求解样例输入时的求解过程。

输入要求：输入 1 行，包含若干个整数，第 1 个整数是木料的长度，第 2 个整数为每次分割损坏的木块的长度，后面的整数则为该家具制作时所需的木块的长度。

输出要求：输出 1 个整数，即制作该家具所需的木料的块数。

样例输入：

1000 100 250 500 650 1000

样例输出：

3

参考程序如下：

```
#include "stdafx.h"
#include<conio.h>
#include<iostream>
#define MAXM 10
void Perm(int *,int,int,int,int,int set[MAXM][MAXM],int nlist[MAXM]);
using namespace std;
int qlist[MAXM];       //存放木块长度
int loss;              //存放每次切割的损失
int n,len,mn=0;        //存放木块的数量，长度，最少需要的木料的数量
int main()
{
    int *list;
    int set[MAXM][MAXM];
    int nlist[MAXM];
    memset(qlist, 0, sizeof(qlist));
    cin >> len;
    cin >> loss;
    n = 0;
    //每组数据输入之后用-1 作为结束，本程序只处理一组数据,如要处理多组数据可
    //做适当修改
    while (cin >> qlist[n] && qlist[n]!=-1)
        n++;
    mn = n;
    list = new int[n];
    for (int i = 0; i < n; i++) {
        list[i] = i;
    }
    memset(set, 0, sizeof(set));
```

```
29.     memset(nlist, 0, sizeof(nlist));
30.     Perm(list,0,0,n-1,1,set,nlist);
31.     return 0;
32. }
/*********************************************************************************************
   Perm 函数参数定义：
   list-->原始序列，存放每一个木块的编号
   level->搜索到第几层
   k-->list 中搜索的起始位置
   m-->list 中搜索的最后位置
   count->第几块木料，木料的数量
   set->二维矩阵，存放每一块木料中放的是第几个木块，set[i][j]=1 表示第 i 块木料可
        以锯成第 j 块木块
   nlist->当前树中存放的 list 中的第几个木块，存放了所有可以组成当前木料的木块
*********************************************************************************************/
33. void Perm(int *list, int level,int k, int m,int count,int set[MAXM][MAXM],int nlist[MAXM])
34. {
35.     int i;
36.     int nset[MAXM][MAXM];
37.     int mlist[MAXM];
38.     int klist[MAXM];
39.     if (m == 0) {
40.         for (int l = k; l <= m; l++) {
41.             set[count - 1][list[l]] = 1;
42.         }
43.         //判断每一行中木块长度总和+每次废掉的木块是否超过一块木料的长度
44.         int flag = 0;
45.         for (int l = 0; l < count; l++) {
46.             int sum = 0;
47.             for (int h = 0; h < n; h++) {
48.                 if (set[l][h] == 1) {
49.                     sum +=qlist[h] + loss;
50.                 }
51.             }
```

```
52.                 if (sum > len + loss) {
53.                     flag = 1;
54.                     break;
55.                 }
56.             }
57.             //如果所有的都满足，则当前木料的块数是一种选择
58.             if (flag == 0)
59.                 mn = mn > count ? count : mn;
60.             return;
61.         }
62.         //如果已经取完所有的木块，则返回上一层选取其他选的方法
63.         if(k>m){
64.             return;
65.         }
66.         else{
67.             //对于每一块木料都有 k-m 种选取方法，并深度遍历，每次取下一个木块，
68.             //确保不重复取
69.             for(i=k;i<=m;i++){
70.                 //获取上一次在该树上选取的木块
71.                 for (int l = 0; l < level; l++)
72.                     klist[l] = nlist[l];
73.                 //继续取下一层的木块
74.                 klist[level] = list[i];
75.                 //获取还剩余的木块
76.                 int left = 0;
77.                 for (int l = 0; l <= m; l++) {
78.                     int s;
79.                     for( s = 0; s <= level; s++){
80.                         if (list[l] == klist[s])
81.                             break;
82.                     }
83.                     if (s > level) {
84.                         mlist[left++] = list[l];
85.                     }
```

```
86.             }
87.             //设置当前木料所选取的所有木块，nset[l][h]=1 表示第 l 块木料可能锯
88.             //成第 h 块木块
89.             for (int l = 0; l < MAXM; l++) {
90.                 for (int h = 0; h < MAXM; h++) {
91.                     nset[l][h] = set[l][h];
92.                 }
93.             }
94.             for (int l =0; l <=level; l++) {
95.                 nset[count-1][klist[l]] = 1;
96.             }
97.             //如果所有的木块均已取完，则是一种选择，求出满足条件的木料的数
98.             //量，并计算最小的木料数
99.             left--;
100.            if (left < 0) {
101.                int flag = 0;
102.                for (int l = 0; l < count; l++) {
103.                    int sum = 0;
104.                    for (int h = 0; h < n; h++) {
105.                        if (nset[l][h] == 1) {
106.                            sum += qlist[h]+loss;
107.                        }
108.                    }
109.                    if (sum > len + loss) {
110.                        flag = 1;
111.                        break;
112.                    }
113.                }
114.                if(flag==0)
115.                    mn = mn > count ? count : mn;
116.            }
117.            //将剩余木块作为下一个木料的所有选取方法的输入
118.            Perm(mlist,0,0,left,count+1,nset,klist);
119.
```

```
120.                //取当前木料所对应的下一种木块的取法，每次取下一个木块，确保不
121.                //重复取
122.                Perm(list,level+1,i+1, m,count,set,klist);
123.            }
124.        }
125.        return;
126.    }
```

实验预习：

（1） 分析该问题及上面的程序，画出求解样例输入时对应的树或森林（可能由多棵树构成）。

（2） 根据样例输入，给出树或森林中每个结点对应的 list,lelel,i,m,set,klist 变量的值。

上机实验：

（3） 上机调试，在输入样例数据时实验验证你的分析过程及结果是否正确，给出部分调试结果。

（4） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

3.2 穷举法时间复杂度分析

穷举法需要罗列出问题的所有解并判断这些解是否符合问题的解，因此其效率相对比较低下。

例 3.3 分析例 3.1 算法的时间复杂度。

分析：

主程序中，行 8，9，10，15 的时间复杂度为 O(1)，行 11-12 的时间复杂度为 O(n)，行 14 的时间复杂度为 P(0)。其中：

P(0)=2P(1)+1　　且 P(n)=O(n)

因此 $P(0)=2P(1)+1=2^2P(2)+2^2-1=2^3P(3)+2^3-1=\cdots=2^nP(n)+2^n-1=O(n2^n)$

因此整个程序的时间复杂度为 $T(n)= O(n2^n)$。

实验 3.2 分析例 3.2 程序的时间复杂度。要求：

实验预习：

（1） 分析程序的时间复杂度，写出分析过程。

（2） 编写程序求 n 分别为 5，6，7，8，9 时函数 Perm 的运行时间，并保存到文件中。

上机实验：

（3） 上机执行步骤（2）编写的程序，并根据执行结果绘制相应的曲线。

（4） 比较（1）的分析结果及程序的实际运行时间，撰写实验报告。实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

3.3 穷举法的应用

穷举法由于其效率低下，因此并不提倡使用这种算法来解决实际应用问题。但由于很多算法均是从穷举法演变改进而来，因此了解如何利用穷举法解决实际应用问题也是十分重要的。

例 3.4 在 n 个不同的数中取 h 个数，求其所有的取法。

分析：

假设有 5 个数，分别为 1,2,3,4,5，希望从其中取任意 3 个数，得到的组合为：{1,2,3}，{1,2,4}，{1,2,5}，{1,3,4}，{1,3,5}，{1,4,5}，{2,3,4}，{2,3,5}，{2,4,5}，{3,4,5} 共计 10 个组合结果。

通过前面例 3.1，3.2 可以发现，组合问题可以利用构造树的结构来获得。该组合可以通过构建一个深度为 3 的树来得到。其中树的第一层的结点有 1，2，3 三种选择；第二层的结点中，1 的孩子结点有：2，3，4 可选，2 的孩子结点有 3，4 可选，3 的孩子结点只有 5 可选；第三层中从左到右，2 的孩子有 3，4，5，3 的孩子有 4，5，4 的孩子有 5…,树的结构如图 3-6 所示。

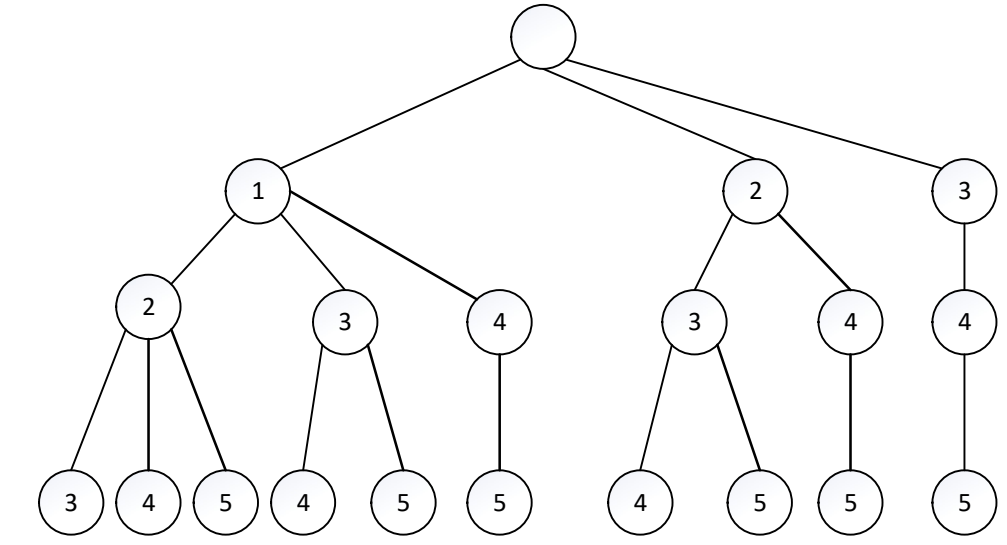


图 3-6 从 5 个中取 3 个数构成的树形结构

通过对前面例 3.2 的程序结构进行修改，可以得到解决该问题的程序如下：

```
1. #include "stdafx.h"
2. #include<conio.h>
3. #include<iostream>
```

```
4.  void Perm(int *,int,int,int, int*);
5.  using namespace std;
6.  int h;
7.  int main()
8.  {
9.      int *list,n,*slist;
10.     cin >> n>>h;
11.     list = new int[n];
12.     slist = new int[h];
13.     for (int i = 0; i < n; i++)
14.         cin >> list[i];
15.     Perm(list,0,0,n-1,slist);
16.     return 0;
17. }
    /*********************************************************************************************
    * list:原始序列
    * slist: 选取的数的组合
    * level:当前已取了几个数，也就是树的层数
    * k: 每一层取数的起始位置
    * m:list 中最后一个元素的位置，每一层取的数为 m-h+level+1 个
    *********************************************************************************************/
18. void Perm(int *list,int level, int k, int m,int *slist)
19. {
20.     int i;
21.     if(level==h){
22.         for(i=0;i<h;i++)
23.             printf("%d ",slist[i]);
24.         printf("\n");
25.         level = 0;
26.     }
27.     else{
28.         for(i=k;i<=m-h+level+1;i++){
29.             slist[level] = list[i];
30.             Perm(list,level+1,i+1,m,slist);
```

```
31.             }
32.         }
33.     return;
34. }
```

该算法的执行过程以及复杂度分析在此就不做进一步的介绍，大家可以根据前面的分析方法得出相应的结果。

实验 3.3 有 n（n>=1）个任务需要分配给 n 个人执行，每个任务只能分配给一个人，每个人只能执行一个任务，第 i 个人执行第 j 个任务的成本是 c[i][j]（1<=i ，j<=n）。求出总成本最小的一种分配方案。

输入：

输入 n 行，每行有 n 个正整数。第 i 行的第 j 个数表示第 i 个人执行第 j 个任务所需要的成本。

输出：

输出一个正整数，表示最小的总成本。

样例输入：

9 2 7 8
6 4 3 7
5 8 1 8
7 6 9 4

实验预习：

（1） 根据样例输入，采用穷举法求出一种总成本最小的分配方案，写出求解过程。

（2） 编写程序，采用穷举法求解上述问题。

上机实验：

（3） 上机调试，验证你的求解过程是否正确，并设计新的测试用例，对程序进行进一步验证，写出验证过程。

（4） 撰写实验报告，内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

实验总结：

本章通过具体介绍了组合和排列两种典型的穷举算法的求解过程以及程序框架，分析了其算法的求解过程，时间复杂度分析方法，以及如何设计穷举法解决实际问题。通过本章实验，学生可以正确理解穷举法的特点以及实际应用中的局限性，并为后续章节的学习打下一定的基础。

实验 4 分治法设计与实验

实验目的：

（1） 理解分治算法的求解过程。

（2） 通过范例学习分治策略设计技巧，学会分析分治算法的时间复杂度。

（3） 掌握用分治算法求解具体问题，了解其面临的瓶颈。

4.1 分治算法的求解过程

分治算法的主要求解思想是将规模为 n 的问题分解为 k 个规模较小的子问题，使这些子问题相互独立可分别求解，再将 k 个子问题的解合并成原问题的解。如子问题的规模仍很大，则反复分解直到问题小到可直接求解为止。

例 4.1 二分查找问题：给定一个 n 个元素有序的（升序）整型数组 nums 和一个目标值 target，写一个函数搜索 nums 中的 target，如果目标值存在则返回下标，否则返回 -1。

示例：

输入：nums = [-1,0,3,5,9,12], target = 9

输出：4（备注：9 出现在 nums 中并且下标为 4）

分析：

二分查找是一种基于比较目标值和数组中间元素的典型分治策略算法。

如果目标值等于中间元素，则找到目标值。

如果目标值较小，继续在左侧搜索。

如果目标值较大，则继续在右侧搜索。

其程序如下：

```
1.  int search(vector<int>& nums, int target)
2.  {
3.      int mid, left = 0, right = nums.size() – 1;
4.      while (left <= right) {
5.          mid = left + (right – left) / 2;
6.          if (nums[mid] == target) return mid;
7.          if (target < nums[mid]) right = mid – 1;
8.          else left = mid + 1;
9.      }
10.     return -1;
```

```
11.    }
```

上述程序是二分查找的经典写法，需要注意的三点：

（1） 循环退出条件，注意是 <= ，而不是 < 。

因为初始化 right 的赋值是 nums.length – 1，即最后一个元素的索引，而不是 nums.length。这二者可能出现在不同功能的二分查找中，区别是：前者相当于两端都闭区间 [left, right]，后者相当于左闭右开区间 [left, right]，因为索引大小为 nums.length 是越界的。本算法中使用的是前者 [left, right] 两端都闭的区间，因此循环退出条件是 <=。

（2） left 和 right 的更新：left = mid + 1，right = mid – 1。

有的二分查找算法更新是：left = mid，right = mid，差异在哪里？那是因为本算法的搜索区间是两端都闭的，即 [left, right]，当发现索引 mid 不是要找的 target 时，下一步当然是去搜索 [left, mid-1] 或者 [mid+1, right]，因为 mid 已经搜索过，应该从搜索区间中去除。

（3） mid 的取值：不用（left+right）/2 是为防止溢出。

二分查找的递归程序如下：

```
1.   int searchInternally(vector<int>& nums, int target,int low,int high)
2.   {
3.            if(low>high)
4.                 return -1;
5.            int mid=low+((high-low)>>1);
6.            int val=nums[mid];
7.            if(val>target)
8.                 return searchInternally(nums,target,low,mid-1);
9.            else if(val<target)
10.                return searchInternally(nums,target,mid+1,high);
11.           else
12.                return mid;
13.  }
```

实验 4.1 在未排序的数组中找到第 k 个最大的元素。

备注：你需要找的是数组排序后的第 k 个最大的元素，而不是第 k 个不同的元素。

数据示例 1:

输入: [3,2,1,5,6,4] 和 k = 2

输出: 5

数据示例 2:

输入: [3,2,3,1,2,4,5,5,6] 和 k = 4

输出: 4。

实验预习：

（1） 设计算法，分析样例输入时算法的执行过程。提示：可以采用基于快速排序的选择方法，先对原数组排序，再返回倒数第 k 个位置。

（2） 编写程序实现上述算法。

上机实验：

（3） 上机实验验证数据实例与程序的实际执行结果是否一致。

(4) 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

4.2 分治算法设计实例及时间复杂度分析

分治算法设计时要注意其适用的问题特征, 一般来讲需要考虑如下几个方面: 首先该问题的规模缩小到一定的程度就可以容易地解决, 其次该问题可以分解为若干个规模较小的相同问题, 且分解出的子问题的解可以合并为原问题的解, 还需要分解出的各个子问题是相互独立的。

例 4.2 采用分治策略编写程序实现循环赛日程表。设有 n=2k 个运动员要进行网球循环赛。现要设计一个满足以下要求的比赛日程表：

（1）每个选手必须与其它 n-1 个选手各赛一次

（2）每个选手一天只能赛一场

（3）循环赛进行 n-1 天。

分析：

循环赛日程表先将选手分为两部分，分别排序，再将两部分合并，合并时由于循环赛的规律得知直接将左上角的排序表复制到右下角，左下角的排序表复制到右上角即可。分成两部分时需要利用递归不断分下去直至只剩下一位选手。算法过程如下：

（1）分解: 将所有的选手分为两组, 各 n/2 人，分别安排各组的日程表. 如果每组人数大于 2, 可递归地对选手进行分割，直到每组剩下 2 个选手时。

（2）求子问题的解:直接安排这 2 个选手的比赛表。

（3）合并子问题成原问题:自底向上, 逐级将两组赛程对调，得到各组与另一组选手的赛程。

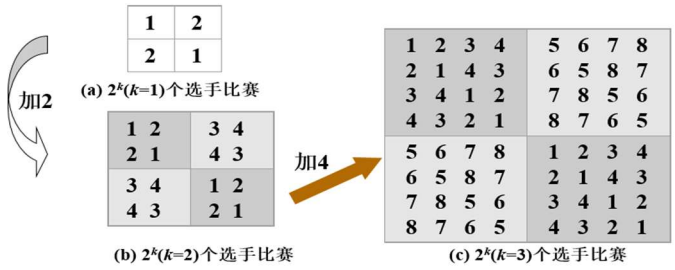


图 4-1，循环赛日程表设计思路

参考程序如下：

```
#include <iostream>
#include<math.h>
#include<iomanip>
using namespace std;

#define a_size 200
void cycleScheduleSort(int a[a_size][a_size],int k,int startRow);
void cs_merge(int a[a_size][a_size],int k,int startRow);
```

```
1.  int main()
2.  {
3.      int k;
4.      cout<<"Enter k : ";
5.      cin>>k;
6.      int m = pow(2,k);
7.      cout<<"Number of competitors : "<<m<<endl;
8.      int a[a_size][a_size];
9.      for(int i=1;i<m;i++){
10.         a[0][i]=i;
11.     }
12.     for(int i=1;i<m+1;i++){
13.         a[i][0]=i;
14.     }
15.
16.     cycleScheduleSort(a,k,1);
17.
18.     cout<<"A round-robin schedule :"<<endl;
```

```
19.         for(int i=0;i<m+1;i++){
20.             for(int j=0;j<m;j++){
21.                 if(a[i][j] != 0)
22.                     cout<<setiosflags(ios::left)<<setw(3)<<a[i][j];
23.                 else
24.                     cout<<setiosflags(ios::left)<<setw(3)<<" ";
25.             }
26.             cout<<endl;
27.         }
28.         return 0;
29.  }

30.  void cycleScheduleSort(int a[a_size][a_size],int k,int startRow){

31.         if(k>0){
32.             int m = pow(2,k-1);
33.             cycleScheduleSort(a,k-1,startRow);
34.             cycleScheduleSort(a,k-1,startRow+m);
35.             cs_merge(a,k,startRow);
36.         }
37.  }

38.  void cs_merge(int a[a_size][a_size],int k,int startRow){

39.         int m = pow(2,k-1);
40.         for(int i=startRow;i<m+startRow;i++){
41.             for(int j=0;j<m;j++){
42.                 a[i+m][j+m] = a[i][j];
43.             }
44.         }
45.         for(int i=startRow+m;i<startRow+m*2;i++){
46.             for(int j=0;j<m;j++){
47.                 a[i-m][j+m] = a[i][j];
48.             }
49.         }
```

```
50. }
```

实验 4.2　编写一个分治算法来搜索 m x n 矩阵 matrix 中的一个目标值 target，该矩阵具有以下特性：每行的元素从左到右升序排列。每列的元素从上到下升序排列。

示例 1：

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

输入：matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]], target = 5

输出：true

实验预习：

（1） 采用二分搜索策略实现问题求解程序，验证输入输出结果，并对下述三种设计算法的时间复杂度进行对比分析：

方法一：蛮力法，对于每一行可以像搜索未排序的一维数组——通过检查每个元素来判断是否有目标值。

方法二：二分法搜索策略，矩阵已经排过序，可以采用二分查找加快搜索效率（通过行列切片）。

方法三：搜索空间缩减策略，可以将已排序的二维矩阵划分为四个子矩阵，其中两个可能包含目标，其中两个肯定不包含，进一步提升问题的求解效率。

（2） 编写程序实现上述算法。

上机实验：

（3） 上机实验，验证样例输入时程序的执行过程以及算法复杂度分析结果与实际运行时间是否一致。

（4） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

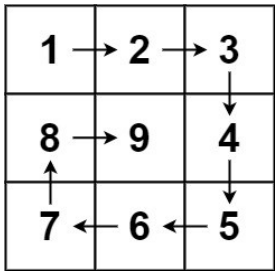
4.3 分治算法在实际问题中的应用

分治法在实际问题解决中，需要考虑两个关键问题：一是平衡子问题，最好使子问题的规模大致相同。也就是将一个问题划分成大小相等的 k 个子问题（通常 k＝2），这种使子问题规模大致相等的做法是出自一种平衡（Balancing）子问题的思想，它几乎总是比子问题规模不等的做法要好。二是独立子问题，各子问题之间相互独立，这个问题涉及到分治法的效率，如果各子问题是不独立的，则分治法要做许多不必要的工作，重复地解公共的子问题，此时虽然也可用分治法，但一般用动态规划较好。

例 4.3 给一个正整数 n，生成一个包含 1 到 n^2 所有元素，且元素按顺时针顺序螺旋排列的 n x n 正方形矩阵 matrix 。

输入：n = 3

输出：[[1,2,3],[8,9,4],[7,6,5]]。



分析：

通过观察可发现螺旋矩阵其实非常容易分层且每层的操作是相同的，我们只需要把公共操作提炼出来进行分治处理，并设置好递归终止条件即可。

1.分层：本题最终输出的一定是一个边长为 n 的正方形矩阵，我们可以把它想象成一个不停缩小的正方形，当 n 为奇数时，最中心将会剩余一个元素；当 n 为偶数时，最中心将会剩余一个边长为 2 的小正方形。由于我们对每一层的操作都是相同的，即沿着四个方向添加元素，因此直接在此基础上提炼出递归操作。

2.终止条件：当我们一圈一圈缩小递归到最中心时，当 n 为奇数，start 与 end 会收束到同一位置，即中心点，在此直接添加最大的那个数字（n^2）即可；当 n 为偶数时，start 会在最后一层超过 end,无需添加元素，直接返回。

3.因此本题采用顺时针画矩阵的过程:

a.填充上行从左到右

b.填充右列从上到下

c.填充下行从右到左

d.填充左列从下到上

由外向内一圈一圈这么画下去。

通过上述过程分析，可以发现在一个循环中，有如此多的边界条件，如果不

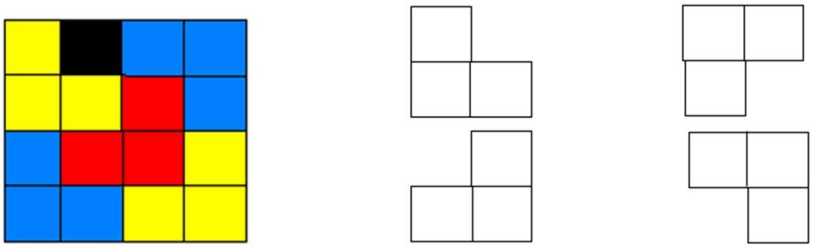
按照固定规则来遍历，就很容易出错。一般来讲，每画一条边都要坚持一致的循环不变量原则，比如我们根据统一的左闭右开原则来生成每条边的数字。

程序实现参考：

```
vector<vector<int>> generateMatrix(int n)
{
    vector<vector<int>> res(n, vector<int>(n, 0));
    int startx = 0, starty = 0;
    int loop = n / 2;
    int mid = n / 2;
    int count = 1;
    int offset = 1;
    int i,j;
    while (loop --) {
        i = startx;
        j = starty;
        // 填充上行从左到右(左闭右开)
        for (j = starty; j < starty + n – offset; j++) {
            res[startx][j] = count++;
        }
        // 填充右列从上到下(左闭右开)
        for (i = startx; i < startx + n – offset; i++) {
            res[i][j] = count++;
        }
        // 填充下行从右到左(左闭右开)
        for (; j > starty; j--) {
            res[i][j] = count++;
        }
        // 填充左列从下到上(左闭右开)
        for (; i > startx; i--) {
            res[i][j] = count++;
        }
        // 第二圈开始的时候，起始位置要各自加 1
        startx++;
        starty++;
```

```
32.                 // offset 控制每一圈里每一条边遍历的长度
33.                 offset += 2;
34.             }
35.             // 如果 n 为奇数的话，需要单独给矩阵最中间的位置赋值
36.             if (n % 2) {
37.                 res[mid][mid] = count;
38.             }
39.             return res;
40.         }
```

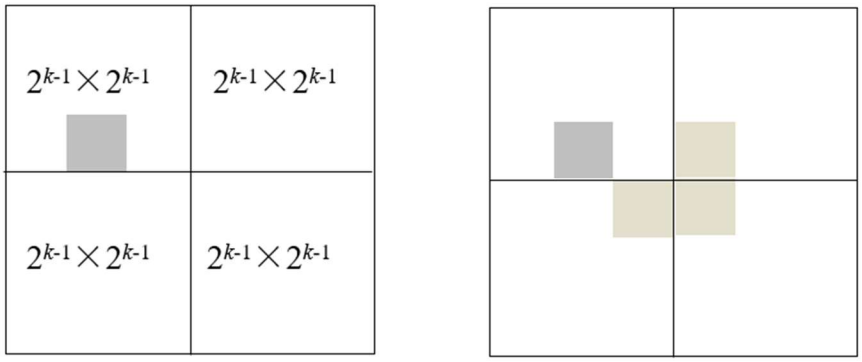
实验 4.3 棋盘覆盖问题：在一个 $2^k \times 2^k$ （k≥0）个方格组成的棋盘中，恰有一个方格与其他方格不同（图 a），称该方格为特殊方格。棋盘覆盖问题要求用图 b 所示的 4 种不同形状的 L 型骨牌覆盖给定棋盘上除特殊方格以外的所有方格，且任何 2 个 L 型骨牌不得重叠覆盖。



(a) k=2时的一种棋盘　　(b) 4种不同形状的L型骨牌

分析：

分治法求解棋盘覆盖问题的技巧在于划分棋盘，使划分后的子棋盘的大小相同，并且每个子棋盘均包含一个特殊方格，从而将原问题分解为规模较小的棋盘覆盖问题。



如上图所示，当 k>0 时，可将 $2^k \times 2^k$ 的棋盘划分为 4 个 $2^{k-1} \times 2^{k-1}$ 的子棋盘，这样划分后，由于原棋盘只有一个特殊方格，所以，这 4 个子棋盘中只有一个子棋盘包含该特殊方格，其余 3 个子棋盘中没有特殊方格。为了将这 3 个没有特殊

方格的子棋盘转化为特殊棋盘，以便采用递归方法求解，可以用一个 L 型骨牌覆盖这 3 个较小棋盘的会合处，从而将原问题转化为 4 个较小规模的棋盘覆盖问题。递归地使用这种划分策略，直至将棋盘分割为 1×1 的子棋盘。

实验预习：

（1） 设计算法求解该问题，写出算法的伪代码，并分析算法的复杂度。

参考设计思路：采用分治+递归设计策略，每次都对分割后的四个小方块进行判断，判断特殊方格是否在里面。这里的判断的方法是每次先记录下整个大方块的左上角（top left coner）方格的行列坐标，然后再与特殊方格坐标进行比较，就可以知道特殊方格是否在该块中。如果特殊方块在里面，这直接递归下去求即可，如果不在，这根据分割的四个方块的不同位置，把右下角、左下角、右上角或者左上角的方格标记为特殊方块，然后继续递归。在递归函数里，还要有一个变量 s 来记录边的方格数，每次对方块进行划分时，边的方格数都会减半，这个变量是为了方便判断特殊方格的位置。其次还要有一个变 nCount 来记录 L 型骨牌的数量。

（2） 根据分治算法思想，分析其求解过程，编写程序实现上述问题的求解。

上机实验：

（1） 调试程序，验证程序运行结果与算法设计分析结果是否一致。

（2） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

实验总结：

本章通过具体实例介绍了分治算法的分析求解过程，时间复杂度分析方法，以及如何设计分治算法解决实际问题。通过本章实验，学生可以正确理解分治算法的特点以及在实际应用中如何正确设计并使用分治算法。

实验 5 动态规划算法设计与实验

实验目的：

（1） 理解动态规划算法的求解过程。

（2） 分析动态规划算法的时间复杂度，比较动态规划算法与其他算法的效率差异。

（3） 学会如何利用动态规划算法求解具体问题，了解动态规划算法的局限性。

5.1 动态规划算法的求解过程

动态规划算法主要的特点是利用此前解决某子问题时的解来解决当前问题。也就是当前问题的解可能由若干个子问题构成，而其中一些子问题的解在此前解决某个问题时已经得到，当前问题可以直接利用此前子问题的解进行求解。

例 5.1 设 F(1)=1,F(2)=2， 采用动态规划算法求解方程 F(n)=F(n-1)+F(n-2)，分析求解过程。

分析：

一般而言，上述算法采用递归方式求解。其程序如下：

```
1.  int F(int n)
2.  {
3.      if(n==1)
4.          return 1;
5.      if(n==2)
6.          return 2;
7.      return F(n-1)+F(n-2);
8.  }
```

该程序在 n=7 时的求解过程如图 5-1 所示。从图可以看出，在求解过程，有很多项进行了重复的计算，比如求 F(n)时需要得到 F(n-2)的值，而 F(n-2)的值可在求解 F（n-1）时得到，因此上面的求解过程可以用图 5-2 所示的求解过程进行求解。图 5-2 对图 5-1 的求解过程进行了优化，在求解下一个子问题时利用了前一个子问题求解时得到的部分结果，从而优化了求解过程，当然也就提高了求解的时间效率。

如何将前面的程序修改为优化后的求解过程？因为每次计算时，构成 F(n)的 F(n-1)或者 F(n-2)在前面已经得到了相应的结果，因此可以设置一个变量，记录

对应的值。从而在后续计算中利用前面的结果进行计算，从而降低运算的复杂度。

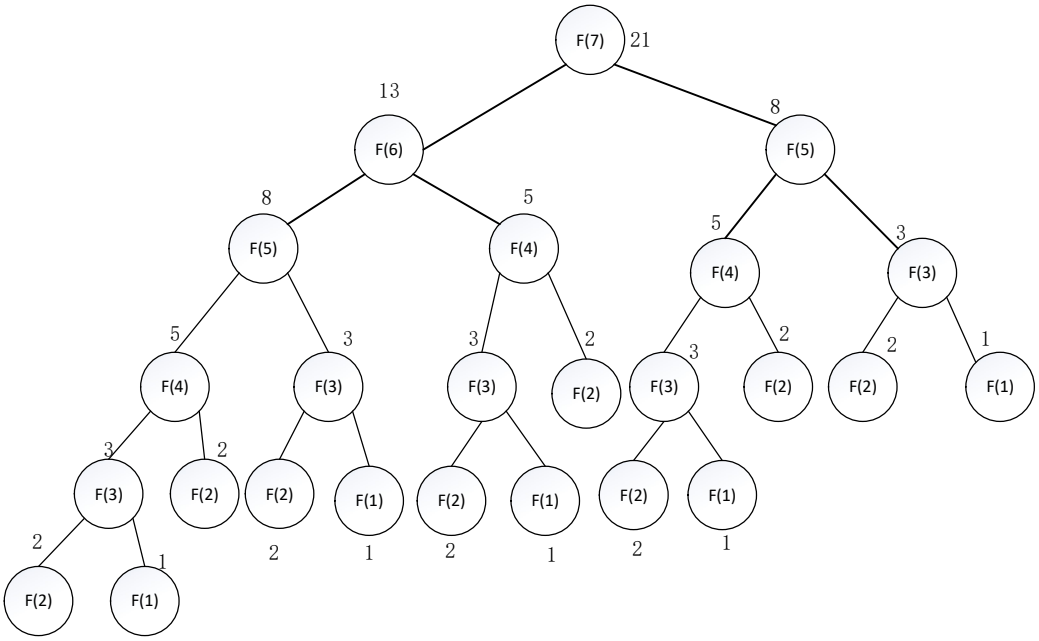


图 5-1 函数 F(n)=F(n-1)+F(n-2)以 n=7 为例的求解过程

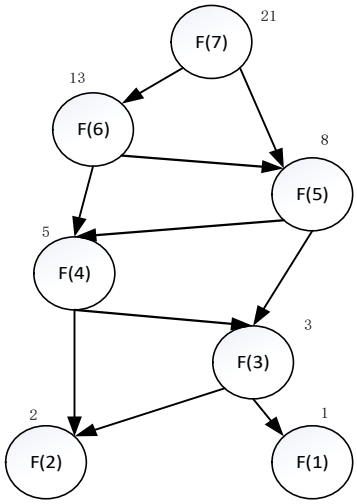


图 5-2 函数 F(n)=F(n-1)+F(n-2)优化后的求解过程

设 sValue[n]表示 F(n)的值，初始化 sValue[1]=1，sValue[2]=2,其他所有元素的值为 0，上面的程序可修改如下：

```
1.  int F(int n)
2.  {
3.      if(sValue[n])
4.          return sValue[n];
5.      return sValue[n]=F(n-1)+F(n-2);
6.  }
```

上述程序在求解 F(n)时，如果 F(n-1)和 F(n-2)的值前面已经求得，并保存在 sValue[n-1]和 sValue[n-2]中，则直接返回然后求和得到，从而避免了重复求解的过程。

基于上面的求解过程，也可以将上面的程序修改为下面的形式:

```
1.  int F(int n)
2.  {
3.      sValue[1]=1;
4.      sValue[2]=2;
5.   for(int i=3;i<=n;i++)
6.          sValue[i]=sValue[i-1]+sValue[i-2];
7.      return sValue[n];
8.  }
```

可以看到，该程序采用循环的方式实现，去除了递归调用，节省了递归调用所需要的存储空间的大小，时间复杂度为 O(n)。

在 n=7 时上述程序的执行过程如表 5-1 所示。

表 5-1 n=7 时函数 F(n)的求解过程

i	sValue	i	sValue	i	sValue
3	3	5	8	7	21
4	5	6	13		

实验 5.1 已知某问题采用动态规划算法求解的程序如下所示。

实验预习：

（1） 设计具体的测试用例分析该程序的执行过程，写出程序执行过程中主要变量的变化情况。

（2） 将该程序 dp 函数修改为采用递归调用实现，然后分析（1）中所生成的测试用例的执行过程。

上机实验：

（3） 上机验证（1）的分析结果与程序的实际执行结果是否一致。

（4） 上机验证（2）中的分析结果与程序实际执行结果是否一致。

（5） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

```
1.  #include <stdio.h>
2.  #include <string.h>
3.  int vis[101][101];
4.  int map[101][101];
```

```
5.  int R,C;
6.  int dp(int i,int j)
7.  {
8.      int max = 0;
9.      if(vis[i][j]>0) return vis[i][j];
10.     if(i-1>=0)
11.         if(map[i-1][j]<map[i][j])
12.             if(max<dp(i-1,j))   max = dp(i-1,j);
13.     if(i+1<R)
14.         if(map[i+1][j]<map[i][j])
15.             if(max<dp(i+1,j))   max = dp(i+1,j);
16.     if(j-1>=0)
17.         if(map[i][j-1]<map[i][j])
18.             if(max<dp(i,j-1))   max = dp(i,j-1);
19.     if(j+1<C)
20.         if(map[i][j+1]<map[i][j])
21.             if(max<dp(i,j+1))   max = dp(i,j+1);
22.     return vis[i][j] = max+1;
23. }
24. int main()
25. {
26.     int i,j,ans,max;
27.     scanf("%d%d",&R,&C);
28.     for(i=0;i<R;i++)
29.         for(j=0;j<C;j++)
30.             scanf("%d",&map[i][j]);
31.     max = 0;
32.     for(i=0;i<R;i++)
33.     {
34.         memset(vis[i],-1,sizeof(vis[i]));
35.         for(j=0;j<C;j++)
36.         {
37.             ans = dp(i,j);
38.             if(ans>max) max = ans;
```

```
39.             }
40.         }
41.         printf("%d\n",max);
42.         return 0;
43.  }
```

5.2 动态规划算法时间复杂度分析比较

动态规划算法利用此前的求解结果解决后续问题，从而提高算法的运算效率，降低算法运算的复杂度。

例 5.2 比较例 5.1 中不同算法的执行时间，分析算法的时间复杂度。

分析：采用估计法（具体可参照课程教案），可得到例 5.1 算法改进前的时间复杂度为：$O(\frac{(1+\sqrt{5})}{2}^n)$。在算法改进后采用动态规划算法的时间复杂度为 O(n)。

算法的执行时间曲线如图 5-3 所示。

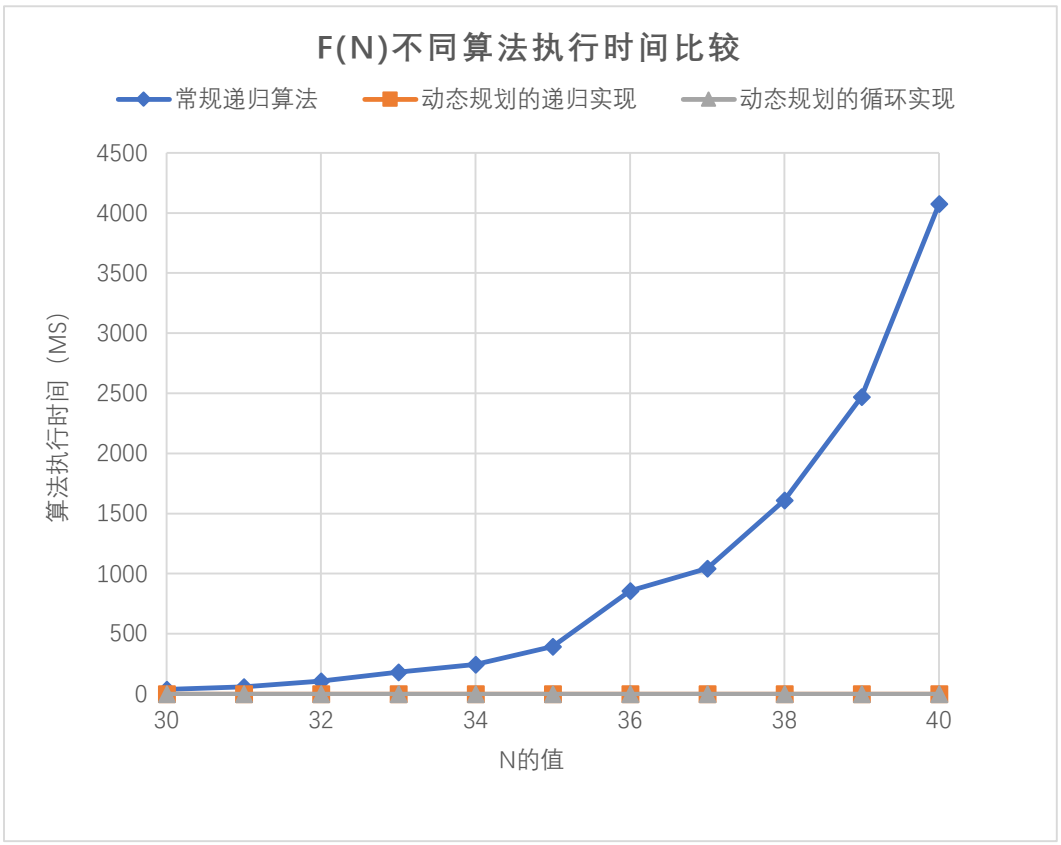


图 5-3 F(n)=F(n-1)+F(n-2)不同算法执行时间比较

图 5-3 所采用的软硬件环境与第 2 章图 2-1 测试环境相同。从图可以看出，采用常规的递归算法在 n 很小时其运算时间也很大，而同样的 n 值采用动态规划算法则小于 1ms 的执行时间。

实验 5.2　算法复杂度分析比较。

实验预习：

（1） 分析实验 5.1 中两种不同算法的时间复杂度。

上机实验：

（2） 绘制曲线比较两种算法在相同输入情况下的运行时间，并与（1）中的分析结果进行对比。

（3） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

5.3 动态规划算法在实际中的应用

从动态规划算法的求解过程可以看出，动态规划算法往往在求解当前问题的解时需要用到此前问题的解，因此利用动态规划算法求解实际问题时往往可采用逆推的方法。也就是可以假定已经求得当前问题的解，然后分析当前解的构成，明确当前解与此前规模较小问题的解之间的关系，从而得到相应的递推式或者递归调用形式。

例 5.3 某小孩有 N 块大小相同的方形积木，他希望用这 N 块积木来搭建不同的楼梯。楼梯各个台阶的积木数要求严格递减，也就是从上到下上一层的积木数必须小于下一层的积木数。且要求楼梯至少包含两层，每层楼梯至少一块积木。请问可以建造多少种不同的楼梯。

输入：包含若干组数据，每组数据 1 行，包含一个整数 N，表示总的积木的数量。0 表示输入结束。

输出：每组数据输出 1 行，包含一个整数，表示有多少种不同的搭建方法。

要求：

（1） 设积木数量为 i，楼梯从上到下第一层积木的数量为 j 时楼梯的搭建方法有 m[i][j]种，求 m[i][j]的表达式。

（2） 设积木数量为 i，楼梯从上到下最后一层的积木数量为 j 时楼梯的搭建方法有 h[i][j]，求 h[i][j]的表达式。

（3） 设积木数量为 i，f[i][j]表示最底层积木数量最大为 j 时的楼梯建造方法，求 f[i][j]的表达式。

（4） 分析积木数量为 7 时，问题的求解过程。

（5） 设计算法求解该问题，并分析算法的时间复杂度。

（6） 编写程序实现上述算法。

解：

（1） 可将楼梯看成两个部分，第一层为一部分，在第一层积木数确定的情况下，第二层到最底层（第二层也可能是最底层），每一层的积木总数为 i-j，且第二层的积木数可以为 j+1，j+2，…，i-j，并且要求 i-j>=j+1。因此如果将第二

层到最底层也看成是一个单独的楼梯，其对应的搭建方法将分别为：

m[i-j][j+1],m[i-j][j+2],…,m[i-j][i-j]

所以：

m[i][j]=m[i-j][j+1]+m[i-j][j+2]+…+m[i-j][i-j]

其中 m[i][i]=1,j<(i+1)/2,否则 m[i][j]=0。

上述表达式也就是在最顶层为 j 时的楼梯搭建方法数。

显然当积木数为 N 时总的楼梯搭建方法数为：

m[N][1]+m[N][2]+…+m[N][k]

其中 k=(N+1)/2-1

（2） 换一个角度，从下往上看，如果积木总数为 i，而最底层的积木数为 j，则其上各层总的积木数为 i-j,倒数第二层的积木数必须小于 j，因此可以为 j-1,j-2,…,1，因此可以得到如下的表达式：

h[i][j]=h[i-j][j-1]+h[i-j][j-2]+…+h[i-j][1]

其中 h[i][i]=1, i>j≥⌈(i+1)/2⌉,否则 h[i][j]=0。

上述表达表示最底层为 j 时的楼梯建造方法。

这样当有 N 块积木时的楼梯搭建方法总数为：

h[N][N-1]+h[N][N-2]+…+h[N][k]

其中 k=(N+1)/2-1

（3） 如果设 f[i][j]表示积木总数为 i，最底层的积木数最大为 j 时的楼梯搭建方法，很显然：

f[i][j]=h[i][j]+h[i][j-1]+h[i][j-2]+…+h[i][2]

也就是其搭建方法为最底层为 j 时的搭建方法数加上最底层为 j-1 时的搭建方法数直到最底层为 2 时的搭建方法数的总和,其中 j>=2。同时有：

f[i][2]=1;

如果 j 为 i，则表示最底层最大为 i 块时的搭建方法数，也就是我们要求的结果。当然考虑到必须有两层，当总数为 i 时，最底层最大为 i-1,因此构建楼梯的方法总数为

f[i][i-1]+1=f[i][i]-1

显然：

f[i][j-1]= h[i][j-1]+h[i][j-2]+…+h[i][2]

而

h[i][j]=h[i-j][j-1]+h[i-j][j-2]+…+h[i-j][1]=f[i-j][j-1]

所以：

f[i][j]=f[i][j-1]+f[i-j][j-1]

利用上式可以求得在积木数为 N 时的楼梯构造方法数。

（4） 在 N=7 时，采用（1）的方法，其计算过程如下：

m[3][1]=m[2][2]=1;m[3][2]=0;m[3][3]=1;

m[4][1]=1,m[4][2]=0;m[4][3]=0;m[4][4]=1;

m[5][1]=1;m[5][2]=1;m[5][3]=0;m[5][4]=0;m[5][5]=0;

m[6][1]=m[5][2]+m[5][3]+m[5][4]+m[5][5]=2;

m[6][2]=m[4][3]+m[4][4]=1;

m[7][1]=1+m[6][2]=2; m[7][2]=1;m[7][3]=1;

因此 N=7 时的总的楼梯搭建方法为 m[7][1]+m[7][2]+m[7][3]=4;

注意在上面的计算过程中，但利用上述推导的公式进行计算时，如果 i,j 不满足条件者 m[i][j]的值为零，计算时不再参与计算。

采用（2）的方法，其计算过程如下：

h[3][2]=1;

h[4][3]=1;

h[5][3]=1,h[5][4]=1;

h[6][3]=h[3][3]+h[3][2]=2,h[6][4]=1;h[6][5]=1;

h[7][4]=h[3][2]+h[3][3]=2; h[7][5]=1; h[7][6]=1;

同样 N=7 时总的搭建方法为 4 种。

同样可采用方法（3）求解 N=7 时的楼梯搭建方法。

思考：

若将构成楼梯的每块积木的数量看成相应的数字，显然构造的楼梯可以看成是 N 为若干个不同整数之和。这样该问题就变成了将 N 分解成若干个不同数之和的个数。请问如何实现上述算法？

（5） 如果采用方法（1）求解，则可采用如下的算法实现：

输入：积木数 N

输出：楼梯搭建方法数 sum

```
int Ladder(int n)
{
1    sum←0;
2    int** dp;
3    allocate memory for dp and initialize dp to 0;
4    dp[3][1] = 1;
5    dp[4][1] = 1;
6    dp[i][i]=1;
```

```
    //dp[i][j]:i 个积木，顶层为 j 个积木的方案数
7   for (int i = 5; i <= n; i++)
8     for (int j = 1; j <= (i + 1) / 2 - 1; j++)
9       for (int k = j + 1; k <= i - j; k++)
10        dp[i][j] += dp[i - j][k];
11   for (int i = 1; i <= (n + 1) / 2 - 1; i++)
12      sum += dp[n][i];
13   free dp memory;
14   return sum;
15  }
```

算法复杂度分析如下：

算法中第 1，2，4，5，6,14 行所需的时间复杂度为 0（1），算法第 3,8 行的时间复杂度为 0（n^2）,算法 7,11，12,13 行的时间复杂度为 0（n）,算法 9，10 行的时间复杂度为 0(n^3),因此整个算法的时间复杂度为 0（n^3）。

（6） 根据（5）所介绍的算法，完整程序如下：

```
#include “stdafx.h”
#include <cmath>
#include <iostream>
#include <algorithm>
int Ladder(int n);

using namespace std;
int main()
{
1    int n,sum;       //积木的个数
2    cin >> n;
3    sum = Ladder(n);
4    cout << sum << endl;
5    return 0;
}

int Ladder(int n)
{
6    int sum = 0;
```

```
7      int** dp;        //动规数组
8      dp = new int* [n + 1];
9      for (int i = 0; i < n + 1; i++)
10          dp[i] = new int[n + 1];
11     for (int i = 0; i < n + 1; i++)
12       for (int j = 0; j < n + 1; j++)
13           dp[i][j] = 0;
14     dp[3][1] = 1;
15     dp[4][1] = 1;
16     for (int i = 1; i <= n; i++)
17         dp[i][i] = 1;        //只有一层的情况
       //dp[i][j]:i 个积木，顶层为 j 个积木的方案数
18      for (int i = 5; i <= n; i++)
19         for (int j = 1; j <= (i + 1) / 2 - 1; j++)
20           for (int k = j + 1; k <= i - j; k++)
21               dp[i][j] += dp[i - j][k];
22      for (int i = 1; i <= (n + 1) / 2 - 1; i++)
23         sum += dp[n][i];
24      for (int i = 0; i < n + 1; i++)
25         delete dp[i];
26      delete[]dp;
27      return sum;
    }
```

实验 5.3 某城市四面环水，只有左下角和右上角有一座可以到达城市的桥。城市被分成 M*N 个方形的房子，房子之间可以通路，如图 5-4。随着城市的不断发展，原来的小房子逐渐被高楼大厦所取代，每座大厦可能会占用好几个小的房子，但仍保留原来的街道风格，如图 5-4 所示。

设左下角的坐标为（1，1），右上角坐标为（m,n），城市大厦采用左下角坐标和大厦占用的水平小房子数量和垂直小房子数量来描述。编写程序计算某人从左下角以最短的距离到到右上角一共有多少种不同的路径。

输入：

有多个测试用例。每个测试用例的第一行包含两个整数 m,n，表示原来的城市垂直和水平方向上有多少个小房子。第 2 行为整数 B，表示大厦的数量。其后

有 B 行，每一行包含 4 个整数 x,y,z,b，分别表示大厦左下角坐标以及大厦垂直和水平方向上占用的小房子的数量。当 m,n 都为 0 时，表示输入结束。

输出：每一组测试数据输出一个整数，也就是从左下角到右上角总的路径数。

样例输入：

5　6

1

2 4 3 2

0 0

样例输出：

192

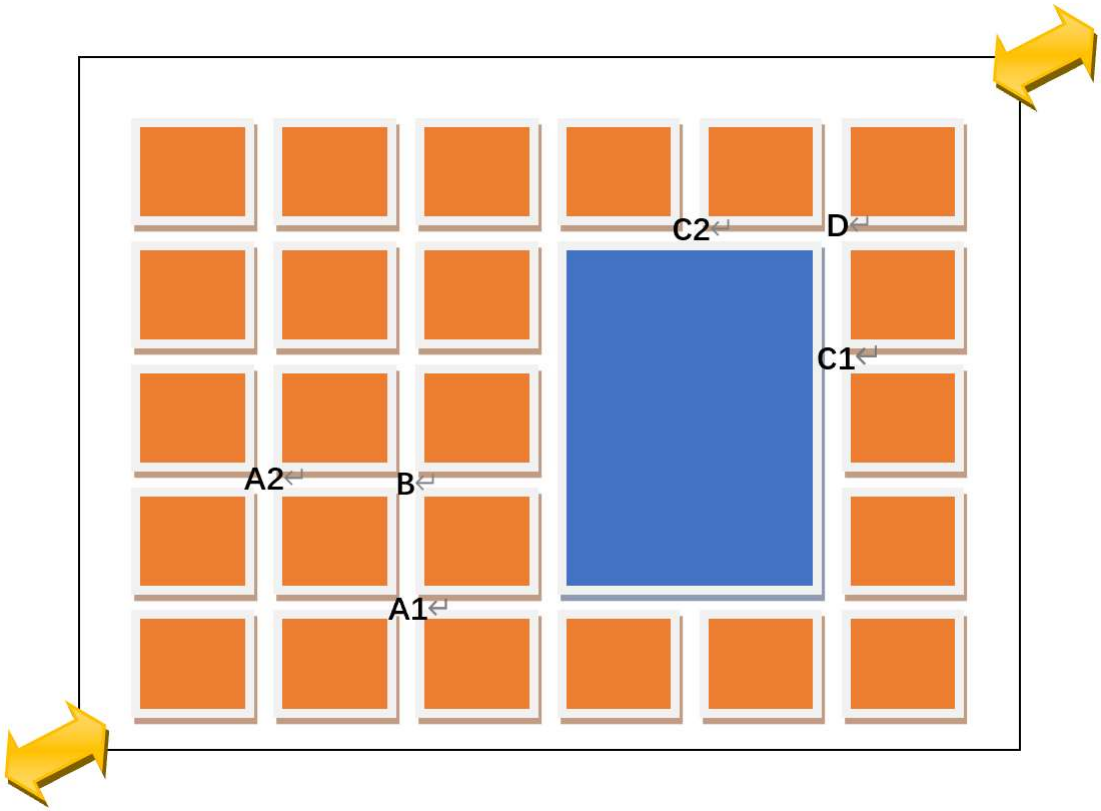


图 5-4 修建大厦之后的城市道路情况

实验预习：

（1） 图 5-4 中，设从左下角以最短的距离到达 A1 点的路径数为 X1，到达 A2 的路径数为 X2，求从左下角以最短距离到达 B 点的路径数为多少？

（2） 图 5-4 中，设从左下角以最短的距离到 C1 点的路径数为 Y1，到达 C2 的路径数为 Y2，求从左下角以最短距离到达 D 点的路径数为多少？

（3） 图 5-4 中，基于（1）的假设，求从左下角以最短路径到达 C1,C2 点的路径是多少？

（4） 设计算法求解该问题，写出算法的伪代码，并分析算法的复杂度。

（5） 根据算法，分析样例输入情况下的求解过程，写出求解过程中主要变量的变化过程。

（6） 编写程序求解上述问题。

上机实验：

（7） 调试程序，比对样例输入情况下程序运行结果与（5）中分析结果是否一致。

（8） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

实验 5.4 有一个二维矩阵，矩阵中元素有正也有负。定义子矩阵的和为其所有元素之和，最大子矩阵为子矩阵和值最大的子矩阵。

输入：

输入的第一行为N,表示矩阵的行数和列数。其后的N行每行包含N个元素，表示矩阵每行元素的值，元素之间用空格隔开。

输出：

输出一行，包含一个整数，表示其最大子矩阵的和。

样例输入：

```
 4
 0 -2 -7 0
 9  2 -6  2
-4  1 -4  1
-1 8  0 -2
```

实验预习：

（1） 将矩阵的某一行看成一个序列，设计算法求该序列的最大子序列。

（2） 将矩阵相邻两行对应列的元素相加形成一个新的序列，试分析该子序列中每一个元素的含义，其最大子序列的含义。

（3） 设计算法求解该问题，分析样例输入情况下算法的执行过程、主要变量的变化情况，并分析算法的复杂度。

（4） 编写程序实现（3）中的算法。

上机实验：

（5） 上机调试，验证程序执行过程与（3）中的执行过程分析是否一致。

（6） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

实验总结：

本章通过具体实例介绍了动态规划算法的分析求解过程，时间复杂度分析方法，与其他算法的效率差异，以及如何设计动态规划算法解决实际问题。通过本实验，学生可以正确理解动态规划算法的特点以及在实际应用中如何正确设计并使用动态规划算法。

实验 6 贪心算法设计与实验

实验目的：

(1) 理解贪心算法的求解过程。

(2) 通过范例学习贪心策略设计技巧。

(3) 掌握用贪心算法求解具体问题。

6.1 贪心算法的求解过程

贪心算法的主要求解思想是将问题的求解过程看作一系列选择（每次选择确定一个输入值），每次选择都是当前状态下的最好选择(局部最优解)。每作一次选择后，所求问题会简化为一个规模更小的子问题，从而通过每一步的最优解逐步达到整体最优解。

例 6.1 找零问题：当前有面值分别为 2 角 5 分，1 角，5 分，1 分的硬币，请给出找 n 分钱的最佳方案（要求找出的硬币数目最少）。

分析：

找零问题是典型的贪心问题，但是并不代表所有的找零都能用贪心算法找到最优解。只有满足贪心选择性质的找零才能找到最优解，本题满足贪心选择性质，直接先一直选面值最大的硬币，再一次减小即可。

参考实现程序如下：

```
#include <iostream>
using namespace std;

int main()
{
    int number;
    int remain;  //记录剩下的数额
    cout<<”Please enter the amount of change you want (in cents)：”;
    cin>>number;
    //建立 a 数组保存已有面值硬币，单位为分
    double a[4]={25,10,5,1};
    //建立 b 数组保存找回每个硬币的数量
    double b[4]={0,0,0,0};
```

```
11      for(int i=0;i<4;i++){

12          int x = 0;   //记录硬币数量
13          while(number>=a[i]){
14              x++;
15              number -= a[i];
16          }
17          b[i] = x;
18      }
19      cout<<”25 cents: “<<b[0]<<endl;
20      cout<<”10 cents: “<<b[1]<<endl;
21      cout<<”5  cents: “<<b[2]<<endl;
22      cout<<”1  cents: “<<b[3]<<endl;
23      return 0;
24  }
```

实验 6.1 给定一个以字符串表示的非负整数 num，移除这个数中的 k 位数字，使得剩下的数字最小。

注意:

num 的长度小于 10002 且 ≥ k。

num 不会包含任何前导零。

示例 1 :

输入: num = “1432219”, k = 3

输出: “1219”

解释: 移除掉三个数字 4, 3, 和 2 形成一个新的最小的数字 1219。

示例 2 :

输入: num = “10200”, k = 1

输出: “200”

解释: 移掉首位的 1 剩下的数字为 200. 注意输出不能有任何前导零。

实验预习：

(1) 设计算法求解上述问题，分析样例输入时算法的执行过程。

(2) 编写程序实现上述算法。

上机实验：

(3) 上机调试，验证数据输入输出实例与算法实现程序的实际执行结果是否一致。

（1） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

6.2 贪心算法设计实例分析

采用贪心算法设计求解问题需要考虑如下几个方面:

(1) 候选集合 C: 为了构造问题的解决方案，有一个候选集合 C 作为问题的可能解，即问题的最终解均取自于候选集合 C。例如，在付款问题中，各种面值的货币构成候选集合。

(2) 解集合 S: 随着贪心选择的进行，解集合 S 不断扩展，直到构成一个满足问题的完整解。例如，在付款问题中，已付出的货币构成解集合。

(3) 解决函数 solution: 检查解集合 S 是否构成问题的完整解。例如，在付款问题中，解决函数是已付出的货币金额恰好等于应付款。

(4) 选择函数 select: 即贪心策略，这是贪心法的关键，它指出哪个候选对象最有希望构成问题的解,选择函数通常和目标函数有关。例如,在付款问题中,贪心策略就是在候选集合中选择面值最大的货币。

(5) 可行函数 feasible: 检查解集合中加入一个候选对象是否可行，即解集合扩展后是否满足约束条件。例如，在付款问题中，可行函数是每一步选择的货币和已付出的货币相加不超过应付款。

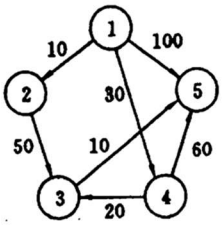
例 6.2 设计贪心算法实现单源最短路径问题

分析：

（1）问题描述：给定带权有向图 G=(V,E)， 其中每条边的权是一个非负实数.要计算从 V 的一点 v0(源)到所有其他各顶点的最短路长度，路长指路上各边权之和。

（2）算法思路：设最短路长已知的终点集合为 S,初始时 v0 属于 S， 其最短路长为 0,然后用贪心选择逐步扩充 S:每次在 V-S 中,选择路径长度值最小的一条最短路径的终点 x 加入 S。

（3）设计要点：单源最短路径的算法思想并不难，但是在实际编码过程中还是有很多小问题需要注意，首先，一定要新建数组存储路径变化，因为后面计算路径时会用到原数组，如果直接在原数组上更改后面就找不到原数据了，那么就会出现偏差。其次就是建议先写伪代码，判断的 if-else 语句比较多，容易搞混。一个简单的计算实例如下：



∞	10	∞	30	100
∞	∞	50	∞	∞
∞	∞	∞	∞	10
∞	∞	20	∞	60
∞	∞	∞	∞	∞

1) v1 → v2: 10
2) v1 → v3: 50
3) v1 → v4: 30
4) v1 → v5: 60

迭代	S	vi	d[2]	d[3]	d[4]	d[5]
初始	{1}		10	∝	30	100
1	{1,2}	2	*10*	60	30	100
2	{1,2,4}	4	*10*	50	*30*	90
3	{1,2,3,4}	3	*10*	*50*	*30*	60
4	{1,2,3,4,5}	5	*10*	*50*	*30*	*60*

参考程序如下：

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{   //原始带权有向图
    int a[5][5] = {{0,10,-1,30,100},
                   {-1,0,50,-1,-1},
                   {-1,-1,0,-1,10},
                   {-1,-1,20,0,60},
                   {-1,-1,-1,-1,0}};
    //单源最短路径图
    int b[5][5] = {{0,10,-1,30,100},
                    {0,0,0,0,0},
                    {0,0,0,0,0},
                    {0,0,0,0,0},
                    {0,0,0,0,0}};
    //创建数组红蓝点集
    int red[5] = {0,-1,-1,-1,-1};
    int red_N = 0;
    int blueToRed = 0;
    int blue[5] = {-1, 1, 2, 3, 4};
    int blue_N = 0;
```

```
20      for(int i=0; i<5-1; i++){
21          //从蓝点集中选出与源点最短路径的蓝点
22          //初始化随机蓝点
23          int blue_min;
24          for(int j=0; j<5; j++){
25              if(blue[j]>=0)
26                  blue_min = j;
27          }
28          for(int j=0; j<5; j++){
29              if(blue[j]>0 && b[i][j]>=0 && b[i][j]<b[i][blue_min]){
30              //blue[j]>0 表示 j 是蓝点，a[i][j]>0 表示 i,j 两点连通
31              blue_min = j;
32              }
33          }
34          //将蓝点集中与源点最短路径的点加入到红点集
35          red[++red_N] = blue_min;
36          //将该点从蓝点集中去除
37          blue[blue_min] = -1;
38
39          //更改加入后的数据
40          for(int j=0; j<5; j++){
41              //判断 j 点是否为蓝点
42              if(blue[j]>=0){                 //j 是蓝点
43                  if(a[blue_min][j]>0){   //新加入的蓝点与 j 点连通
44                      if(b[i][j]<0){      //上一步 i,j 两点不连通
45                          b[i+1][j] = b[i][blue_min]+a[blue_min][j];
46                      }
47                      //上一步 i,j 两点连通且加起来路径比原来短
48                      else if(b[i][blue_min]+ a[blue_min][j] < b[i][j]){
49                          b[i+1][j] = b[i][blue_min]+a[blue_min][j];
50                      }
51                      else{   //上一步 i,j 两点连通但加起来路径比原来长
52                          b[i+1][j] = b[i][blue_min]+a[blue_min][j];
53                      }
```

```
54                  }
55                  else{   //新加入蓝点与 j 点不连通
56                      b[i+1][j] = b[i][j];
57                  }
58              }
59              else{   //j 不是蓝点
60                  b[i+1][j] = b[i][j];
61              }
62          }
63      }
64      //输出原始带权有向图的邻接矩阵图
65      cout<<”Original:”<<endl;
66      //输出第一行
67      cout<<setw(3)<<”  “;
68      for(int i=0; i<5; i++)
69          cout<<setw(3)<<i;
70      cout<<endl;
71      for(int i=0; i<5; i++){
72          //输出行号点数
73          cout<<setw(3)<<i;
74          for(int j=0; j<5; j++){
75              cout<<setw(3)<<a[i][j];
76          }
77          cout<<endl;
78      }
79      cout<<endl;
80      //输出最终结果
81      cout<<”Changed:”<<endl;
82      //输出第一行
83      cout<<setw(3)<<”  “;
84      for(int i=0; i<5; i++)
85          cout<<setw(3)<<i;
86      cout<<endl;
87      for(int i=0; i<5; i++){
```

```
88          //输出行号点数
89          cout<<setw(3)<<i;
90          for(int j=0; j<5; j++){
91              cout<<setw(3)<<b[i][j];
92          }
93          cout<<endl;
94      }
95      return 0;
96  }
```

实验 6.2　贪心法求解加油问题：在一条环路上有 N 个加油站，其中第 i 个加油站有汽油 gas[i]升。你有一辆油箱容量无限的的汽车，从第 i 个加油站开往第 i+1 个加油站需要消耗汽油 cost[i]升。你从其中的一个加油站出发，开始时油箱为空。如果你可以绕环路行驶一周，则返回出发时加油站的编号，否则返回 -1。

示例　1:

输入:

gas　= [1,2,3,4,5]

cost = [3,4,5,1,2]

输出: 3

解释:

从 3 号加油站(索引为 3 处)出发，可获得 4 升汽油。此时油箱有 = 0 + 4 = 4 升汽油

开往 4 号加油站，此时油箱有 4 - 1 + 5 = 8 升汽油

开往 0 号加油站，此时油箱有 8 - 2 + 1 = 7 升汽油

开往 1 号加油站，此时油箱有 7 - 3 + 2 = 6 升汽油

开往 2 号加油站，此时油箱有 6 - 4 + 3 = 5 升汽油

开往 3 号加油站，你需要消耗 5 升汽油，正好足够你返回到 3 号加油站。

因此，3 可为起始索引。

实验预习：

（1） 设计算法，分析样例输入时算法的执行过程，写出分析过程。

（2） 编写程序实现上述算法。

上机实验：

（3） 上机测试，验证输入输出结果。

（4） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

6.3 贪心算法在实际问题中的应用

在实际应用中，贪心法主要用于解决具备贪心选择和最优子结构性质的最优化问题。贪心选择性质是指所求问题的整体最优解可以通过一系列局部最优的选择，即贪心选择来达到。这是贪心算法可行的第一个基本要素，也是贪心算法与动态规划算法的主要区别；最优子结构性质指问题的整体最优解中包含着它的子问题的最优解。贪心算法通常以自顶向下的方式进行，以迭代的方式作出相继的贪心选择，每作一次贪心选择就将所求问题简化为规模更小的子问题。

例 6.3 课程表问题：有 n 门不同的在线课程，他们按从 1 到 n 编号。每一门课程有一定的持续上课时间（课程时间）t 以及关闭时间第 d 天。一门课要持续学习 t 天直到第 d 天时要完成，你将会从第 1 天开始。给出 n 个在线课程用（t, d）对表示，设计算法找出最多可以修几门课。

输入：[[100, 200], [200, 1300], [1000, 1250], [2000, 3200]]

输出：3

解释：

这里一共有 4 门课程，但是你最多可以修 3 门：

首先，修第一门课时，它要耗费 100 天，你会在第 100 天完成，在第 101 天准备下门课。

第二，修第三门课时，它会耗费 1000 天，所以你将在第 1100 天的时候完成它，以及在第 1101 天开始准备下门课程。

第三，修第二门课时，它会耗时 200 天，所以你将会在第 1300 天时完成它。

第四门课现在不能修，因为你将会在第 3300 天完成它，这已经超出了关闭日期。

分析：

首先先根据结束时间进行从小到大排序，然后对每个课程的时间依次相加，如果有课程的时间超过了结束时间，那么从前面的课程中去除一个耗时最长的课程，最后统计大顶堆中的课程个数。

程序实现参考：

```
1   class Solution {
2     public:
3         static bool cmp(vector<int> &a, vector<int> &b)
4         {
```

```
5             return a[1] < b[1];
6           }
7         int scheduleCourse(vector<vector<int>>& courses) {
8             sort(courses.begin(), courses.end(), cmp);
9             int t = 0;
10            priority_queue<int, vector<int>, less<int>> q;
11            for(int i = 0; i < courses.size(); i++)
12            {
13              t += courses[i][0];
14              q.push(courses[i][0]);
15              if(t > courses[i][1])
16              {
17                  auto x = q.top();
18                  q.pop();
19                  t -= x;
20              }
21          }
22          return q.size();
23        }
24  };
```

实验 6.3 要求贪心算法设计实现一个任务调度器：字符数组 tasks 表示 CPU 需要执行的任务列表，其中每个字母表示一种不同种类的任务。任务可以以任意顺序执行，并且每个任务都可以在 1 个单位时间内执行完。在任何一个单位时间，CPU 可以完成一个任务，或者处于待命状态。另外，两个相同种类的任务之间必须有长度为整数 n 的冷却时间，因此至少有连续 n 个单位时间内 CPU 在执行不同的任务，或者处于待命状态。请计算完成所有任务所需要的最短时间 。

示例 1:

输入: tasks = [“A”,”A”,”A”,”B”,”B”,”B”], n = 2

输出: 8

解释: A -> B -> (待命) -> A -> B -> (待命) -> A -> B

在本示例中，两个相同类型任务之间必须间隔长度为 n = 2 的冷却时间，而执行一个任务只需要一个单位时间，所以中间出现了（待命）状态。

示例 2:

输入：tasks = [“A”,”A”,”A”,”B”,”B”,”B”], n = 0

输出：6

解释：在这种情况下，任何大小为 6 的排列都可以满足要求，因为 n = 0

[“A”,”A”,”A”,”B”,”B”,”B”]

[“A”,”B”,”A”,”B”,”A”,”B”]

[“B”,”B”,”B”,”A”,”A”,”A”]

...

实验预习：

（1） 设计贪心算法，分析样例输入时算法的执行过程，写出分析过程。

（2） 编写程序实现该算法。

上机实验：

（3） 采用贪心法设计实现程序并上机调试，验证数据输入输出实例与算法实现程序的实际执行结果是否一致。

（4） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

实验总结：

本章通过具体实例介绍了贪心算法的分析求解过程，以及如何设计贪心算法解决实际问题。通过本章实验，学生可以正确理解贪心算法的思想、过程以及在实际应用中如何有效设计并使用贪心算法。

实验 7 回溯法设计与实验

实验目的：

(1) 理解回溯法的求解过程。

(2) 分析回溯法的时间复杂度，比较回溯法算法与其他算法的时间效率差异。

(3) 学会如何利用回溯法求解具体问题,了解动回溯法的应用范围及在实际应用中的局限性。

7.1 回溯法求解过程分析

回溯法是一种广度遍历方法，是在穷举法的基础上，在深度遍历过程中通过约束函数和/或限界函数去除不必要的分支，从而提高搜索的速度。

例 7.1 0-1 背包问题的回溯法求解过程分析

设有 n 个物品，每个物品的重量为 wi 价值为 pi，背包的容量为 c,希望将其中的部分物品装入背包中，在不超过背包容量的情况下，希望物品的总价值最大。

输入：

输入第一行为物品数量 n 以及背包容量 c,第 2 行到第 n+1 行，每行有两个整数 wi,pi,分别表示物品的重量以及价值。

输出：

输出一行，包含一个整数 v，表示背包所能装物品的最大价值。

样例输入：

6 80

40 50

60 70

30 40

20 30

50 60

10 20

样例输出：

110

求解该问题的程序如下：

```
#include “stdafx.h"
#include “iostream"
using namespace std;
template <class Typep, class Typew>
```

```
Typep Knapsack(Typep [], Typew [], Typew ,int);
template <class Typew,class Typep>
class Knap{
    friend Typep Knapsack(Typep [], Typew [], Typew, int);
public:
    Typep Bound(int i);
    void Backtrack(int i);
    Typew c;    //背包容量
    int n;      //物品数
    Typew *w;   //物品重量数组
    Typep *p;   //物品价值数组
    Typew cw;   //当前重量
    Typep cp;   //当前价值
    Typep bestp; //当前最优价值
};

template <class Typep, class Typew>
class Object{
    friend Typep Knapsack(Typep [], Typew [], Typew ,int);
public:
    int operator<(Object a)const
    {
        return d>=a.d;
     }
     int ID;
     float d;
};

//按单位价值从大到小排序
template <class Typep, class Typew>
void Sort(class Object<Typep,Typew> *Q, int n);

int main()
{
```

```
1   int *p,*w,c,n,bestp;
2   cin>>n>>c;
3   p=new int[n+1];
4   w=new int[n+1];
5   for(int i=1;i<=n;i++){
6        cin>>w[i]>>p[i];
7    }
8   bestp=Knapsack(p,w,c,n);
9   cout << bestp << endl;
10  return 0;
11  }

    template <class Typew, class Typep>
    void Knap<Typew,Typep>::Backtrack(int i)
12  {
      //如果已经搜索到最后一个物品，则输出最优价值
13    if(i>n){
14      bestp=cp;
15      return;
16    }
        //如果已装物品的重量+当前待装物品的重量小于背包的容量，
        //则将物品装入其中
17    if(cw+w[i]<=c){
18       cw+=w[i];
19       cp+=p[i];
20       Backtrack(i+1);
21       cw-=w[i];
22       cp-=p[i];
23     }
      //如果以i+1为根的子树上所有物品的价值的最大值比当前找到的最大价值要大
      //则有必要继续搜索该子树。
24    if(Bound(i+1)>bestp)
25       Backtrack(i+1);
26  }
```

```
    //计算当前结点处的上限，此处采用的实际上是贪心算法。
    //可以证明通过该算法获得的值在一般情况下是不能达到的
    template <class Typew, class Typep>
    Typep Knap<Typew,Typep>::Bound(int i)
27  {
28    Typew cleft=c-cw;   //背包所能承受的剩余的重量
29    Typep b=cp;         //当前价值
      //在物品数量小于总的物品数量以及物品重量小于背包的剩余重量时，
      //将该物品装入背包中，并增加相应的价值
30    while(i<=n && w[i]<=cleft){
31        cleft-=w[i];
32        b+=p[i];
33        i++;
34    }
      //如果所装物品的数量小于整个物品的个数，则可装入第i个物品的部分，
      //从而达到背包所能装入的物品的最大价值
35    if(i<=n)
36       b+=p[i]*cleft/w[i];
37    return b;
38  }

template <class Typep, class Typew>
Typep Knapsack(Typep p[], Typew w[], Typew c, int n)
39  {
40    Typew W=0;
41    Typep P=0;
42    Object<Typep,Typew> *Q=new Object<Typep,Typew>[n+1];
      //计算单位重量的价值，并计算总的价值和总的重量
43    for(int i=1;i<=n;i++){
44       Q[i].ID=i;
45       Q[i].d=1.0*p[i]/w[i];
46       P+=p[i];
47       W+=w[i];
```

```
48      }
        //如果总的重量小于背包所能承受的重量，则返回总的价值
49      if(W<=c)
50          return P;
        //按照单位价值的大小从大到小排序
51      Sort(Q,n);
        //按照单位价值的大小从大到小的排列，并将其价值和重量并保存到p和w中
52      Knap<Typew,Typep> K;
53      K.p=new Typep[n+1];
54      K.w=new Typew[n+1];
55      for(int i=1;i<=n;i++){
56          K.p[i]=p[Q[i].ID];
57          K.w[i]=w[Q[i].ID];
58      }
59      K.cp=0;
60      K.cw=0;
61      K.c=c;
62      K.n=n;
63      K.bestp=0;
    //搜索最优值
64      K.Backtrack(1);
65      delete [] Q;
66      delete [] K.w;
67      delete [] K.p;
68      return K.bestp;
69  }

    template <class Typep, class Typew>
    void Sort(class Object<Typep,Typew> *Q, int n)
70  {
71      int k;
        //直接插入排序算法，按从大到小的顺序排序
72      for(int i=2;i<=n;i++){
73          if(Q[i]<Q[i-1]){
```

```
74              Q[0]=Q[i];
75              for(k=i-1;!(Q[k]<Q[0]);k--){
76                  Q[k+1]=Q[k];
77               }
78              Q[k+1]=Q[0];
79          }
80    }
81    return;
82  }
```

要求：

（1） 分析在样例输入时函数 Sort 执行过程及其结果,并实验验证分析结果是否与实际运行结果一致。

（2） 分析在样例输入时函数 K.Backtrack(1)的执行过程及其结果。

（3） 分析函数 Bound(i+1)的作用。

解：(1) 函数 Sort 采用直接插入排序算法对 Q 中所有元素进行排序，初始时，Q 中元素的值如表 7-1 所示。

表 7-1 Q 中元素的初始值（d 值精确到小数点后两位）

Q	Q[1]	Q[2]	Q[3]	Q[4]	Q[5]	Q[6]
ID	1	2	3	4	5	6
d	1.25	1.17	1.33	1.50	1.20	2.00

运行程序到上面程序第 40 行，打印 Q 的值，如图 5-1 所示。

名称	值	类型
Q[1]	{ID=1 d=1.25000000 }	Object<int,int>
Q[2]	{ID=2 d=1.16666663 }	Object<int,int>
Q[3]	{ID=3 d=1.33333337 }	Object<int,int>
Q[4]	{ID=4 d=1.50000000 }	Object<int,int>
Q[5]	{ID=5 d=1.20000005 }	Object<int,int>
Q[6]	{ID=6 d=2.00000000 }	Object<int,int>

图 7-1 函数 Sort 运行之前 Q 的初始值

Sort 函数执行过程中，变量的变化情况如表 7-2 所示。

表 7-2 Sort 函数执行过程中主要变量的变化情况（d 值精确到小数点后两位）

i	Q[0]	Q[1]	Q[2]	Q[3]	Q[4]	Q[5]	Q[6]
2		1	2	3	4	5	6
		1.25	1.17	1.33	1.50	1.20	2.00
3	3	3	1	2	4	5	6
	1.33	1.33	1.25	1.17	1.50	1.20	2.00

表 7-2（续表）

i	Q[0]	Q[1]	Q[2]	Q[3]	Q[4]	Q[5]	Q[6]
4	4	4	3	1	2	5	6
	1.50	1.50	1.33	1.25	1.17	1.20	2.00
5	5	4	3	1	5	2	6
	1.20	1.50	1.33	1.25	1.20	1.17	2.00
6	6	6	4	3	1	5	2
	2.00	2.00	1.50	1.33	1.25	1.20	1.17

执行结束后 Q 中元素的值如图 7-2 所示。



图 7-2 函数 Sort 运行之后 Q 的初始值

从图 7-1，7-2 以及表 7-1，7-2 可以看出，分析结果与程序的实际运行结果一致。

（2）函数 Backtrack 采用递归调用方式执行，可用树形结构描述其执行过程中变量的变化情况，如图 7-3 所示。

（3）Bound(i+1)函数采用贪心算法，计算搜索树中某个结点可能的最大价值。也就是根据已放入背包中物品的价值和剩余容量可能的最大价值来计算当前状态下该结点可能的最大价值。剩余重量可能的最大价值采用贪心算法求解，首先所有物品按照单位价值比从大到小进行排序，根据剩余物品和剩余容量，取单位价值比最大的物品放入背包中，确保剩余容量全部用完所能得到的物品的最大价值。

Bound 函数作为限界函数，可以极大的提高搜索的效率。从程序执行过程可以看出，当搜索树中后续结点的可能最大价值小于此前已求得的背包的最大价值时，可以不再继续搜索后续的结点，从而达到剪枝的效果。在样例输入情况下，结点 2 的所有右子树均通过 Bound 函数进行了剪枝，极大的提高了运算效率。

当然上述算法也可以进行进一步的优化，从而进一步提升求解的效率。

思考：

分析上述算法的求解过程，考虑 Bound 函数的使用，是否可以进一步提高搜索的速度，提升算法效率？

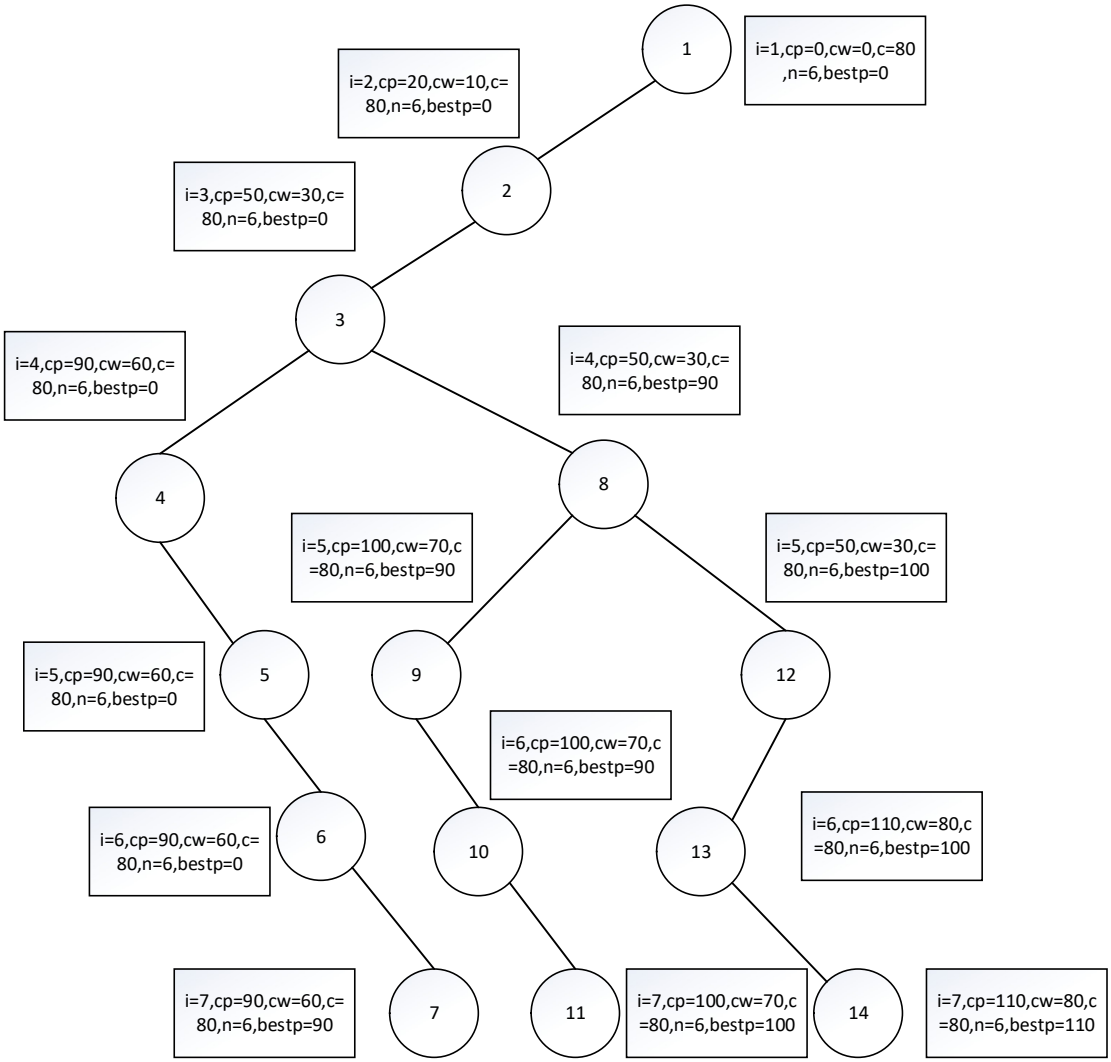


图 7-3 函数 Backtrack 的执行过程

实验 7.1 旅行商问题

某旅行商希望从某城市出发经过一系列的城市最后再回到出发的城市。这些城市之间均可直航，他希望只经过这些城市一次且旅行的总线路最短。设有 n 个城市，城市的编号从 1 到 n。

输入：

输入第一行为整数 n，表示城市的数量。其后 1 行，有 n(n-1)/2 个整数，用空格隔开，表示城市之间的距离。其中的第 1 个数表示城市 1 和城市 2 之间的距离，第 2 个数表示城市 1 和城市 3 之间的距离,…，第 n-1 个数表示城市 1 和城市 n 之间的距离，第 n 个数表示城市 2 和城市 3 之间的距离,…,依次类推。城市 i 和城市 j 之间的距离和城市 j 与城市 i 之间的距离相同。

输出：

输出 1 个整数，表示从城市 1 出发经过所有城市一次最后回到城市 1 的最短路径。

样例输入：（如图 7-4 所示）

5

4 8 6 5 3 2 10 8 7 3

样例输出：

21

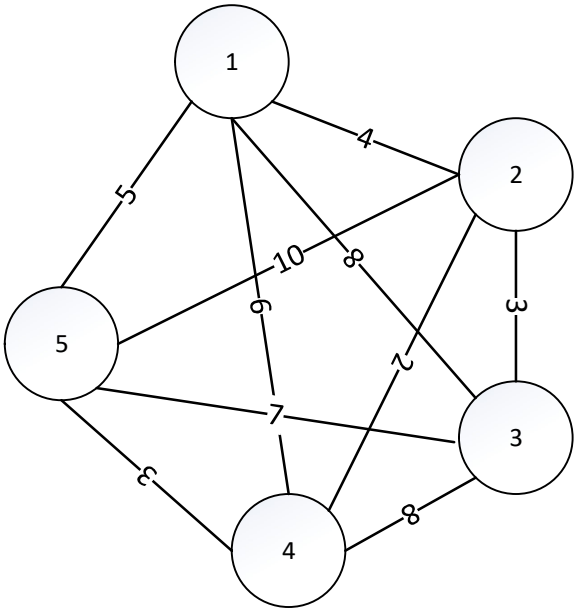


图 7-4 城市之间的连接关系及距离

求解上述问题的程序如下：

```
#include "stdafx.h"
#include "iostream"
using namespace std;
template <class Type>
class Traveling{
        friend Type Tsp(int **, int[],int, Type);
public:
    void Backtrack(int i);
    //n→图G的顶点数, x→存放当前解的结点编号
    //bestx→存放最优解的结点编号
    int n, *x, *bestx;
    //a→图G的邻接矩阵， cc→当前已求得的费用
    // bestc→当前已求的的最佳路径的最优值
    Type **a, cc, bestc;
```

```
};
template <class Type>
Type Tsp(Type **a, int v[], int n);
int main()
{
1   int **a,*v, n,bestc;
2   cin>>n;
3   v= new int[n+1];
4   a=new int* [n+1];
5   for(int i=0;i<=n;i++)
6       a[i]=new int [n+1];
7   for(int i=1;i<=n;i++)
8      for(int j=i+1;j<=n;j++){
9          cin>>a[i][j];
10         a[j][i]=a[i][j];
11      }
12    bestc=Tsp(a,v,n);
13    return 0;
14  }
    template <class Type>
    void Traveling <Type>::Backtrack(int i)
15  {
16     int temp;
17     if(i==n){
18         if(a[x[n-1]][x[n]]!=0 && a[x[n]][x[1]]!=0 &&
19              (cc+a[x[n-1]][x[n]]+a[x[n]][x[1]]<bestc || bestc ==0)){
                //循环将当前的最优路径保存到bestx中
20             for(int j=1;j<=n;j++)
21                   bestx[j]=x[j];
22             bestc=cc+a[x[n-1]][x[n]]+a[x[n]][x[1]];
23           }
24      }
25      else{
```

```
            //在剩余的顶点中寻找满足条件的结点，因此j是从i号结点开始
26          for(int j=i;j<=n;j++)
27             if(a[x[i-1]][x[j]]!=0 &&
28               (cc+a[x[i-1]][x[j]]<bestc || bestc==0)){
29                   temp=x[i];
30                   x[i]=x[j];
31                   x[j]=temp;
32                   cc+=a[x[i-1]][x[i]];
33                   Backtrack(i+1);
34                   cc-=a[x[i-1]][x[i]];
35                   temp=x[i];
36                   x[i]=x[j];
37                   x[j]=temp;
38               }
39         }
40     }
    template <class Type>
    Type Tsp(Type **a, int v[], int n)
41  {
42      Traveling <Type> Y;
        //初始化Y
43      Y.x=new int[n+1];
        //存放x的初始值
44      for(int i=1;i<=n;i++)
45         Y.x[i]=i;
46      Y.a=a;
47      Y.n=n;
48      Y.bestc=0;
49      Y.bestx=v;
50      Y.cc=0;
51      Y.Backtrack(2);
52      delete [] Y.x;
53    return Y.bestc;
54  }
```

实验预习：

（1） 分析函数 Backtrack 在样例输入情况下的执行过程以及执行过程中变量的变化情况。

（2） 分析程序 28 行表达式 cc+a[x[i-1]][x[j]]<bestc || bestc==0 的作用，如何修改该表达式进一步提高搜索效率。

上机实验：

（3） 给出在样例输入下函数 Backtrack 的实际执行结果，并与（1）中的分析结果进行比较。

（4） 实验比较 28 行表达式改进前后在样例输入下的执行过程,并分析其搜索效率提升情况。

（5） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

7.2 回溯法时间复杂度分析比较

回溯法采用深度遍历的方式对解空间树进行搜索，在搜索过程中根据约束条件和限界函数对解空间树进行剪枝，从而大大减少搜索的范围。但在计算算法的时间复杂度时，由于无法确定剪枝的具体情况，往往无法通过时间复杂度确定其算法的具体时间效率。

例 7.2 针对例 7.1 中 0-1 背包问题，**要求：**

（1） 分析例 7.1 中算法的时间复杂度。

（2） 采用动态规划算法求解例 7.1 中的问题，分析其算法时间复杂度。

（3） 分析比较（1）和（2）两种算法的特点及适用范围。

解：

（1） 例 7.1 中算法复杂度分析：

程序中行 2，3，4，9，10，40，41，42，49，50，52 到 54，59 到 68 的时间复杂度为 O(1)，行 5，6，43 到 47 的时间复杂度为 O(n),行 51 的时间复杂度为 $O(n^2)$。

函数 Backtrack 可通过如下递推式求解其时间复杂度：

T(i)=2T(i+1)+1

T(n+1)=O(1)

通过递推可以得到：

$T(1)=O(2^n)$

因此，整个程序的时间复杂度为：$O(2^n)$。

（2） 如果采用动态规划算法求解 0-1 背包问题，则其程序如下：

```
#include “stdafx.h"
#include"iostream"
```

```
using namespace std;
template <class Type>
void Knapsack(Type *v, int *w, int c, int n, Type **m);
template <class Type>
void Traceback(Type **m, int *w, int c, int n, int *x);
int min(int i, int j);
int max(int i, int j);
int main()
{
    int *v, *w, c, n, **m;
    cin >> n;
    cin >> c;
    v = new int[n+1];
    w = new int[n+1];
    m = new int*[n+1];
    for (int i = 0; i <= n; i++) {
        m[i] = new int[c+1];
     }
     for (int i = 1; i <=n; i++) {
        cin >> w[i];
        cin >> v[i];
      }
      Knapsack(v, w, c, n, m);
      delete v;
      delete w;
      for (int i = 0; i <= n; i++) {
         delete m[i];
       }
      delete m;
      return 0;
}
    template <class Type>
    void Knapsack(Type *v, int *w, int c, int n, Type **m)
{
```

```
    /*m[i][j]表示存放第i,i+1,…,n之间的物品，显然如果i=n,则表明要么该物品能放在背
    包中，要么不能；如果背包容量小于第n个物品的重量，则该背包无法存放第n个物品，
    因此m[n][j]=0;*/
25    int jMax=min(w[n]-1,c);
26    for(int j=0;j<=jMax; j++)
27        m[n][j]=0;
      /*如果背包容量大于第n个物品的重量，则该物品可放在背包中，因此
        m[n][j]=v[n]*/
28    for(int j=w[n];j<=c;j++)
29        m[n][j]=v[n];
      //对于其他的物品，则有如下的几种可能性
30    for(int i=n-1;i>1;i--){
31        jMax=min(w[i]-1,c);
        /*如果背包的总容量小于第i件物品的重量，则背包所放物体的价值不变
32        for(int j=0;j<=jMax;j++)
33            m[i][j]=m[i+1][j];
          //反之，则可能放进去，也可能不放进去，取二者的最大值
34        for(int j=w[i];j<=c;j++)
35            m[i][j]=max(m[i+1][j],m[i+1][j-w[i]]+v[i]);
36    }
      //采用同样的方法,求m[1][c]
37    m[1][c]=m[2][c];
38    if(c>=w[1])
39        m[1][c]=max(m[1][c],m[2][c-w[1]]+v[1]);
40    return;
41 }

  int min(int i, int j)
42 {
43    if (i > j)
44        return j;
45    else
46      return i;
47  }
```

```
    int max(int i, int j)
48  {
49      if (i > j)
50          return i;
51      else
52          return j;
53    }
```

程序中行2到7，16，17，21，22，25，37到40的时间复杂度为O(1),行8到13，18，19的时间复杂度为O(n)，行26，27的时间复杂度为O(min(w[n]-1,c)),行28，29的时间复杂度为O(c),行30，31为O(n)，行32，33为O(n* min(w[i]-1,c)),行34，35为O(nc)，将上述语句时间复杂度相加，并化简后得到该程序的时间复杂度为O(nc)。

（3） 从（1）和（2）的时间复杂度分析结果可以看出，回溯法的时间复杂度与 n 有关，而动态规划算法的时间复杂度与 n 和 c 有关。采用与第二章相同的软硬件测试环境，物体的重量和价值为 100 以内的随机数，可得到图 7-5～图 7-8 所示的运行时间曲线。

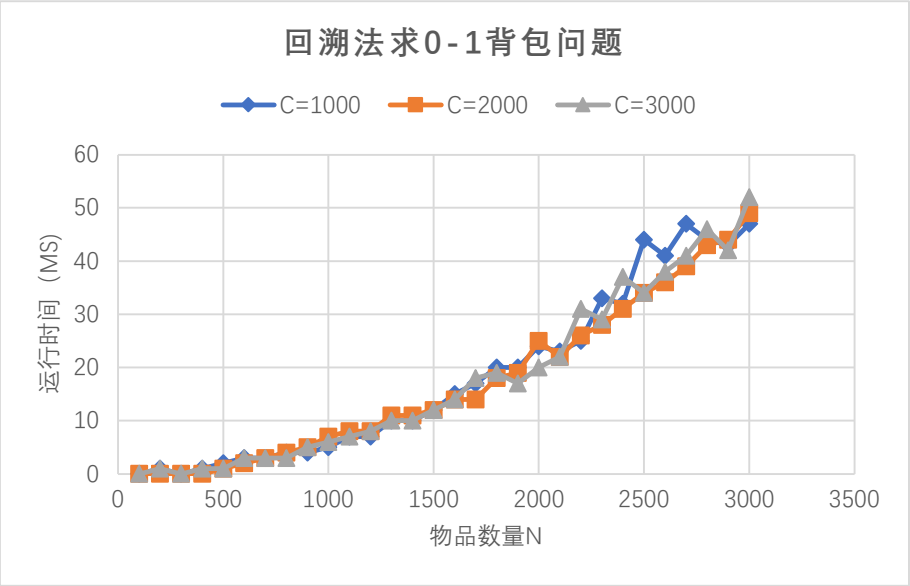


图7-5 回溯法求0-1背包问题运行时间曲线

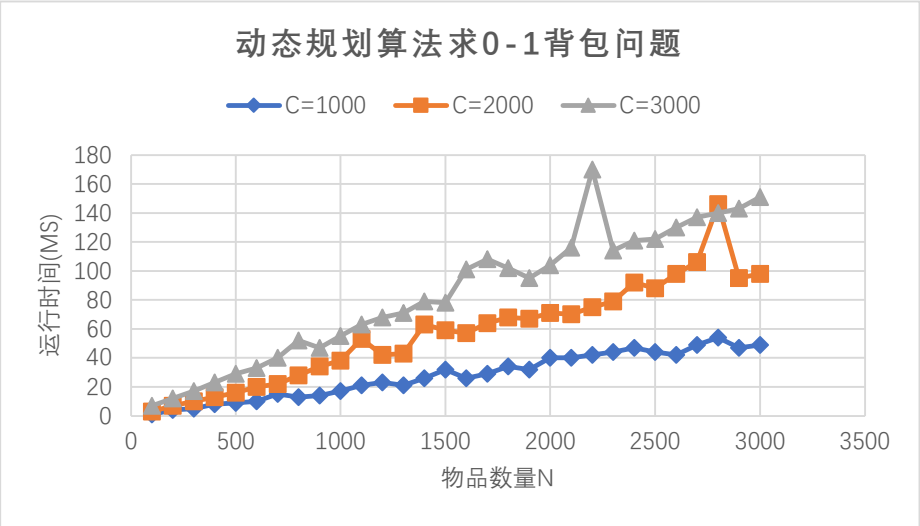


图7-6 动态规划算法求0-1背包问题运行时间曲线

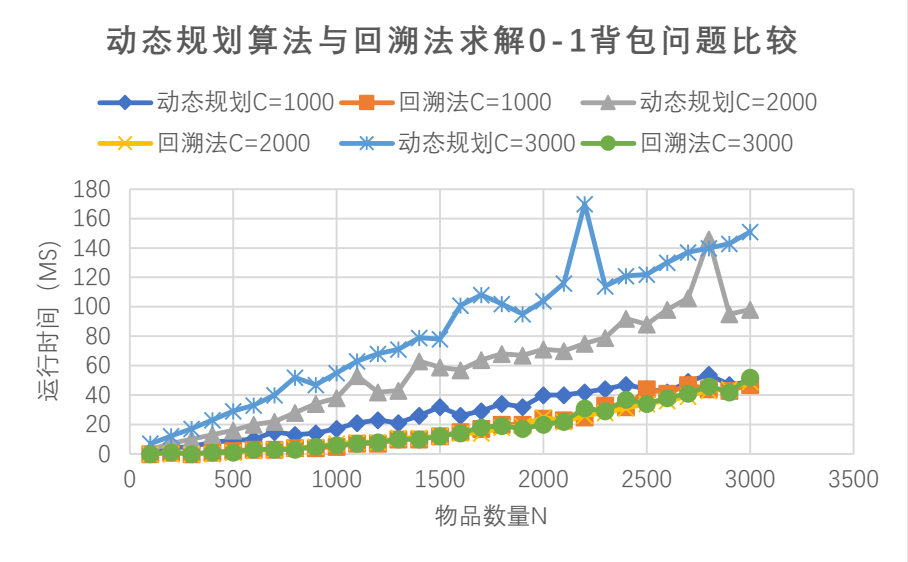


图7-7 动态规划算法与回溯法运行时间曲线比较

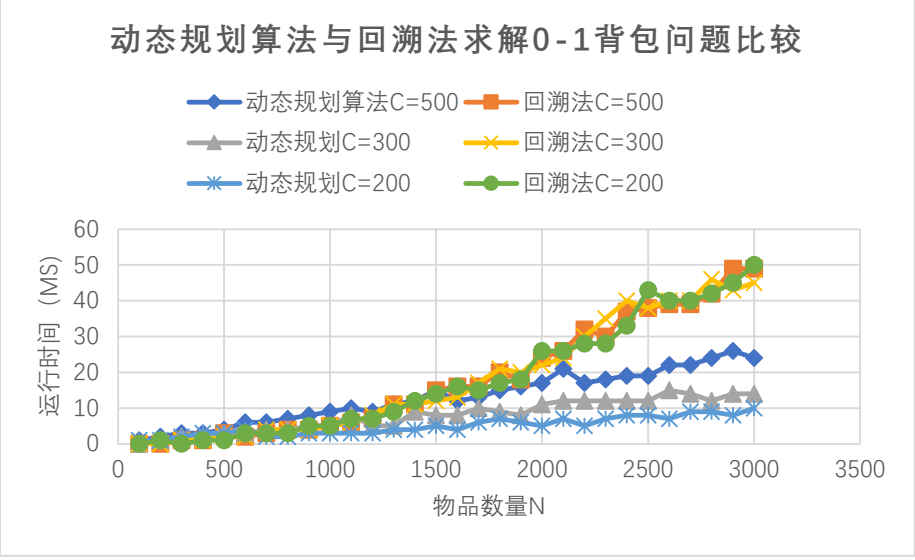


图7-8 动态规划算法与回溯法求解0-1背包问题运行时间曲线比较

从上面的曲线可以看出：

（1） 回溯法通过限界函数可以大大降低运算复杂度，即使在n值很大的情况下，也可以在较短的时间内进行求解。当然随着n值的不断增大，通过递归调用的方式求解会导致内存大大增加，从而会增加求解的难度。

（2） 在C值较大的情况下，回溯法实际运算的时间低于动态规划算法。

（3） 在C值较小时，动态规划算法的运行时间要优于回溯法。

（4） 回溯法的运行时间与C值的大小基本没有关系。而动态规划算法与C值关系较大。

由此可以看出，动态规划算法适用于C值较小而n值较大的情况；而回溯法无论C值有多大，均不会对其运行时间产生影响，但随着n值的增大其运行时间会增加较快，因此适用于n值较小的情况。

实验 7.2 针对实验 7.1 中旅行商问题。

实验预习：

（1） 分析实验 7.1 中算法的时间复杂度。

（2） 采用动态规划算法求解实验 7.1 中的问题，分析其算法时间复杂度。

（3） 分析比较（1）和（2）两种算法的特点及适用范围。

上机实验

（4） 实验比较回溯法及动态规划算法程序的运行时间与城市数量之间的关系，并与前面的分析结果进行比较。

（5） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

7.3 回溯法在实际中的应用

利用回溯法解决实际问题时，需要明确以下几个方面的内容:

(1) 目标函数: 如何判断已经得到了问题的一个解或者最优解。

(2) 约束条件:求解过程中如何判断哪一种情况或分支才真正满足问题的要求。

(3) 限界函数: 求解过程中，在当前情况下解的预期结果是什么，如何评判当前状态下是否可能得到问题的解或最优解。当然不是所有问题都能够明确其限界函数，有的情况下很难得到当前状态下的预期值。

(4) 问题的解空间: 构成问题的所有可能解。

(5) 搜索空间: 对解空间按照某种算法进行搜索时所形成的求解空间。

(6) 结点的状态: 由若干的变量构成，用于描述解空间或搜索空间树中的每一个结点 (可能性)。在递归调用中，一般包含递归调用函数的输入变量或者某些全局性的变量。

例 7.3 采用回溯法求解下面的问题。

在一个给定形状的棋盘（形状可能是不规则的）上面摆放棋子，棋子没有区别。要求摆放时任意的两个棋子不能放在棋盘中的同一行或者同一列，请编程求解对于给定形状和大小的棋盘，摆放 k 个棋子的所有可行的摆放方案 C。

输入：

输入含有多组测试数据。每组数据的第一行是两个正整数，n k，用一个空格隔开，表示了将在一个 n*n 的矩阵内描述棋盘，以及摆放棋子的数目。 N <= 8 , k <= n，当为-1 -1 时表示输入结束。随后的 n 行描述了棋盘的形状：每行有 n 个字符，其中 # 表示棋盘区域，. 表示空白区域(数据保证不出现多余的空白行或者空白列)。

输出：

对于每一组数据，给出一行输出，输出摆放的方案数目 C （数据保证 C<231)。

样例输入：

4 3

.#.#

..#.

.#.#

#.#.

-1 -1

样例输出：

10

要求：

（1） 写出采用回溯法求解上述问题的目标函数，约束条件以及限界函数。

（2） 画出采用回溯法在求解样例输入时的解空间树。

（3） 画出采用回溯法求解样例输入时的搜索空间树。

（4） 采用回溯法求解上述问题，写出算法的实现过程，简要分析算法的时间复杂度。

（5） 编写程序求解上述问题。

解：

（1） 设 k 为需要棋盘中摆放的棋子的数量，ans 表示摆放的方案数，每在棋盘中放一个棋子，则 k 减 1，因此其目标函数为 k=0;且在 k=0 时，对应的摆放方案数 ans++。

设 chess[i][j]表示棋盘中第 i 行第 j 列位置的值，该值要么为#，表示棋子可以摆放，要么为.，表示棋子不能摆放。每次摆放，棋子必须放在#所在的位置，因此回溯法搜索过程中，必须满足 chess[i][j] = ‘#’。此为其约束条件之一。另外，如果该棋子即使为#，但此前在该行已经摆放了棋子，则也不能再放棋子。设:

sign[j]=1: 表示第 j 行已经摆放了棋子

sign[j]=0: 表示第 j 行未摆放棋子

因此其约束条件为：sign[j]==0 && chess[i][j]==’#’

在搜索过程中如果已经搜索到了棋盘的最后一行，则不能再继续搜索。设本轮搜索的起始行为 x,棋盘有 n 行，则其限界函数为：x+k-1<=n。

（2） 采用回溯法，在样例输入的情况下其解空间树如图 7-9 所示。

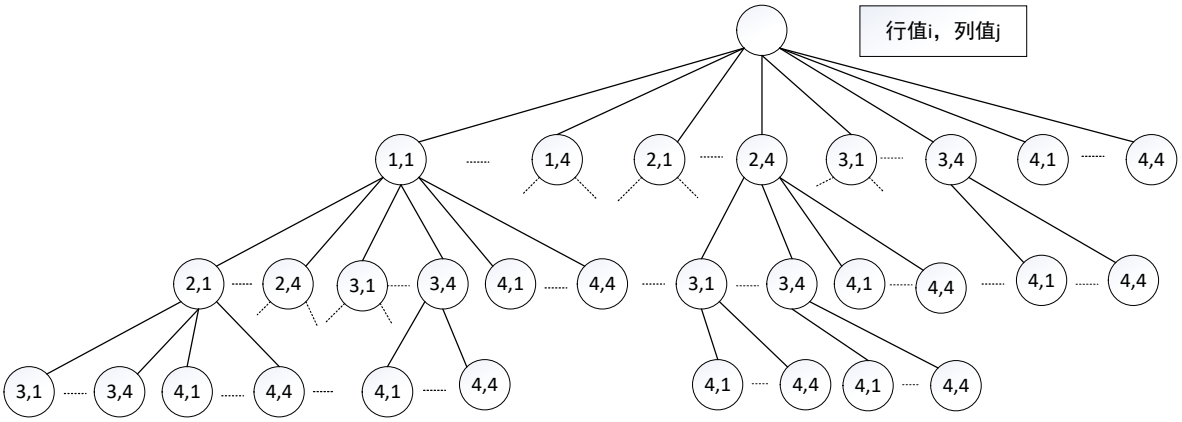


图 7-9 样例输入下的解空间树示意图

由于树中结点比较多，因此图 7-9 中省略了一些结点。

（3） 利用回溯法，利用前面的目标函数，约束条件以及限界函数，在（2）的解空间树上进行深度遍历，可以得到如图 7-10 所示的搜索空间树。

树中的结点为搜索过程中访问的结点，未访问的结点未在图中罗列。显然相比解空间树，通过约束条件和限界函数极大的降低了搜索的范围，提高了算法的时间效率。

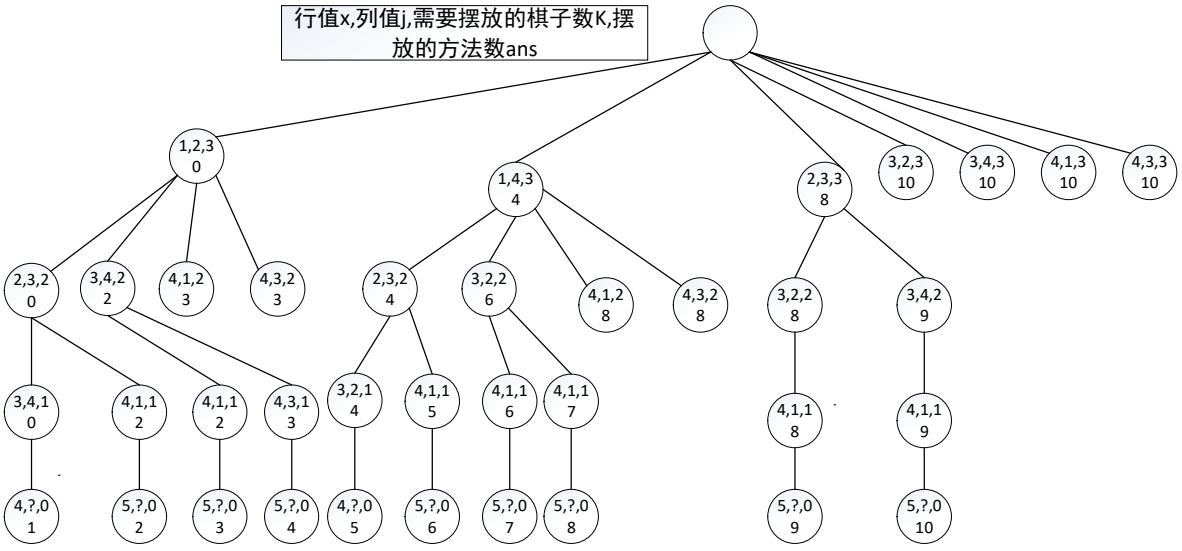


图 7-10 回溯法求解样例输入时的搜索空间树

（4） 设 chess[i][j]存储了棋盘中第 i 行第 j 列的元素的值，x 表示 chess 的

行值，k 表示当前棋盘状态下需要摆放的棋子数量，sign[j]表示第 j 列是否已经摆放了棋子，ans 表示在 chess 中摆放 k 个棋子的最大摆放方法，函数 dfs(x,k)用于求解上述问题，对应的算法描述如下：

输入：棋盘状态 chess,当前棋盘格所在行 x,棋盘中需要摆放的棋子数量 k

输出：将 k 个棋子摆放在 chess 中的最大摆放方法

```
1   void dfs(int x, int k)
2   {
3       //如果找到了一种摆放方法，则摆放方法数加1
4       if (k == 0) {
5           ans++;
6           return;
7       }
8       //如果已经找到了最后一行，则返回
9       if (x+k-1 > n) {
10          return;
11      }
12      //从第x行开始查找，一直找到最后一行
13      for (int i = x; i <= n; i++) { //行数
14          //查找第i行的第j列
15          for (int j = 1; j <= n; j++) {//列数
16              //如果这一行的第j个元素还没有查找，且为'#'，则放一个棋子
17              if (!sign[j] && chess[i][j] == '#') {
18                  //设置为已摆放
19                  sign[j] = 1;
20                  //深度搜索，查找下一行，并将能放的棋子数-1。
21                  dfs(i + 1, k - 1);
22                  //回溯，并恢复成未摆放状态，查找该行的下一列的元素。
23                  sign[j] = 0;
24              }
25          }
26      }
27  }
```

（5） 求解该问题的程序如下：

```
1   #include<cstdio>
```

```
2   #include<cstring>
3   #include<iostream>
4   using namespace std;
5   char chess[10][10];
6   int sign[10];
7   int n, k, ans;
8   void dfs(int x, int k)
9   {
10      //如果找到了一种摆放方法，则摆放方法数加1
11      if (k == 0) {
12          ans++;
13          return;
14      }
15      //如果已经找到了最后一行，则返回
16      if (x+k-1 > n) {
17          return;
18      }
19      //从第x行开始查找，一直找到最后一行
20      for (int i = x; i <= n; i++) { //行数
21          //查找第i行的第j列
22          for (int j = 1; j <= n; j++) {//列数
23              //如果这一行的第j个元素还没有查找，且为'#'，则放一个棋子
24              if (!sign[j] && chess[i][j] == '#') {
25                  //设置为已摆放
26                  sign[j] = 1;
27                  //深度搜索，查找下一行，并将能放的棋子数-1。
28                  dfs(i + 1, k – 1);
29                  //回溯，并恢复成未摆放状态，查找该行的下一列的元素。
30                  sign[j] = 0;
31              }
32          }
33      }
34  }
35  int main()
```

```
{
    while (cin >> n >> k) {
        memset(chess, 0, sizeof(chess));
        memset(sign, 0, sizeof(sign));
        if (n == -1 || k == -1) {
            break;
        }
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                cin >> chess[i][j];
            }
        }
        ans = 0;
        dfs(1, k);
        cout << ans << endl;
    }
    return 0;
}
```

实验 7.3 给定一个整数 t 和 n 个整数的集合 list，将 list 中的数相加使其和值为 t，给出所有的组合。比如 t=4 和 n 个数{4, 3, 2, 2, 1, 1}，则对应的和值为 4 的所有数的组合有：4, 3+1, 2+2, 2+1+1。某个数在 list 中出现多少次，则该数最多可使用多少次，也可作为加数单独使用。请编写程序求解上述问题。

输入要求：

输入包含一个或多组测试数据。每组测试数据占一行。每行的第一个数据为 t，也就是对应的和值，第 2 个数据为 n，表示 list 中有多少个数据，后面紧跟 n 个整数，为 list 中的数据。N=0 则表示输入结束。T 是一个小于 1000 的正整数，n 是 1～12 之间的正整数，list 中的数小于 100 大于 0.所有的数之间用空格隔开。List 中数可能重复，且无序输入。

输出要求：

对于每一组测试数据，输出对应的加法表达式，每一个表达式占一行。每一个加法表达式中的加数必须按降序排列且输出结果中不能出现相同的加法表达式。如果不存在相应的加法表达式，则输出 NONE。

样例输入：

4 6 4 3 2 2 1 1

5 3 2 1 1

样例输出：

4

3+1

2+2

2+1+1

NONE

实验预习：

（1） 写出采用回溯法求解上述问题的目标函数，约束条件以及限界函数。

（2） 画出采用回溯法在求解样例输入时的解空间树。

（3） 画出采用回溯法求解样例输入时的搜索空间树。

（4） 采用回溯法求解上述问题，写出算法的实现过程，简要分析算法的时间复杂度。

（5） 编写程序求解上述问题。

上机实验：

（6） 上机验证程序在样例输入时的执行过程是否与步骤（3）所得到的搜索空间树一致，写出程序调试过程中搜索空间树上每个结点的值，并与前面的结果进行比对。

（7） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

实验 7.4 一个小孩手中有 N 块正方形的积木，他总是想不同的方法来搭建各种不同的楼梯。他搭建的楼梯必须满足如下条件：

（1）楼梯每个台阶的砖块数不能相同，且严格递减。

（2）每个楼梯至少包含两个台阶

（3）必须用完所有的积木

请编写程序计算该小孩最多有多少种满足上述条件的不同的搭建方法。

输入要求：

输入 1 行，积木的个数 N，N 大于等于 3。

输出要求：输出 1 行，有多少种不同的楼梯搭建方法。

输入样例：

10

输出样例

9

实验预习：

（1） 写出采用回溯法求解上述问题的目标函数，约束条件以及限界函数。

（2） 画出采用回溯法在求解样例输入时的解空间树。

（3） 画出采用回溯法求解样例输入时的搜索空间树。

（4） 采用回溯法求解上述问题，写出算法的实现过程，简要分析算法的时间复杂度。

（5） 编写相应的程序。

上机实验：

（6） 上机验证程序在样例输入时的执行过程是否与步骤（3）所得到的搜索空间树一致，写出程序调试过程中搜索空间树上每个结点的值，并与前面的结果进行比对。

（7） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

实验总结：

本章通过具体实例介绍了回溯法的分析求解过程，时间复杂度分析方法，与其他算法的效率差异,以及如何设计回溯法解决实际问题。通过本实验，学生可以正确理解回溯法的特点以及在实际应用中如何正确设计并使用回溯算法。

实验 8 分支限界法设计与实验

实验目的：

(1) 理解分支限界法的求解过程。

(2) 分析分支限界法的时间复杂度，比较分支限界法算法与其他算法的时间效率差异。

(3) 学会如何利用分支限界法求解具体问题，了解动分支限界法的应用范围及在实际应用中的局限性。

8.1 分支限界法求解过程分析

分支限界法采用广度搜索方式对问题的解空间进行搜索，并在搜索过程中对不能得到最优解的分支进行剪枝，从而提高搜索的效率。

分支限界法在求解过程中一般采用队列或者堆的存储结构来存储需要搜索的结点。只有那些满足约束条件和/或限界条件的结点才能进入队列或者堆。如果采用常规的队列存储需要搜索的结点，与普通的广度遍历并没有什么区别，而采用堆的存储结构则可以使最有可能获得最后解的结点得到优先处理，因此分支限界法往往采用堆的存储结构来存储需要搜索的结点。

例 8.1 0-1 背包问题的分支限界法求解过程分析

参照例 7.1 中背包问题，采用分支限界法求解时的程序如下：

```
#include “stdafx.h"
#include"stdio.h"
#include “malloc.h"
#define N  20
struct HeapNode{
    int w;            //当前重量
    int v;            //当前价值
    int level;        //所在层
    int bound;        //上界
    int route[N];     //路径，解向量的各个分量
};
void InsertHeap(HeapNode b[], HeapNode x, int &length);
void DeleteHeap(HeapNode b[], int &length);
```

```
int MaxUpBound(HeapNode b[], int length);
void Knapsack(int n, int w[], int v[], int sw);
HeapNode heap[N];
int heapLength=0;
int level=0;
int main()
{
    int n;
    int *w, *v, sw;
    scanf("%d", &n);
    scanf("%d", &sw);
    w = (int *)malloc(sizeof(int)*(n + 1));
    v = (int *)malloc(sizeof(int)*(n + 1));
    for (int i = 1; i <= n; i++){
        scanf("%d", &w[i]);
        scanf("%d", &v[i]);
    }
    Knapsack(n, w, v, sw);
    free(w);
    free(v);
    return 0;
}
//在HeapNode堆中插入结点x构建大顶堆，并修改堆的长度
void InsertHeap(HeapNode b[], HeapNode x, int &length)
{
    int i;
    for (i = length + 1; i > 1; i = i / 2){
        if (x.bound <= b[i / 2].bound)
            break;
        else
            b[i] = b[i / 2];
    }
    b[i] = x;
    length += 1;
```

```
29  }
    //删除栈顶元素，并调整堆使其形成新的大顶堆
30  void DeleteHeap(HeapNode b[], int &length)
31  {
32      int i;
33      b[1] = b[length];           //最后一个元素取代栈顶元素
34      HeapNode temp = b[1];
35      b[length].bound = 0;        //最后一个元素给一个最小值
36      for (i = 2; i < length; i *= 2){
37          if (b[i].bound < b[i + 1].bound)
38              i++;
39          if (temp.bound>b[i].bound)
40              break;
41          b[i / 2] = b[i];        //将左右子树大者取代双亲节点
42      }
43      i = i / 2;                  //找到其双亲结点
44      b[i] = temp;                //取代双亲结点
45      length -= 1;                //长度减1
46  }

    //求堆中所有结点的界的最大值
47  int MaxUpBound(HeapNode b[], int length)
48  {
49      int max=0;
50      for (int i = 1; i <= length; i++){
51          if (max < b[i].bound)
52              max = b[i].bound;
53      }
54      return max;
55  }
56  void Knapsack(int n, int w[], int v[], int sw)
57  {
58      HeapNode rootNode;
59      rootNode.w = 0;
```

```
60      rootNode.v = 0;
61      rootNode.bound = sw* (v[1]*1.0 / w[1]);
62      for (int i = 0; i < N; i++)
63          rootNode.route[i] = 0;
64      rootNode.level = 0;
65      InsertHeap(heap, rootNode, heapLength);   //在heap堆中插入根节点
66      while (heapLength>0){
67          HeapNode temp;                        //temp指向堆顶元素
68          temp = heap[1];
69          level = temp.level;
70          DeleteHeap(heap,heapLength);          //删除堆顶元素
71          if (level == n && temp.v >= MaxUpBound(heap, heapLength)){
72              for (int i = 1; i <= n; i++){
73                  printf("%d ", temp.route[i]);
74              }
75              printf("\n%d", temp.v);
76              return;
77          }
78          else{
79              for (int j = 0; j <= 1; j++){
80                  if (temp.w + j*w[level + 1] <= sw){
81                      HeapNode node;
82                      node.w = temp.w + j*w[level + 1];
83                      node.v = temp.v + j*v[level + 1];
84                      if (level < n - 1)
85                          node.bound = node.v + (sw - node.w)*(v[level
                                        + 2]*1.0 / w[level + 2]);
86                      else
87                          node.bound = node.v;
88                      for (int i = 1; i <= level; i++)
89                          node.route[i] = temp.route[i];
90                      node.route[level + 1] = j;
91                      node.level = level + 1;
92                      InsertHeap(heap, node, heapLength);
```

```
93                          }
94                      }
95                  }
96              }
97  }
```

要求：

（1） 分析函数Knapsack在下面的样例输入下的执行过程及主要变量的变化情况，并通过实验对分析结果进行验证。

6 80

10 20

20 30

30 40

40 50

50 60

60 70

（2） 程序中84行～87行计算当前结点的最大可能价值，请对该部分程序进行改进，使该值更接近于该结点的最大可能价值，并分析该值对算法的影响。

解：

（1） Knapsack函数的执行过程就是对堆的操作过程。在本例中，堆中的结点包含当前背包所装物品的重量w，当前背包中物品的价值v，当前已经处理了多少个物品level，当前物品的价值以及剩余重量可能装的物品的最大可能价值bound，当前背包中装了哪些物品，保存在route中。Knapsack函数中的主要变量保存在heap堆中，且以heap.bound作为堆结点的值来构造大顶堆。在样例输入情况下其变化过程如图8-1所示。

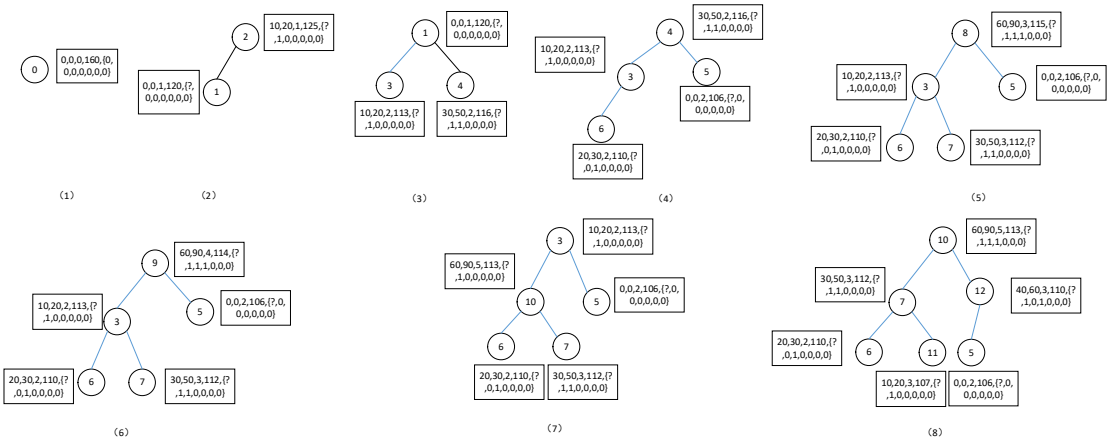
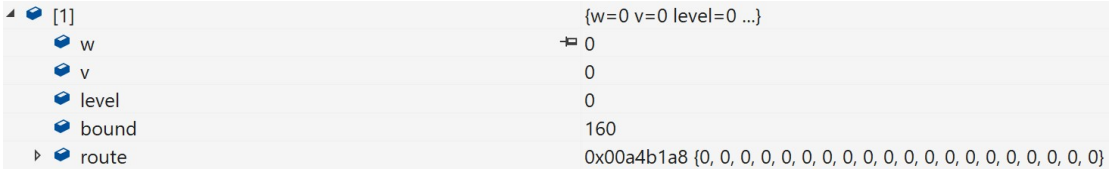




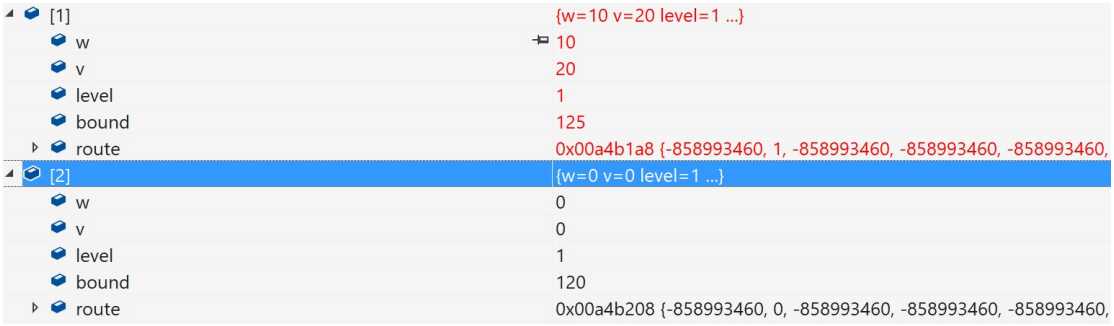
图 8-1 函数 Knapsack 中 heap 变量的变化过程

其中圆形结点内的值表示其进入堆的序号，也就是先后顺序。矩形框中用逗号隔开的变量依次为 w, v, level, bound 和 route。上述过程反应的是程序执行到 70 行时 heap 的值。该行对应的操作为首先删除堆顶元素，然后再加入与堆顶元素相关的满足约束条件和限界条件的结点。

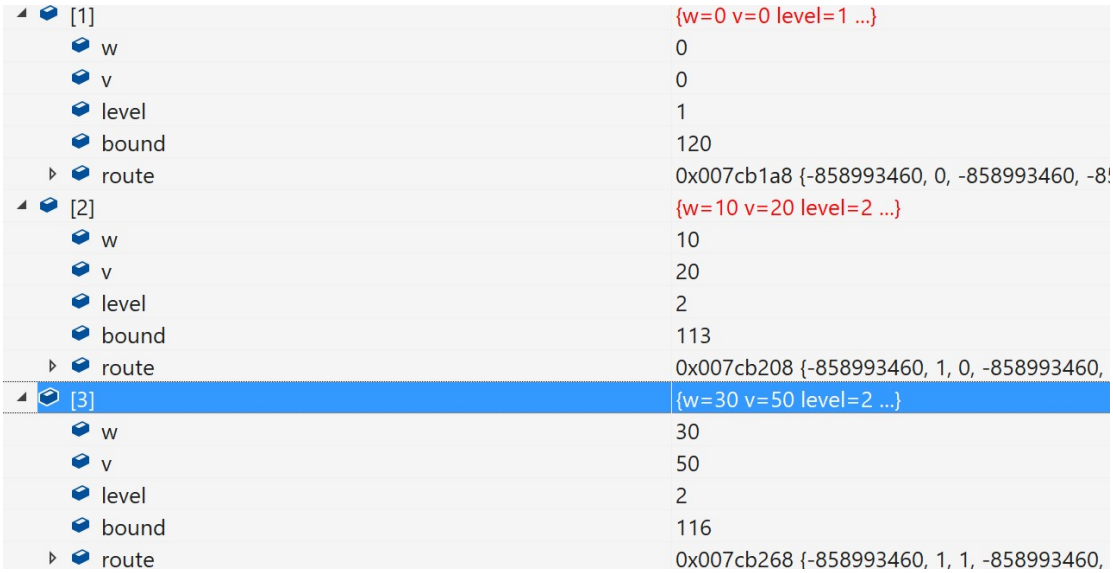
对上述程序进行调试，可得到对应的内存映像，如图 8-2 所示。



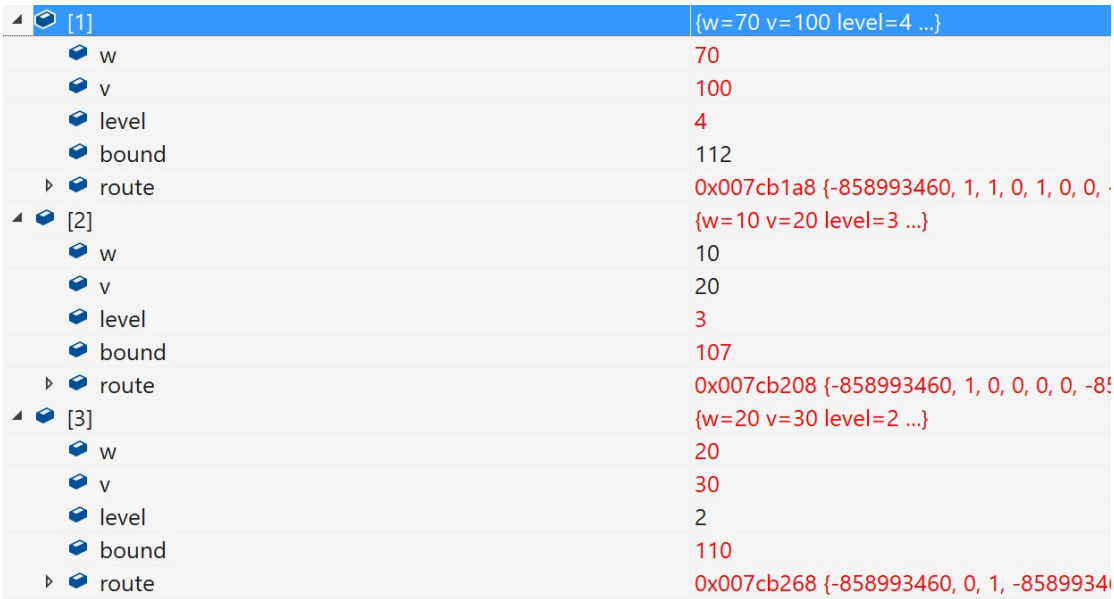
该图与图 8-1 中第一个图对应

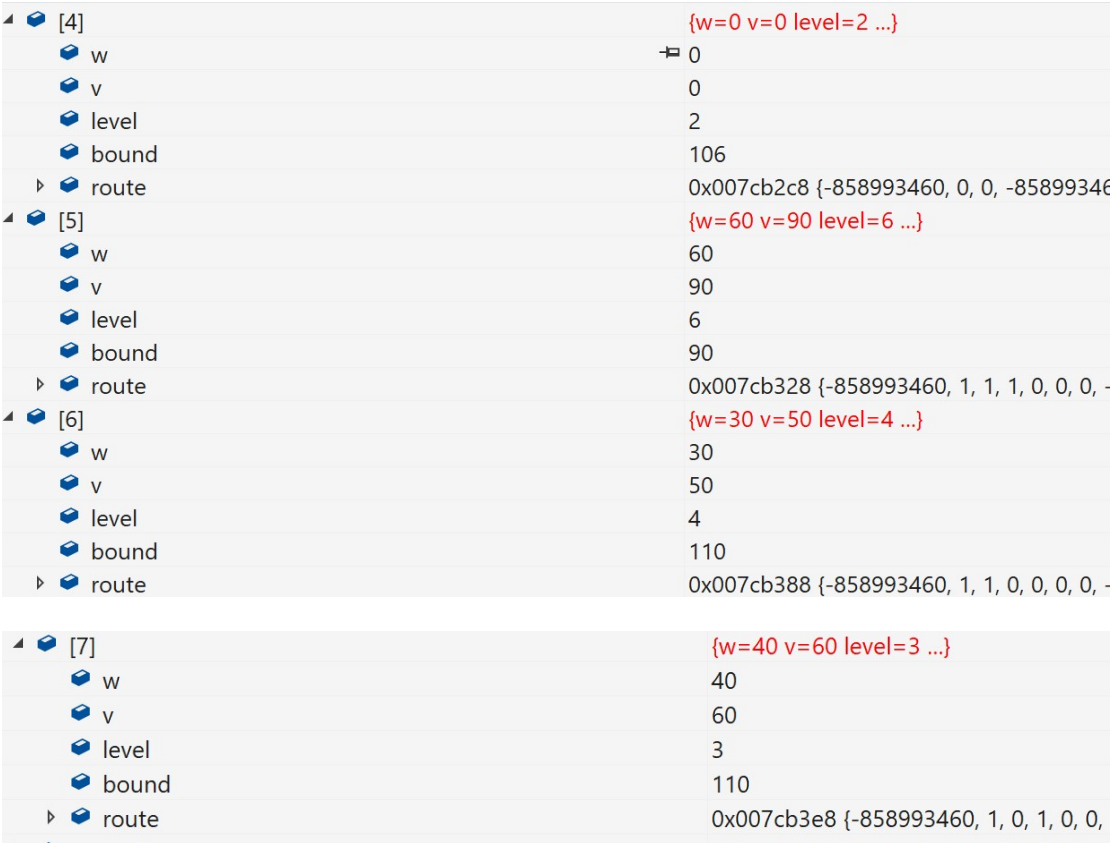


该图与 8-1 中第 2 个图对应



该图与 8-1 中第 3 个图对应





该图与图 8-1 中图（10）对应

8-2 结点对应的 heap 变量的值在内存中的映像

通过实验可以得到其它情况下的内存映像，结果与分析结果一致。

（2） 通过对上述求解过程的分析，可以发现对于拥有6个物品的背包问题，共计处理了26个结点，而如果采用穷举法则需要处理127个结点，因此效率有所提升。同时，对行84～87的分析可以发现，如果计算某个结点的预期最大价值时采用更准确的计算方法，也可以压缩求解过程，使最可能得到最优解的结点优先处理。

通过上述分析，可以对算法做如下改进：

修改 bound 的计算方法，采用贪心法计算当前状态下背包可能得到的最大物品价值，而不是简单的利用剩余重量*剩余物品中最大的单位价值。当然这样处理也可能会增加运算的复杂度。其对应的程序代码如下：

```
node.bound = node.v;
int lw = sw - node.w;
for (int i = level + 2; i <=n; i++) {
    if (lw >= w[i]) {
        node.bound += v[i];
        lw = lw - w[i];
    }
    else {
```

```
            node.bound += lw * v[i] * 1.0 / w[i];
            break;
            }
        }
```

思考：

请分析通过上述修改后的 Knapsack 函数在样例输入情况下的执行过程以及变量变化情况。

实验 8.1 旅行商问题的分支限界法求解

针对实验 7.1 的问题，采用分支限界法求解该问题的程序实现如下：

```
#include “stdafx.h"
#include"stdio.h"
#include “malloc.h"
#include “math.h"

#define N  5
#define BIGNUM 1e9
struct HeapNode{
    int v;            //路径长度
    int level;        //所在层
    int bound;        //上界
    int route[N+1];     //路径，解向量的各个分量
};
void InsertHeap(HeapNode b[], HeapNode x, int &length);
void DeleteHeap(HeapNode b[], int &length);
void Tsp(int C[N][N],int 105pbound);
int RowMinExcept(int C[N][N], int row, int col);
int SiSum(int C[N][N], HeapNode temp);
int RowMinTwo(int C[N][N], int row);
bool Contain(HeapNode temp, int node);
int SjSum(int C[N][N], HeapNode temp);

HeapNode heap[100];
int heapLength = 0;
int level = 0;


int main()
{
    int C[N][N],upbound=0;
    for (int i = 0; i < N; i++) {
        C[i][i] = BIGNUM;
        for(int j = i+1; j < N; j++) {
```

```
7                 scanf("%d", &C[i][j]);
8                 C[j][i] = C[i][j];
9             }
10        }
11        int k = N-1;
12        int i= 0;
13        int max = BIGNUM;
14        int visited[N];
15        int row;
16        for (int j = 0; j < N; j++)
17            visited[j] = 0;
18        while (k) {
19            for (int j = 0; j < N; j++) {
20                if (j == i || visited[j])
21                    continue;
22                if (C[i][j] < max) {
23                    max = C[i][j];
24                    row = j;
25                }
26            }
27            upbound += max;
28            visited[i] = 1;
29            i = row;
30            k--;
31            max = BIGNUM;
32        }
33        upbound += C[row][0];
34
35        Tsp(C, upbound);
36        return 0;
37    }
    //在HeapNode堆中插入结点x构建小顶堆，并修改堆的长度
38  void InsertHeap(HeapNode b[], HeapNode x, int &length)
39  {
```

```
40      int i;
41      for (i = length + 1; i > 1; i = i / 2){
42          if (x.bound >= b[i / 2].bound)
43              break;
44          else
45              b[i] = b[i / 2];
46      }
47      b[i] = x;
48      length += 1;
49  }
```

```
    //删除栈顶元素，并调整堆使其形成新的小顶堆
50  void DeleteHeap(HeapNode b[], int &length)
51  {
52      int i;
53      b[1] = b[length];
54      HeapNode temp = b[1];
55      b[length].bound = BIGNUM;
56      for (i = 2; i < length; i *= 2){
57          if (b[i].bound >b[i + 1].bound)
58              i++;
59          if (temp.bound<=b[i].bound)
60              break;
61          b[i / 2] = b[i];
62      i = i / 2;
63      b[i] = temp;
64      length -= 1;
65  }
```

```
    //求C矩阵第Row行中除Col列外的最小元素
66  int RowMinExcept(int C[N][N], int row, int col)
67  {
68      int min = BIGNUM;
69      for (int j = 0; j < N; j++){
```

```
70          if (j == col)
71              continue;
72          if (C[row][j] < min)
73              min = C[row][j];
74      }
75      return min;
76  }


77  int SiSum(int C[N][N], HeapNode temp)
78  {
79      int level = temp.level;
80      int sum = RowMinExcept(C, temp.route[0] - 1, temp.route[1] - 1)
81  + RowMinExcept(C, temp.route[level] - 1, temp.route[level - 1] - 1);
82      return sum;
83  }

    //求矩阵C中第row行中最小的两个元素之和
84  int RowMinTwo(int C[N][N], int row)
85  {
86      int minOne = BIGNUM;
87      int minTwo = 0;
88      int col;
89      for (int j = 0; j < N; j++){
90          if (C[row][j] < minOne){
91              minOne = C[row][j];
92              col = j;
93          }
94      }

95      minTwo = RowMinExcept(C, row, col);

96      return minOne + minTwo;
97  }
```

```
    //判断下一个要选择的结点是否包含已走过的路径
98  bool Contain(HeapNode temp, int node)
99  {
100     int level = temp.level;    //获得当前已处理的结点
101     for (int i = 0; i <= level; i++){
102         if (temp.route[i] == node)
103             return true;
104     }
105     return false;
106 }

107 int SjSum(int C[N][N], HeapNode temp)

108 {
109     int sum = 0;
110     for (int i = 1; i <= N; i++){
111         if (!Contain(temp, i)){
112             sum = sum + RowMinTwo(C, i - 1);
113         }
114     }
115     return sum;
116 }

117 void Tsp(int C[N][N],int 109pbound)

118 {
119     HeapNode rootNode;
120     rootNode.v = 0;
121     rootNode.level = 0;
122     rootNode.route[0] = 1;
123     rootNode.bound = SjSum(C, rootNode) / 2.0;
124     InsertHeap(heap, rootNode, heapLength);    //在heap堆中插入根节点
125     while (heapLength>0){
126         HeapNode temp;                         //temp指向堆顶元素
127         temp = heap[1];
```

```
128         level = temp.level;
129         DeleteHeap(heap, heapLength);
130         if ((level == N))
131         {
132             for (int i = 0; i <= N; i++) {   //输出该结点所对应的路径
133                 printf("%d ", temp.route[i]);
134             }
135             printf("\n%d", temp.v);
136             return;
137         }
138         else {
139             if ((level == N - 1) && C[temp.route[level] -
                     1][temp.route[0] - 1]!= BIGNUM){
140                 temp.route[level + 1] = temp.route[0];
141                 float lb=temp.v+C[temp.route[level] - 1][temp.route[0]
                            - 1];
142                 if (lb<110pbound) {
143                     HeapNode node;
144                     node.v = lb;
145                     node.bound = lb;
146                     for (int i = 0; i <= level+1; i++)
147                         node.route[i] = temp.route[i];
148                     node.level = level + 1;
149                     InsertHeap(heap, node, heapLength);
150                     upBound = lb;
151                 }
152             }
153             else {
154                 //在若干个城市中选择
155                 for (int j = 1; j <= N; j++) {
156                     if ((C[temp.route[level] - 1][j - 1] != BIGNUM)
                                && !Contain(temp, j)) {
157                         temp.route[level + 1] = j;
158                         temp.level = level + 1;
```

```
159                         float lb = ((temp.v + C[temp.route[level]
                                - 1][j - 1]) * 2 + SiSum(C, temp)
                                 + SjSum(C, temp)) / 2.0;
160                         if (lb <= 111pbound) {
161                             HeapNode node;
162                             node.v = temp.v + C[temp.route[level]
                                        - 1][j - 1];
163                             node.bound = lb;
164                             for (int i = 0; i <= level; i++)
165                                 node.route[i] = temp.route[i];
166                             node.route[level + 1] = j;
167                             node.level = level + 1;
168                             InsertHeap(heap, node, heapLength);
169                         }
170                     }
171                 }
172             }
173         }
174     }
175 }
```

实验预习：

（1） 分析函数Tsp在下面的样例输入下的执行过程及主要变量的变化情况。

4 8 6 5 3 2 10 8 7 3

（2） 采用priority_queue对上述程序进行修改，实现程序中对堆的操作。

（3） 修改该程序使其可以输入任意多个城市，并根据城市多少动态分配数组C的大小，同时求解出最短回路。

上机实验：

（4） 调试程序，给出函数 Tsp 在（1）中样例输入情况下主要变量的执行结果，并与（1）中的分析结果进行比较，验证分析结果是否正确。

（5） 调试程序，给出按照（2）修改后的程序在样例输入情况下执行过程中主要变量的变化情况，并与（1）修改前的主要变量变化情况进行对比，验证修改后的程序是否正确。

（6） 上机调试，验证（3）中修改后的程序是否正确，给出相应的测试数据、分析结果及测试结果。

（7） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

8.2 分支限界法时间复杂度分析比较

对于同一个问题，如果其解空间树相同，则无论是分支限界法还是回溯法其时间复杂度也一样。但由于在求解过程中所采用的方法不同，对解空间树进行剪枝的效果也不同，因此算法的实际执行时间也不同的。总的来说，在大多数情况下分支限界法用于求解问题最优解时算法执行时间更少，效率更高。

例8.2 实验比较0-1背包问题在不同物品数量及背包容量下采用回溯法及分支限界法时的算法执行时间。要求：

（1） 实验比较分支限界法和回溯法在相同输入情况下的算法执行时间。

（2） 分析比较两种算法的运行时间。

解：

（1） 采用与第二章相同的软硬件实验环境，采用随机数生成方法生成物品数量以及物品的重量和价值，采用实验7.1相同的回溯算法和实验8.1改进前的分支限界法，可得到8-3所示的曲线。

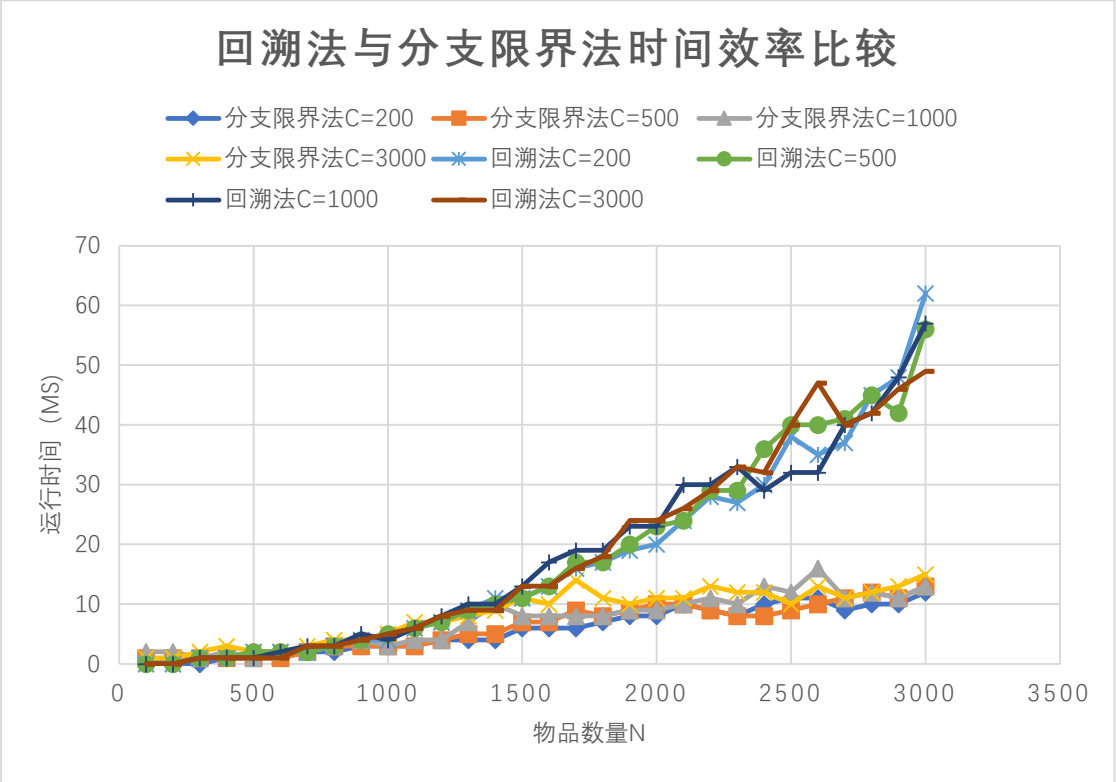


图8-3 不同算法相同背包容量下物品数量与执行时间之间关系

需要注意的是为避免输出打印对程序执行时间的影响，应注释实验8.1中程

序的打印输出语句。

（2） 从8-3的曲线可以看出，背包容量对算法的执行时间几乎没有影响；随着物品数量的增多，两种算法的执行时间均会增加，但回溯法增加的速度相对于分支限界法而言更快。

实验 8.2 实验比较采用分支限界法和回溯法求解旅行商问题时的运行时间。

实验预习：

（1） 分析影响分支限界法程序运行时间的主要因素。

上机实验：

（2） 设计实验，验证（1）中分析结果是否与实验结果一致。

（3） 通过上机实验，绘制分支限界法和回溯法城市数量-运行时间关系曲线，分析算法对程序运行时间的影响。

（4） 撰写相应的实验报告，实验报告内容包括：实验目的、实验任务、实验环境、实验步骤、实验结果及其分析以及实验总结等部分内容。

8.3 分支限界法的应用

分支限界法广泛应用于求问题的最优解，其求解的效率高于回溯法。

采用分支限界法求问题的最优解，首先需要确定问题的解空间，构造解空间树，然后利用约束条件和限界函数对解空间树进行剪枝，通过目标函数确定是否已经得到了问题的解，一旦得到问题的解则退出求解过程。

分支限界法往往利用堆的存储结构来保存求解过程中所产生的活结点，并将堆顶元素作为扩展结点，利用解空间树中结点之间的连接关系，根据约束条件和限界函数选取合适的结点作为活结点放入到堆中，并确保整个过程满足堆的条件。当堆顶的结点满足目标函数时，则得到问题的解，并确保满足目标函数的堆顶结点是问题的最优解。

因此采用分支限界法求解问题，需要构造问题的解空间树，确定目标函数，约束条件及限界函数。在求解过程中需要不断的调整堆中的结点，并得到相应的搜索空间树。

例 8.3 在某博物馆中摆放了非常重要的文物，为了节省人力，该博物馆专门购买了警卫机器人来看管这些文物。该博物馆的房间排列整齐，房间的大小相同。每个警卫机器人能够巡查的范围除本身所在房间外，还包括其起始安放的房间的上下左右四个房间。为了减少摆放的机器人的数量，请你设计一种最佳的摆放方案，使得摆放的机器人数量最少。

输入：

输入一行，有两个整数 m,n,分别表示该博物馆每行的房间数和每列的房间

数。博物馆总房间数即为 m*n。

输出：

输出的第一行表示需要的机器人的数量，其后 m 行，每行有 n 个元素，每个元素的值为 0 或 1，分别表示对应的房间是否摆放机器人。0 表示不摆放，1 表示需要摆放。

样例输入：

5 4

要求：

（1） 给出采用分支限界法求解该问题时的目标函数，约束条件以及限界函数。

（2） 给出采用分支限界法求解样例输入时的解空间树。

（3） 分析采用分支限界法求解样例输入时的求解过程，堆结点的定义，堆结点的值以及堆中元素的变化过程。

(4) 画出采用分支限界法求解样例输入时的搜索空间树。

(5) 编写相应的程序。

解：(1) 设 Node 为当前搜索的结点，Node 的定义如下：

```
typedef struct Node{
   int i,j,robotNums,beenMonitored;
   int **x,**y;
   int m,n;
 }Node;
 Node curNode, preNode;
```

其中:

x[i][j]表示陈列室中放置机器人的情况,y[i][j]表示陈列室中房间被监控的情况。相应的定义如下：

x[i][j]=0：表示陈列室 i,j 房间里未放置机器人。

X[i][j]=1: 表示陈列室 i,j 房间里已放置机器人。

Y[i][j]=0: 表示陈列室 i,j 房间未被监视。

Y[i][j]=1: 表示陈列室 i,j 房间已被监视。

robotNums: 表示 Node 对应的结点已放置的机器人数量

beenMonitored: 当前已经被监视的房间数

m:表示要监视的房间的行数

n:表示要监视的房间的列数

curNode:为要入堆的结点,preNode 为堆中的结点

则目标函数为：curNode.beenMonitored =m*n 且 curNode.robotNums 值最小，也就是所有的陈列室均被监视，且放置的机器人数量最小。

约束条件：

（A） if(curNode.i<m && curNode.y[curNode.i+1][curNode.j]==0)
curNode.x[curNode.i+1][curNode.j]=1;

（B） if(curNode.j<n && curNode.y[curNode.i][curNode.j+1]==0 ||
y[curNode.i][curNode.j+2]==0) curNode.x[i][j+1]=1;

（C） if(curNode.y[curNode.i+1][curNode.j]==0 &&
curNode.y[curNode.i][curNode.j+1]==0))
curNode.x[curNode.i][curNode.j]=1;

满足其中一个条件的结点可加入活结点表中。

限界函数：

（A） curNode.y!=preNode.y;

（B） 当 curNode.y=preNode.y 时，curNode. robotNums<preNode.
robotNums;

满足上述一个条件的结点可加入到活结点表中。

基于上述条件构建小顶堆，安放的机器人数量少的结点优先处理，当机器人数量相同时，监视的陈列室数量越多的结点优先处理。

当然需要注意的是如果使用 **priority_queue** 来定义堆，使用限界函数并不能提升算法运算的效率。因为该限界函数需要判断待插入堆中的结点所监控的陈列室的数量是否与堆中结点所监控的陈列室数量相同，在相同的情况下需要看监控的陈列室是否完全一致，这样就需要遍历堆中的所有结点。而 priority_queue 并不直接提供相关的函数，需要采用额外的方式对堆进行遍历，这样反而会增加运算的时间。但如果自己构造堆的存储结构，并对堆中的节点进行遍历，则采用上述限界函数会在一定程度上提升算法的运行效率。

（2）从题目的要求来看，在博物馆中每一次放置巡查机器人时都可以放置在任意一个陈列室中，一旦该陈列室已经安防巡查机器人，则下一次不应再安放。按照前面的定义，将数组 x[i][j]表示[i,j]位置陈列室，x[i][j]=1 表示在[i,j]位置的陈列室已经安放了巡查机器人，则解空间树每一层的可选结点为数组中为 0 的元素对应的陈列室。其解空间树是一个排序树。如图 8-4 所示。

（3）在样例输入情况下，基于（1）的目标函数，约束条件以及限界函数，采用堆的存储结构来存储活结点和扩展结点，其堆结点的定义采用（1）中的定义。堆中元素的变化过程，也就是在样例输入情况下问题的求解过程如图 8-5 所示。其中方框中第一行的元素对应为 i,j, beenMonitored, robotNums,

其后的 5 行，每行有 4 个元素，表示 x,y 矩阵的合并值。也就是将 x，y 矩阵合并显示在一个矩阵中，其中的 0 表示未监视的房间的位置，*和 1 则表示该房间已被监视。1 表示放置机器人的位置。圆形结点中的值分别表示加入堆中的顺序，机器人放置的行列位置。

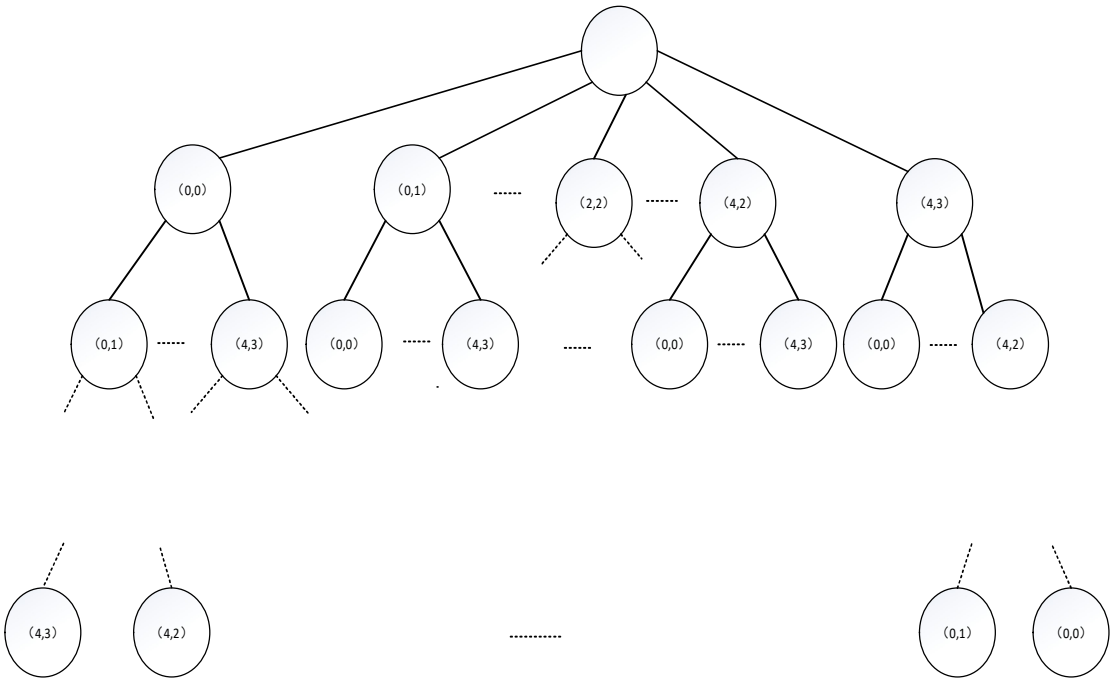
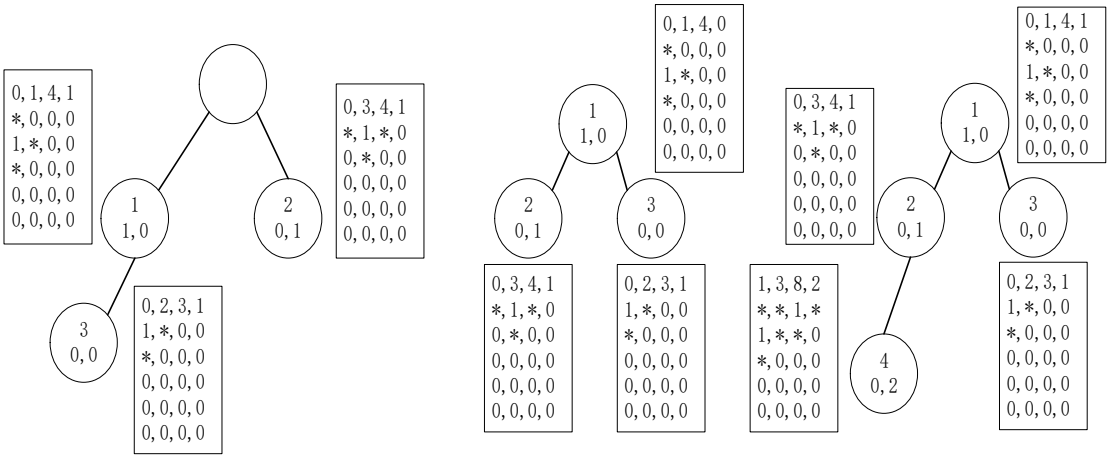
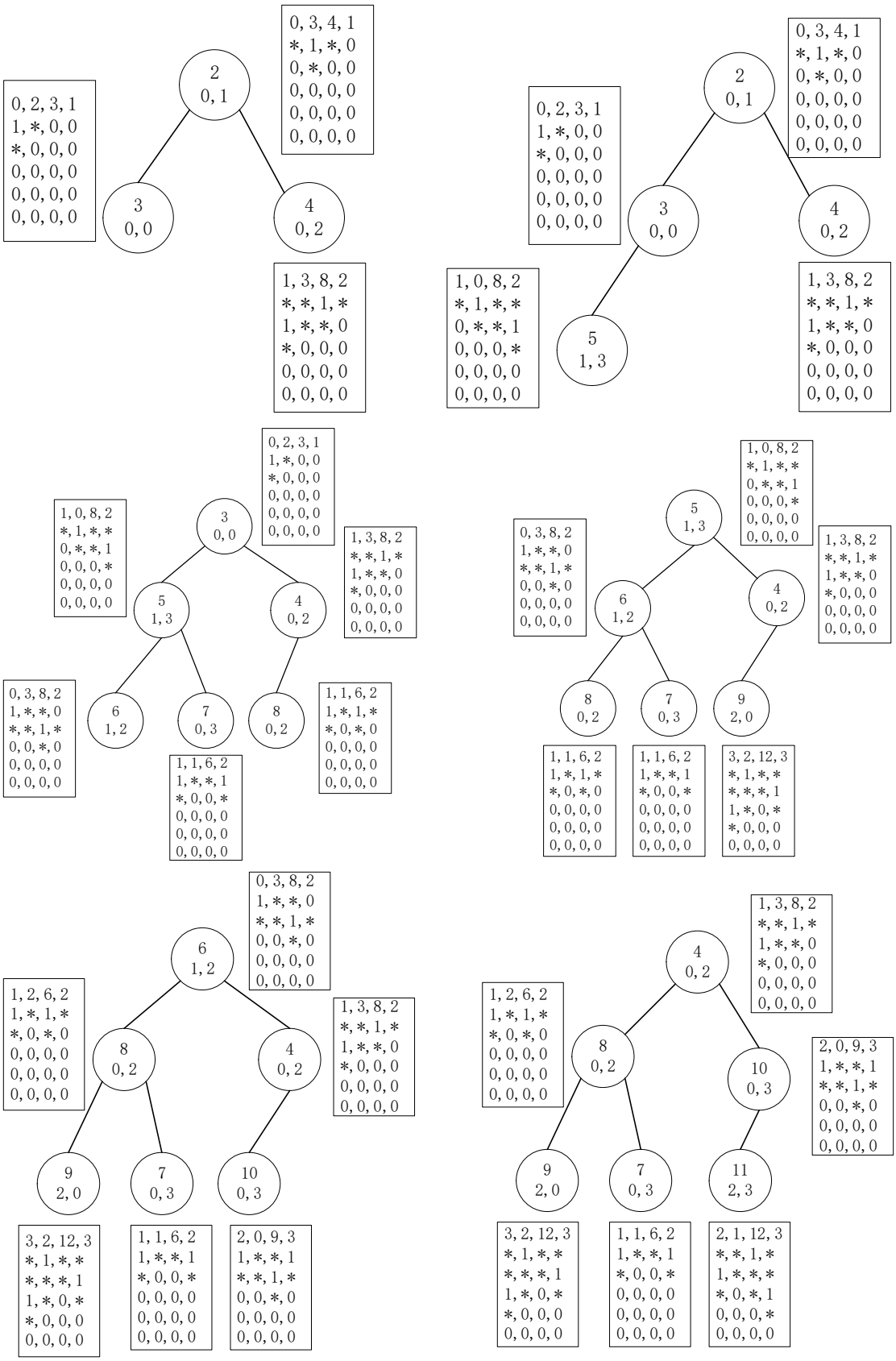
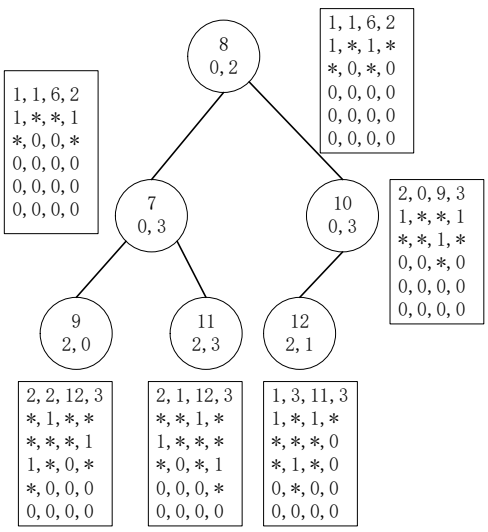
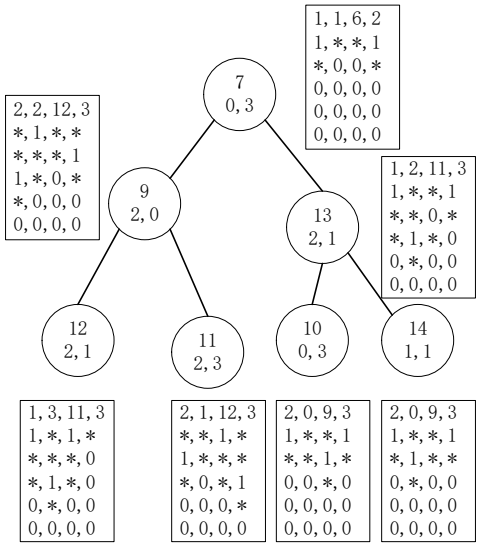


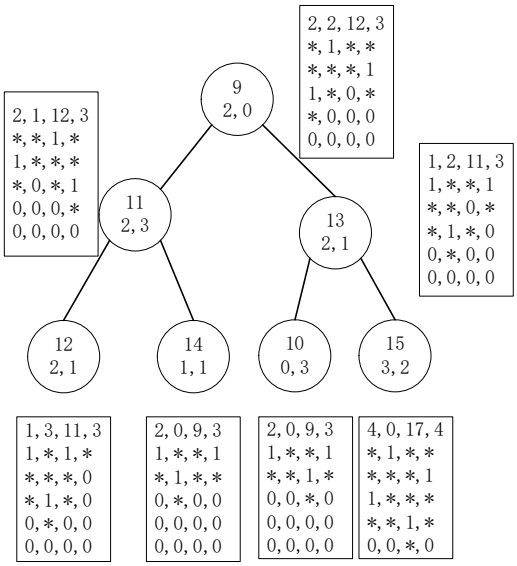
图 8-4 求解博物馆问题的解空间树

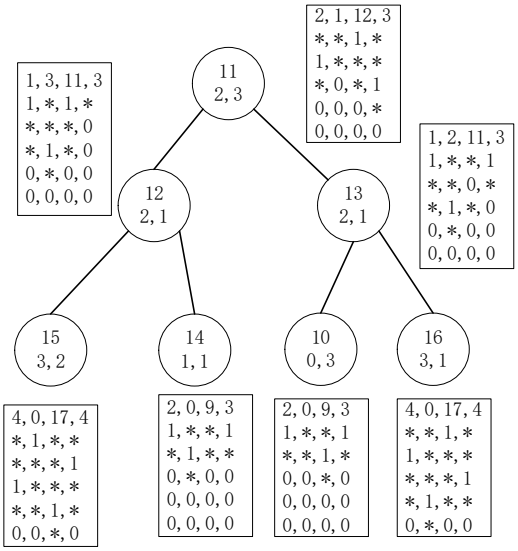


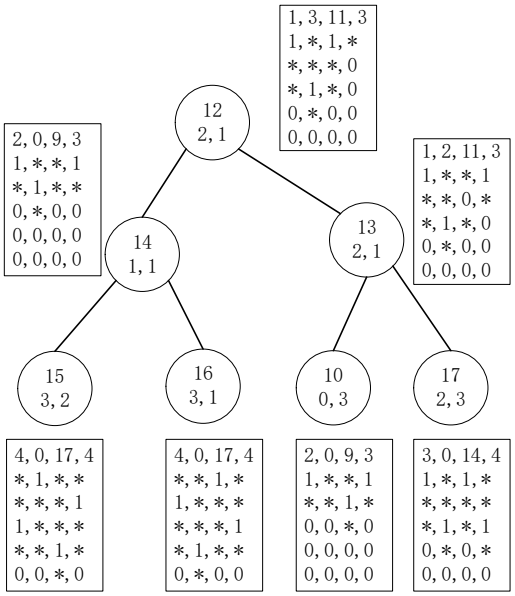

2
0, 1
0, 3, 4, 1
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0
0, 0, 0, 0
0, 0, 0, 0
0, 2, 3, 1
1, *, 0, 0
*, 0, 0, 0
0, 0, 0, 0
0, 0, 0, 0
0, 0, 0, 0
3
0, 0
4
0, 2
1, 3, 8, 2
*, *, 1, *
1, *, *, 0
*, 0, 0, 0
0, 0, 0, 0
0, 0, 0, 0
2
0, 1
0, 3, 4, 1
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0
0, 0, 0, 0
0, 2, 3, 1
1, *, 0, 0
*, 0, 0, 0
0, 0, 0, 0
0, 0, 0, 0
3
0, 0
4
0, 2
1, 0, 8, 2
*, 1, *, *
0, *, *, 1
0, 0, 0, *
0, 0, 0, 0
5
1, 3
1, 3, 8, 2
*, *, 1, *
1, *, *, 0
*, 0, 0, 0
0, 0, 0, 0
3
0, 0
0, 2, 3, 1
1, *, 0, 0
*, 0, 0, 0
0, 0, 0, 0
0, 0, 0, 0
1, 0, 8, 2
*, 1, *, *
0, *, *, 1
0, 0, 0, *
0, 0, 0, 0
0, 0, 0, 0
5
1, 3
4
0, 2
1, 3, 8, 2
*, *, 1, *
1, *, *, 0
*, 0, 0, 0
0, 0, 0, 0
0, 0, 0, 0
6
1, 2
7
0, 3
8
0, 2
0, 3, 8, 2
1, *, *, 0
*, *, 1, *
0, 0, *, 0
0, 0, 0, 0
0, 0, 0, 0
1, 1, 6, 2
1, *, *, 1
*, 0, 0, *
0, 0, 0, 0
0, 0, 0, 0
0, 0, 0, 0
1, 1, 6, 2
1, *, 1, *
*, 0, *, 0
0, 0, 0, 0
0, 0, 0, 0
5
1, 3
1, 0, 8, 2
*, 1, *, *
0, *, *, 1
0, 0, 0, *
0, 0, 0, 0
0, 0, 0, 0
0, 3, 8, 2
1, *, *, 0
*, *, 1, *
0, 0, *, 0
0, 0, 0, 0
0, 0, 0, 0
1, 3, 8, 2
*, *, 1, *
1, *, *, 0
*, 0, 0, 0
0, 0, 0, 0
0, 0, 0, 0
6
1, 2
4
0, 2
8
0, 2
7
0, 3
9
2, 0
1, 1, 6, 2
1, *, 1, *
*, 0, *, 0
0, 0, 0, 0
0, 0, 0, 0
0, 0, 0, 0
1, 1, 6, 2
1, *, *, 1
*, 0, 0, *
0, 0, 0, 0
0, 0, 0, 0
0, 0, 0, 0
3, 2, 12, 3
*, 1, *, *
*, *, *, 1
1, *, 0, *
*, 0, 0, 0
0, 0, 0, 0
6
1, 2
0, 3, 8, 2
1, *, *, 0
*, *, 1, *
0, 0, *, 0
0, 0, 0, 0
0, 0, 0, 0
1, 2, 6, 2
1, *, 1, *
*, 0, *, 0
0, 0, 0, 0
0, 0, 0, 0
0, 0, 0, 0
1, 3, 8, 2
*, *, 1, *
1, *, *, 0
*, 0, 0, 0
0, 0, 0, 0
0, 0, 0, 0
8
0, 2
4
0, 2
9
2, 0
7
0, 3
10
0, 3
3, 2, 12, 3
*, 1, *, *
*, *, *, 1
1, *, 0, *
*, 0, 0, 0
0, 0, 0, 0
1, 1, 6, 2
1, *, *, 1
*, 0, 0, *
0, 0, 0, 0
0, 0, 0, 0
2, 0, 9, 3
1, *, *, 1
*, *, 1, *
0, 0, *, 0
0, 0, 0, 0
0, 0, 0, 0
4
0, 2
1, 3, 8, 2
*, *, 1, *
1, *, *, 0
*, 0, 0, 0
0, 0, 0, 0
0, 0, 0, 0
1, 2, 6, 2
1, *, 1, *
*, 0, *, 0
0, 0, 0, 0
0, 0, 0, 0
0, 0, 0, 0
8
0, 2
10
0, 3
2, 0, 9, 3
1, *, *, 1
*, *, 1, *
0, 0, *, 0
0, 0, 0, 0
0, 0, 0, 0
9
2, 0
7
0, 3
11
2, 3
3, 2, 12, 3
*, 1, *, *
*, *, *, 1
1, *, 0, *
*, 0, 0, 0
0, 0, 0, 0
1, 1, 6, 2
1, *, *, 1
*, 0, 0, *
0, 0, 0, 0
0, 0, 0, 0
2, 1, 12, 3
*, *, 1, *
1, *, *, *
*, 0, *, 1
0, 0, 0, *
0, 0, 0, 0

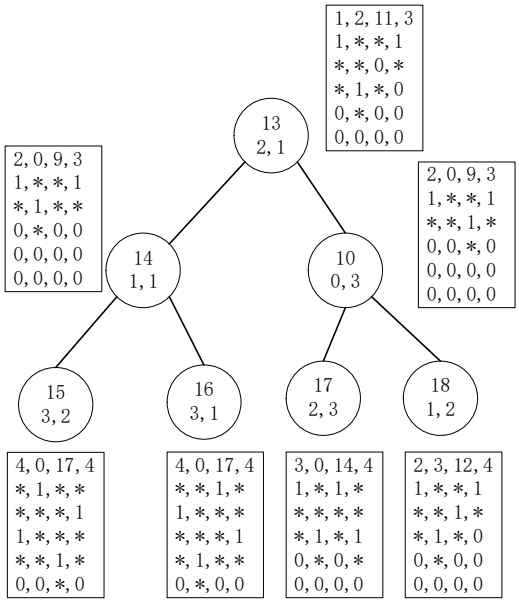

8
0, 2
1, 1, 6, 2
1, *, 1, *
*, 0, *, 0
0, 0, 0, 0
0, 0, 0, 0
1, 1, 6, 2
1, *, *, 1
*, 0, 0, *
0, 0, 0, 0
0, 0, 0, 0
7
0, 3
10
0, 3
2, 0, 9, 3
1, *, *, 1
*, *, 1, *
0, 0, *, 0
0, 0, 0, 0
9
2, 0
11
2, 3
12
2, 1
2, 2, 12, 3
*, 1, *, *
*, *, *, 1
1, *, 0, *
*, 0, 0, 0
0, 0, 0, 0
2, 1, 12, 3
*, *, 1, *
1, *, *, *
*, 0, *, 1
0, 0, 0, *
0, 0, 0, 0
1, 3, 11, 3
1, *, 1, *
*, *, *, 0
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0

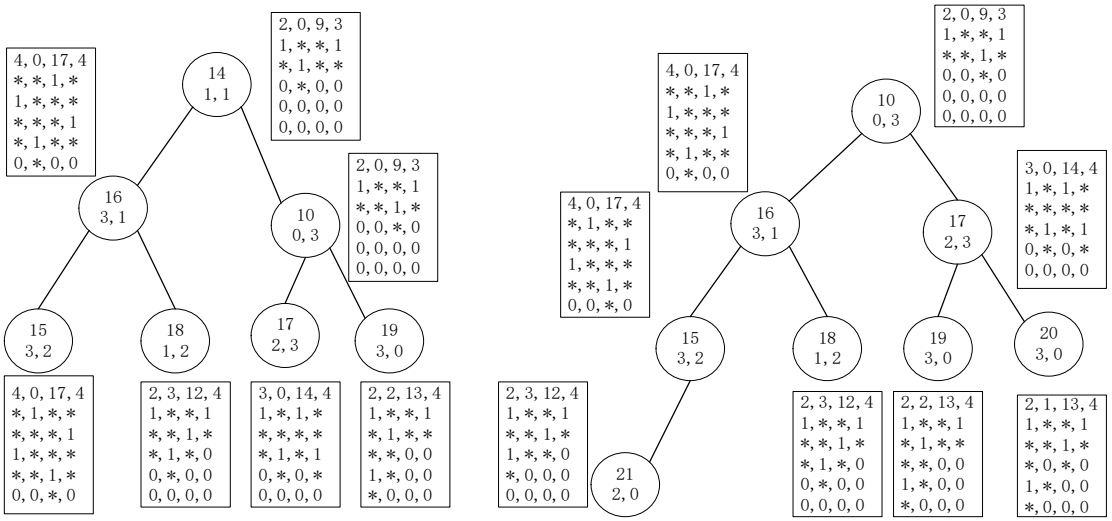

7
0, 3
1, 1, 6, 2
1, *, *, 1
*, 0, 0, *
0, 0, 0, 0
0, 0, 0, 0
2, 2, 12, 3
*, 1, *, *
*, *, *, 1
1, *, 0, *
*, 0, 0, 0
0, 0, 0, 0
9
2, 0
13
2, 1
1, 2, 11, 3
1, *, *, 1
*, *, 0, *
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0
12
2, 1
11
2, 3
10
0, 3
14
1, 1
1, 3, 11, 3
1, *, 1, *
*, *, *, 0
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0
2, 1, 12, 3
*, *, 1, *
1, *, *, *
*, 0, *, 1
0, 0, 0, *
0, 0, 0, 0
2, 0, 9, 3
1, *, *, 1
*, *, 1, *
0, 0, *, 0
0, 0, 0, 0
2, 0, 9, 3
1, *, *, 1
*, 1, *, *
0, *, 0, 0
0, 0, 0, 0


9
2, 0
2, 2, 12, 3
*, 1, *, *
*, *, *, 1
1, *, 0, *
*, 0, 0, 0
0, 0, 0, 0
2, 1, 12, 3
*, *, 1, *
1, *, *, *
*, 0, *, 1
0, 0, 0, *
0, 0, 0, 0
11
2, 3
13
2, 1
1, 2, 11, 3
1, *, *, 1
*, *, 0, *
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0
12
2, 1
14
1, 1
10
0, 3
15
3, 2
1, 3, 11, 3
1, *, 1, *
*, *, *, 0
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0
2, 0, 9, 3
1, *, *, 1
*, 1, *, *
0, *, 0, 0
0, 0, 0, 0
2, 0, 9, 3
1, *, *, 1
*, *, 1, *
0, 0, *, 0
0, 0, 0, 0
4, 0, 17, 4
*, 1, *, *
*, *, *, 1
1, *, *, *
0, 0, *, 0

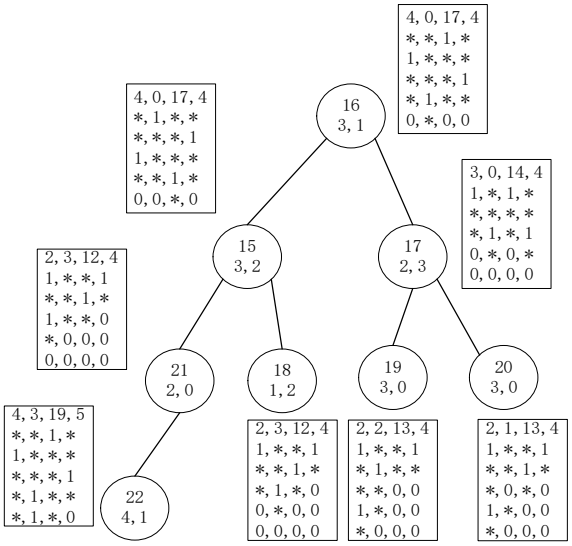

11
2, 3
2, 1, 12, 3
*, *, 1, *
1, *, *, *
*, 0, *, 1
0, 0, 0, *
0, 0, 0, 0
1, 3, 11, 3
1, *, 1, *
*, *, *, 0
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0
12
2, 1
13
2, 1
1, 2, 11, 3
1, *, *, 1
*, *, 0, *
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0
15
3, 2
14
1, 1
10
0, 3
16
3, 1
4, 0, 17, 4
*, 1, *, *
*, *, *, 1
1, *, *, *
*, *, 1, *
0, 0, *, 0
2, 0, 9, 3
1, *, *, 1
*, 1, *, *
0, *, 0, 0
0, 0, 0, 0
2, 0, 9, 3
1, *, *, 1
*, *, 1, *
0, 0, *, 0
0, 0, 0, 0
4, 0, 17, 4
*, *, 1, *
1, *, *, *
*, *, *, 1
*, 1, *, 0
0, *, 0, 0

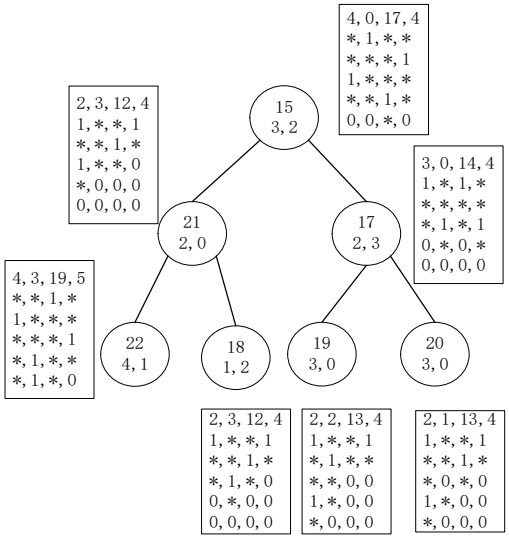

12
2, 1
1, 3, 11, 3
1, *, 1, *
*, *, *, 0
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0
2, 0, 9, 3
1, *, *, 1
*, 1, *, *
0, *, 0, 0
0, 0, 0, 0
14
1, 1
13
2, 1
1, 2, 11, 3
1, *, *, 1
*, *, 0, *
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0
15
3, 2
16
3, 1
10
0, 3
17
2, 3
4, 0, 17, 4
*, 1, *, *
*, *, *, 1
1, *, *, *
*, *, 1, *
0, 0, *, 0
4, 0, 17, 4
*, *, 1, *
1, *, *, *
*, *, *, 1
*, 1, *, *
0, *, 0, 0
2, 0, 9, 3
1, *, *, 1
*, *, 1, *
0, 0, *, 0
0, 0, 0, 0
3, 0, 14, 4
1, *, 1, *
*, *, *, *
*, 1, *, 1
0, *, 0, *
0, 0, 0, 0

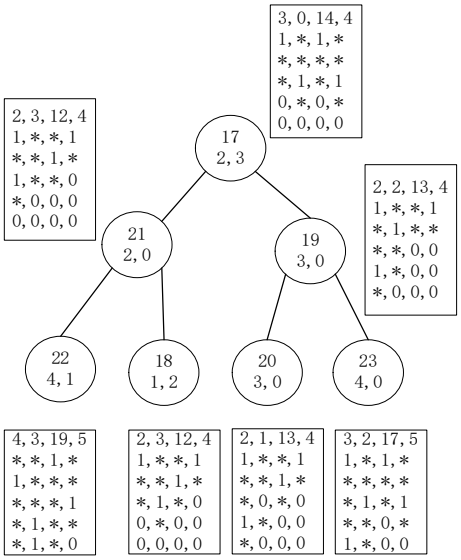

13
2, 1
1, 2, 11, 3
1, *, *, 1
*, *, 0, *
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0
2, 0, 9, 3
1, *, *, 1
*, 1, *, *
0, *, 0, 0
0, 0, 0, 0
0, 0, 0, 0
2, 0, 9, 3
1, *, *, 1
*, *, 1, *
0, 0, *, 0
0, 0, 0, 0
0, 0, 0, 0
14
1, 1
10
0, 3
15
3, 2
16
3, 1
17
2, 3
18
1, 2
4, 0, 17, 4
*, 1, *, *
*, *, *, 1
1, *, *, *
*, *, 1, *
0, 0, *, 0
4, 0, 17, 4
*, *, 1, *
1, *, *, *
*, *, *, 1
*, 1, *, *
0, *, 0, 0
3, 0, 14, 4
1, *, 1, *
*, *, *, *
*, 1, *, 1
0, *, 0, *
0, 0, 0, 0
2, 3, 12, 4
1, *, *, 1
*, *, 1, *
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0

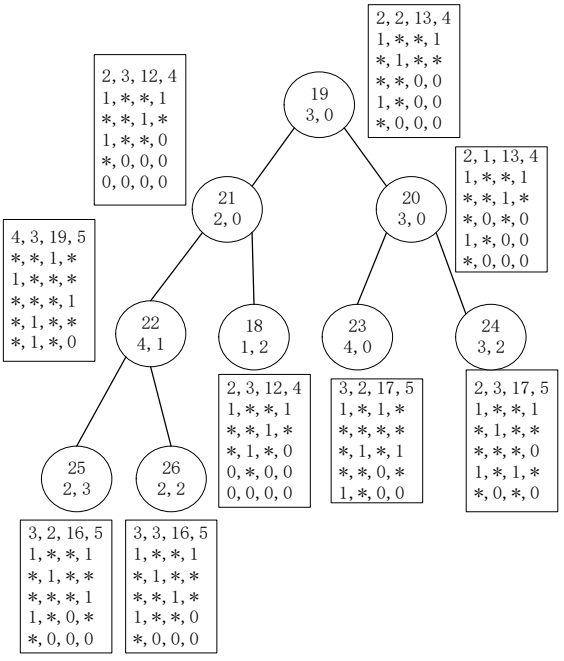

14
1, 1
16
3, 1
10
0, 3
15
3, 2
18
1, 2
17
2, 3
19
3, 0
2, 0, 9, 3
1, *, *, 1
*, 1, *, *
0, *, 0, 0
0, 0, 0, 0
0, 0, 0, 0
4, 0, 17, 4
*, *, 1, *
1, *, *, *
*, *, *, 1
*, 1, *, *
0, *, 0, 0
2, 3, 12, 4
1, *, *, 1
*, *, 1, *
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0
3, 0, 14, 4
1, *, 1, *
*, *, *, *
*, 1, *, 1
0, *, 0, *
0, 0, 0, 0
2, 2, 13, 4
1, *, *, 1
*, 1, *, *
*, *, 0, 0
1, *, 0, 0
*, 0, 0, 0

10
0, 3
16
3, 1
17
2, 3
15
3, 2
18
1, 2
19
3, 0
20
3, 0
21
2, 0
2, 0, 9, 3
1, *, *, 1
*, *, 1, *
0, 0, *, 0
0, 0, 0, 0
0, 0, 0, 0
4, 0, 17, 4
*, *, 1, *
1, *, *, *
*, *, *, 1
*, 1, *, *
0, 0, *, 0
4, 0, 17, 4
*, 1, *, *
*, *, *, 1
1, *, *, *
*, *, 1, *
0, 0, *, 0
3, 0, 14, 4
1, *, 1, *
*, *, *, *
*, 1, *, 1
0, *, 0, *
0, 0, 0, 0
2, 3, 12, 4
1, *, *, 1
*, *, 1, *
1, *, *, 0
*, 0, 0, 0
0, 0, 0, 0
2, 3, 12, 4
1, *, *, 1
*, *, 1, *
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0
2, 2, 13, 4
1, *, *, 1
*, 1, *, *
*, *, 0, 0
1, *, 0, 0
*, 0, 0, 0
2, 1, 13, 4
1, *, *, 1
*, *, 1, *
*, 0, *, 0
1, *, 0, 0
*, 0, 0, 0


16
3, 1
15
3, 2
17
2, 3
21
2, 0
18
1, 2
19
3, 0
20
3, 0
22
4, 1
4, 0, 17, 4
*, *, 1, *
1, *, *, *
*, *, *, 1
*, 1, *, *
0, *, 0, 0
4, 0, 17, 4
*, 1, *, *
*, *, *, 1
1, *, *, *
*, *, 1, *
0, 0, *, 0
3, 0, 14, 4
1, *, 1, *
*, *, *, *
*, 1, *, 1
0, *, 0, *
0, 0, 0, 0
2, 3, 12, 4
1, *, *, 1
*, *, 1, *
1, *, *, 0
*, 0, 0, 0
0, 0, 0, 0
4, 3, 19, 5
*, *, 1, *
1, *, *, *
*, *, *, 1
*, 1, *, *
*, 1, *, 0
2, 3, 12, 4
1, *, *, 1
*, *, 1, *
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0
2, 2, 13, 4
1, *, *, 1
*, 1, *, *
*, *, 0, 0
1, *, 0, 0
*, 0, 0, 0
2, 1, 13, 4
1, *, *, 1
*, *, 1, *
*, 0, *, 0
1, *, 0, 0
*, 0, 0, 0


15
3, 2
21
2, 0
17
2, 3
22
4, 1
18
1, 2
19
3, 0
20
3, 0
4, 0, 17, 4
*, 1, *, *
*, *, *, 1
1, *, *, *
*, *, 1, *
0, 0, *, 0
2, 3, 12, 4
1, *, *, 1
*, *, 1, *
1, *, *, 0
*, 0, 0, 0
0, 0, 0, 0
3, 0, 14, 4
1, *, 1, *
*, *, *, *
*, 1, *, 1
0, *, 0, *
0, 0, 0, 0
4, 3, 19, 5
*, *, 1, *
1, *, *, *
*, *, *, 1
*, 1, *, *
*, 1, *, 0
2, 3, 12, 4
1, *, *, 1
*, *, 1, *
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0
2, 2, 13, 4
1, *, *, 1
*, 1, *, *
*, *, 0, 0
1, *, 0, 0
*, 0, 0, 0
2, 1, 13, 4
1, *, *, 1
*, *, 1, *
*, 0, *, 0
1, *, 0, 0
*, 0, 0, 0


17
2, 3
21
2, 0
19
3, 0
22
4, 1
18
1, 2
20
3, 0
23
4, 0
3, 0, 14, 4
1, *, 1, *
*, *, *, *
*, 1, *, 1
0, *, 0, *
0, 0, 0, 0
2, 3, 12, 4
1, *, *, 1
*, *, 1, *
1, *, *, 0
*, 0, 0, 0
0, 0, 0, 0
2, 2, 13, 4
1, *, *, 1
*, 1, *, *
*, *, 0, 0
1, *, 0, 0
*, 0, 0, 0
4, 3, 19, 5
*, *, 1, *
1, *, *, *
*, *, *, 1
*, 1, *, *
*, 1, *, 0
2, 3, 12, 4
1, *, *, 1
*, *, 1, *
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0
2, 1, 13, 4
1, *, *, 1
*, *, 1, *
*, 0, *, 0
1, *, 0, 0
*, 0, 0, 0
3, 2, 17, 5
1, *, 1, *
*, *, *, *
*, 1, *, 1
*, *, 0, *
1, *, 0, 0


19
3, 0
21
2, 0
20
3, 0
22
4, 1
18
1, 2
23
4, 0
24
3, 2
25
2, 3
26
2, 2
2, 2, 13, 4
1, *, *, 1
*, 1, *, *
*, *, 0, 0
1, *, 0, 0
*, 0, 0, 0
2, 3, 12, 4
1, *, *, 1
*, *, 1, *
1, *, *, 0
*, 0, 0, 0
0, 0, 0, 0
2, 1, 13, 4
1, *, *, 1
*, *, 1, *
*, 0, *, 0
1, *, 0, 0
*, 0, 0, 0
4, 3, 19, 5
*, *, 1, *
1, *, *, *
*, *, *, 1
*, 1, *, *
*, 1, *, 0
2, 3, 12, 4
1, *, *, 1
*, *, 1, *
*, 1, *, 0
0, *, 0, 0
0, 0, 0, 0
3, 2, 17, 5
1, *, 1, *
*, *, *, *
*, 1, *, 1
*, *, 0, *
1, *, 1, *
2, 3, 17, 5
1, *, *, 1
*, 1, *, *
*, *, *, 0
1, *, 1, *
*, 0, *, 0
3, 2, 16, 5
1, *, *, 1
*, 1, *, *
*, *, *, 1
1, *, 0, *
*, 0, 0, 0
3, 3, 16, 5
1, *, *, 1
*, 1, *, *
*, *, 1, *
1, *, *, 0
*, 0, 0, 0

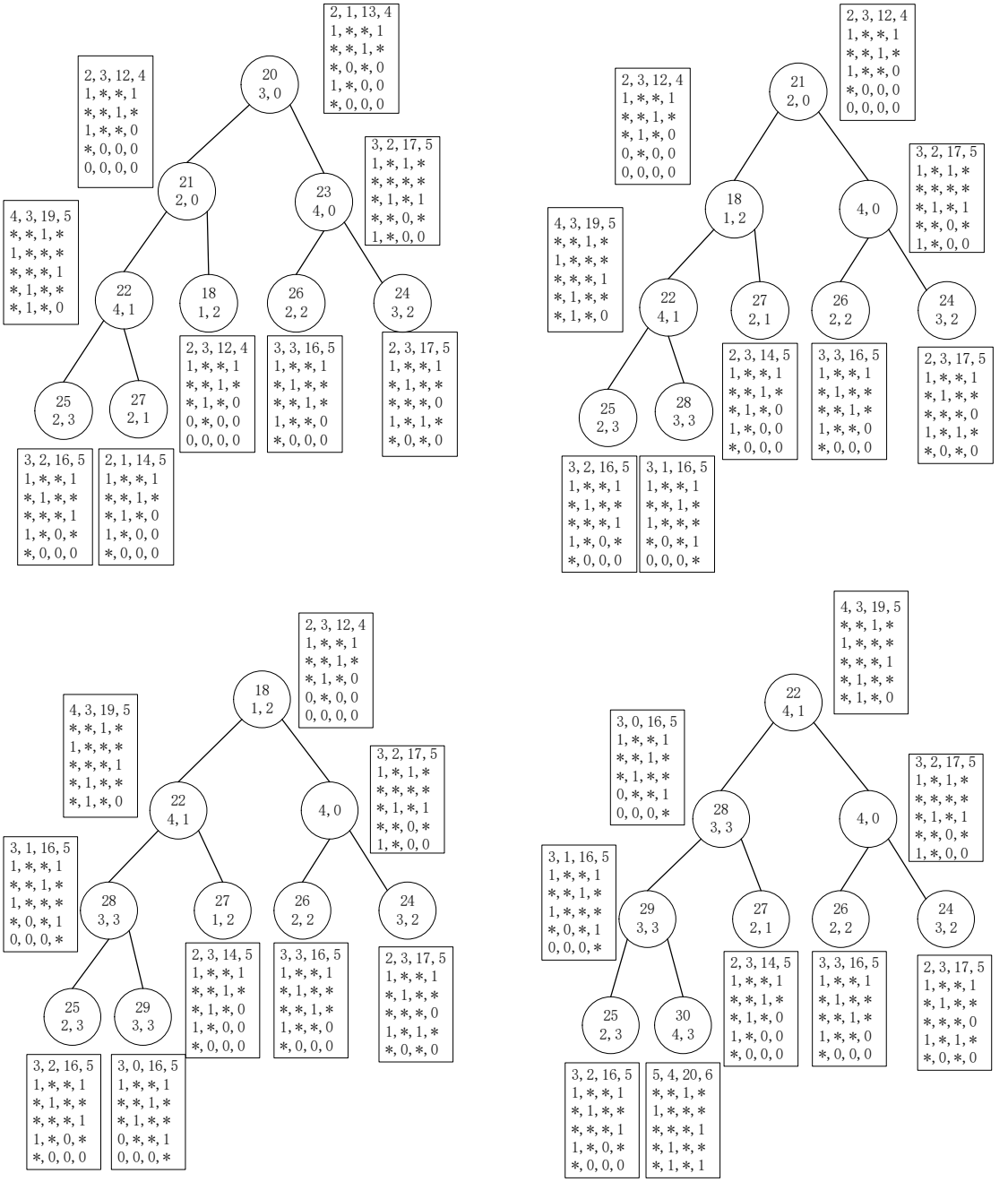


图 8-5 样例输入下堆的变化过程

需要注意的是上面只罗列了部分中间过程，且上述算法考虑了限界函数，但前面说过如果不考虑限界函数，则其求解过程中结点的值将会有所不同。

（4） 基于前面的求解过程，可以得到在样例输入下的搜索空间树，如图 8-6 所示。该搜索空间树中结点的定义与前面的定义相同。

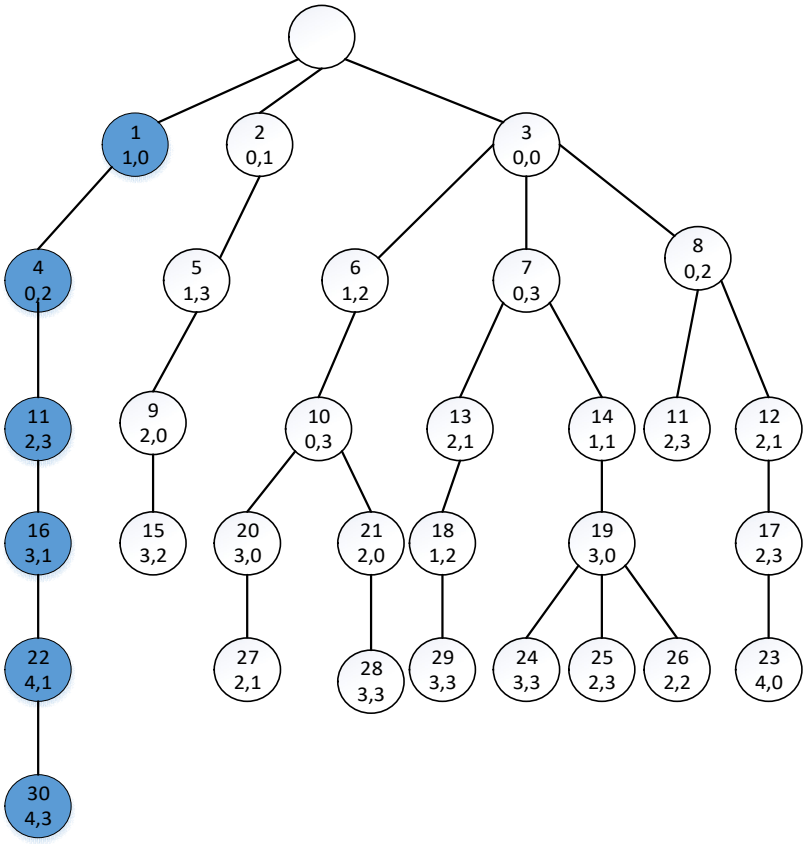


图 8-6 样例输入时的搜索空间树

（5）基于上述算法，在不考虑限界函数的情况下，可设计如下的程序求解该问题。

```
1   #include<iostream>
2   #include<fstream>
3   #include<queue>
4   using namespace std;
5   class Solve;
6   class Node          //Node节点，用来存放搜索树的节点
7   {
8       friend class Solve;
9     private:
10      int i;              //当前要放置的新位置 横坐标i和纵坐标j
11      int j;
12      int robotNums;      //当前节点已经放置的警卫机器人数目
13      int beenMonitored;  //当前已经被监视的房间数
14      int** x;   //当前放置警卫的地方    0表示没有，1表示放置了
15      int** y;   //当前已经被监视的地方    0表示没有，1表示已经监视了
```

```
16      int m;      //行
17      int n;      //列
18
19  public:
20      Node();                 //构造函数
21      Node(int m, int n);     //构造函数,m、n是行列数
22      Node(const Node& a);
        //重载<，用于优先队列的使用
23      friend bool operator<(const Node& a, const Node& b);
24      Node& operator=(const Node& a){
25          if (x || y){
26              for (int i = 0; i < m + 2; ++i){
27                  if (x)
28                      delete[] x[i];
29                  if (y)
30                      delete[] y[i];
31              }
32              delete[] x;
33              delete[] y;
34              x = NULL;
35              y = NULL;
36          }
37          i = a.i;
38          j = a.j;
39          robotNums = a.robotNums;
40          beenMonitored = a.beenMonitored;
41          m = a.m;
42          n = a.n;
43          x = new int* [m + 2];
44          y = new int* [m + 2];
45          for (int i = 0; i < m + 2; ++i){
46              x[i] = new int[n + 2];
47              y[i] = new int[n + 2];
48              for (int j = 0; j < n + 2; ++j){
```

```
49                  x[i][j] = a.x[i][j];
50                  y[i][j] = a.y[i][j];
51              }
52          }
53          return *this;
54      }
55      ~Node();   //析构函数要重载，避免内存泄露
56  };
57  Node::Node()
58  {
59      i = 0;
60      j = 0;
61      m = 0;
62      n = 0;
63      robotNums = 0;
64      beenMonitored = 0;
65      x = NULL;
66      y = NULL;
67  }
68
69  Node::Node(int m, int n)
70  {
71      x = new int* [m + 2];
72      y = new int* [m + 2];
73      this->m = m;
74      this->n = n;
75      for (int i = 0; i < m + 2; ++i){
76          x[i] = new int[n + 2];
77          y[i] = new int[n + 2];
78      }
79      for (int i = 0; i < m + 2; ++i){
80          for (int j = 0; j < n + 2; ++j){
81              x[i][j] = 0;
82              y[i][j] = 0;
```

```
83          }
84      }
85      for (int i = 0; i < m + 2; ++i){
86          y[i][0] = 1;
87          y[i][n + 1] = 1;
88      }
89      for (int i = 0; i < n + 2; ++i){
90          y[0][i] = 1;
91          y[m + 1][i] = 1;
92      }
93      i = 1;
94      j = 1;
95      robotNums = 0;
96      beenMonitored = 0;
97  }
98
99  Node::Node(const Node& a)
100 {
101     i = a.i;
102     j = a.j;
103     robotNums = a.robotNums;
104     beenMonitored = a.beenMonitored;
105     m = a.m;
106     n = a.n;
107     x = new int* [m + 2];
108     y = new int* [m + 2];
109     for (int i = 0; i < m + 2; ++i){
110         x[i] = new int[n + 2];
111         y[i] = new int[n + 2];
112         for (int j = 0; j < n + 2; ++j){
113             x[i][j] = a.x[i][j];
114             y[i][j] = a.y[i][j];
115         }
116     }
```

```
117 }
118 bool operator<(const Node& a, const Node& b)
119 {
120     if (a.robotNums == b.robotNums)
121         return a.beenMonitored < b.beenMonitored;
122     else
123         return a.robotNums > b.robotNums;
124    }
125
126 Node::~Node()
127 {
128     for (int i = 0; i < m + 2; ++i){
129         delete[] x[i];
130         delete[] y[i];
131     }
132     delete[] x;
133     delete[] y;
134 }
135 class Solve     //解决问题的类
136 {
137   private:
138     priority_queue<Node> heap;    //优先队列heap
139     int ans;                      //答案所需要的警卫数目
140     int m;                        //行列数
141     int n;
142     int** result;                 //答案警卫的排列顺序
143   public:
144     Solve();
145     Solve(int m, int n);
146     void run(ofstream& fcout);    //进行整个计算+输出
147     void get_min();               //运用分支限界法，寻找最小值
148     void print(ofstream& fcout);  //打印警卫位置和数目
149     void copy(int** x, int** y);  //将一个二维数组赋值给另一个
        //生成子节点，同时将其添加到heap中
```

```
150     void change(Node& tmp, int i, int j);
151 };

152 Solve::Solve()

153 {
154     ans = INT_MAX;
155     m = 0;
156     n = 0;
157     result = NULL;
158 }

159 Solve::Solve(int m, int n)

160 {
161     ans = INT_MAX;
162     this->m = m;
163     this->n = n;
164     result = new int* [m + 2];
165     for (int i = 0; i < m + 2; ++i) {
166         result[i] = new int[n + 2];
167     }
168 }
169
170 void Solve::copy(int** x, int** y)
171 {
172     for (int i = 0; i < m + 2; ++i)
173         for (int j = 0; j < n + 2; ++j)
174             x[i][j] = y[i][j];
175  }
176
177 void Solve::change(Node& tmp, int i, int j)
178 {
179     int d[5][2] = { {0, 0}, {1, 0}, {-1, 0}, {0, 1}, {0, -1} };
180     Node N = tmp;
```

```
181     N.robotNums = tmp.robotNums + 1;
182     N.x[i][j] = 1;                      //放置新的警卫
183     for (int k = 0; k < 5; ++k) {       //更新监视房间数

184         int p = i + d[k][0], q = j + d[k][1];   //上下左右五个全部遍历

185         ++N.y[p][q];
186         if (N.y[p][q] == 1)          //说明是新添加的被监视到的
187            ++N.beenMonitored;        //被添加数加1
188     }
189     while (!(N.y[N.i][N.j] == 0 || N.j > n)) {  //更新待放置的位置
190         ++N.i;
191         if (N.i > m) {
192             ++N.j;
193             N.i = 1;
194         }
195     }
196     heap.push(N);
197 }
198 void Solve::run(ofstream& fcout)
199 {
200     if (m == 1 && n == 1) {   //行列都为1 的简单情况
                cout << 1 << endl;
201         cout << 1 << endl;
202     }
203     get_min();
204     print(fcout);
205 }
206 void Solve::get_min()
207 {
208     Node tmp(m, n);
209     heap.push(tmp);   //push调用了复制构造函数，一定要定义
210     while (!heap.empty()) {
211         tmp = heap.top();    //调用赋值运算符
212         int i = tmp.i;
```

```
213         int j = tmp.j;
214         int robotNums = tmp.robotNums;
215         int beenMonitored = tmp.beenMonitored;
216         if (beenMonitored == m * n) {   //如果全部房间已经被监视，中止
217             ans = robotNums;
218             copy(result, tmp.x);
219             return;
220         }
221         else{
222             if (j < n && (tmp.y[i][j+1] == 0 || tmp.y[i][j+2] == 0))
223                 change(tmp, i, j+1);
224              if (i < m && tmp.y[i+1][j]==0)
225                 change(tmp, i+1, j);
226             if (tmp.y[i][j + 1] == 0 && tmp.y[i + 1][j] == 0)
227                 change(tmp, i, j);
228         }
229         heap.pop();
230     }
231 }
232
233 void Solve::print(ofstream& fcout)
234 {
235     fcout << ans << endl;
236     for (int i = 1; i < m + 1; ++i)
237     {
238         for (int j = 1; j < n + 1; ++j)
239             fcout << result[i][j] << “  “;
240         fcout << endl;
241     }
242 }
243 int main()
244 {
245     int m, n;
246     cin >> m >> n;
```

```
247     Solve solve(m, n);
248     ofstream fcout;
249     fcout.open("output.txt");
250     if (!fcout.is_open()){
251         cout << "文件 output.txt 未能打开" << endl;
252         return -1;
253     }
254     solve.run(fcout);
255     fcout.close();
256     return 0;
257 }
```

由于该程序未使用限界函数对算法进行优化，因此其执行过程中与前面的分析结果会略有不同。大家可根据上述程序分析算法执行过程中堆中元素的变化以及相应的搜索空间树。此外，该程序中数组元素对应的陈列室的行列起始值从1开始，而分析过程中其行列值从0开始。

实验 8.3 某公司有一个长方形的仓库，仓库内采用机器人来搬运货物，如图 8-7 所示。机器人在搬运货物前，需先移动到与货物临近的空白方格内（没有墙体或其它物体），然后推动货物朝机器人朝向货物的方向移动。机器人和货物在移动过程中不能穿越墙体，也不能移动到已被占用的方格。现希望机器人能够将某货物搬运到目的地，但希望货物移动的次数最少。货物从一个方格移动到另一个方格称为移动一次。

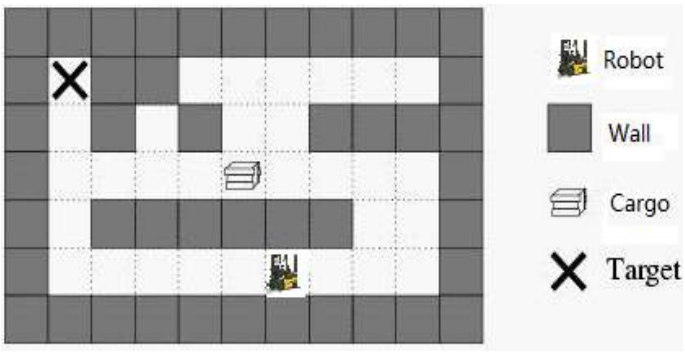


图 8-7 某公司仓库示意图

输入：

输入包含多个测试样例。每个测试样例的第一行包含两个整数 n,m,分别表示该长方形仓库的行值和列值。其后的 n 行，每行有 m 个字符，由#.STB 构成，其中#表示墙体占用的方格，.表示空白方格，S 表示机器人所占用的方格，T 表示货物应到达的位置，B 表示货物所在的位置。

输出：

每个测试样例输出一个整数，占 1 行，表示货物最少的移动次数。

样例输入：

```
7 11
###########
#T##.....#
#.#.#..####
#....B....#
#.#######..#
#.....S...#
###########
```

实验预习：

（1） 给出采用分支限界法求解该问题时的目标函数，约束条件以及限界函数。

（2） 给出采用分支限界法求解样例输入时的解空间树。

（3） 分析采用分支限界法求解样例输入时的求解过程，堆结点的定义，堆结点的值以及堆中元素的变化过程。

(4) 画出采用分支限界法求解样例输入时的搜索空间树。

(5) 编写相应的程序。

上机实验：

（6） 调试步骤（5）的程序，验证样例输入时程序执行过程中堆的变化过程是否与步骤（3）的分析结果一致。

（7） 通过程序的求解过程，分析影响算法时间效率的主要因素有哪些？

实验 8.4 由一系列的左斜杠（/）和右斜杠（\）组成的围墙将一个由 n×m 个方格构成的矩形分割成若干个区域。每个区域由若干个三角形构成，如图 8-8 所示。如果一个人从一个三角形区域经过其相邻的三角形区域一次最后再回到起始出发的三角形,则其走过的所有三角形区域构成一个回路。设每个三角形区域的长度为 1，求迷宫中长度最长的回路的长度。图 8-8 中最长回路的长度为 8。

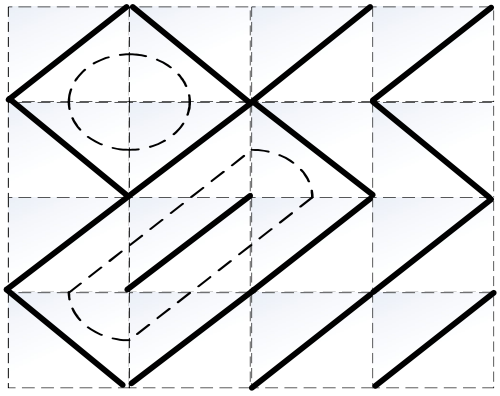


图 8-8 迷宫

输入：

输入包含若干组测试样例，每一组测试样例的第 1 行为两个整数 n,m，分别表示每行和每列有多少个方格。其后的 n 行，每行有 m 个斜杠（\或/），分别表示墙体的方向，所有的斜杠均在方格的对角线上，每个方格均有一个斜杠。

输出：

每个测试数据输出 1 行，包含一个整数，表示最长回路的长度。

样例输入：

```
4 4
/\//
\/\\
////
\///
```

样例输出：

```
8
```

实验预习：

（1） 给出采用分支限界法求解该问题时的目标函数，约束条件以及限界函数。

（2） 给出采用分支限界法求解样例输入时的解空间树。

（3） 分析采用分支限界法求解样例输入时的求解过程，堆结点的定义，堆结点的值以及堆中元素的变化过程。

(4) 画出采用分支限界法求解样例输入时的搜索空间树。

(5) 编写相应的程序。

上机实验：

（6） 调试步骤（5）的程序，验证样例输入时程序执行过程中堆的变化过程是否与步骤（3）的分析结果一致。

（7） 通过程序的求解过程，分析影响算法时间效率的主要因素有哪些？

实验总结：

本章通过具体实例介绍了分支限界法的分析求解过程，与其他算法的运行时间效率比较以及如何采用分支限界法解决实际应用问题。通过本实验，学生可以正确理解分支限界法的特点以及在实际应用中如何正确设计并使用分支限界法。